

METALNET: DEPENDENCY-FREE C++ CNN HEADER-ONLY LIBRARY

A PROJECT REPORT SUBMITTED
IN PARTIAL FULFILMENT OF THE REQUIREMENT
FOR THE AWARD OF THE DEGREE
OF

BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE & ENGINEERING



BY

Raj Kunwar Prabhat 22050440078

Mayan Kumar 22050440058

Sujeet Kumar 22050440097

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
CAMBRIDGE INSTITUTE OF TECHNOLOGY**

TATISILWAI, RANCHI – 835103

JHARKHAND, INDIA

2026

DECLARATION CERTIFICATE

This is to certify that the work presented in the project entitled “**MetalNet: Dependency-Free C++ CNN Header-Only Library**” in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering at Cambridge Institute of Technology, Tatisilwai, Ranchi is an authentic work carried out under my supervision and guidance.

To the best of my knowledge, the content of this project does not form a basis for the award of any previous Degree to anyone else.

Date: _____

PROF. DEEPAK KUMAR VERMA

Dept. of Computer Science and Engineering
Cambridge Institute of Technology
Tatisilwai, Ranchi – 835103

PROF. DEEPAK KUMAR VERMA
(Head of Department)

Dept. of Computer Science and Engineering
Cambridge Institute of Technology
Tatisilwai, Ranchi – 835103

CERTIFICATE OF APPROVAL

The foregoing project entitled “MetalNet: Dependency-Free C++ CNN Header-Only Library” is hereby approved as a creditable work on the topic and has been presented satisfactorily to warrant its acceptance as a prerequisite to the degree for which it has been submitted.

It is understood that by this approval, the undersigned does not necessarily endorse any conclusion drawn or opinion expressed therein, but approves the project for the purpose for which it is submitted.

(Internal Examiner)

(External Examiner)

HEAD OF DEPARTMENT

Dept. of Computer Science and Engineering
Cambridge Institute of Technology
Tatisilwai, Ranchi – 835103

ACKNOWLEDGEMENT

I take this opportunity to thank all who have contributed towards shaping this Final Year Project Report. I would like to express my sincere thanks to Prof. Deepak Kumar Verma (HOD, Department of Computer Science & Engineering) and to my guide Prof. Deepak Kumar Verma, whose help in the course of development of this manuscript is invaluable. As my supervisor, he/she has constantly encouraged me to remain focused on achieving my goal.

I would also like to thank the technical staff members of the Department who have been kind enough to advise and help in their respective roles. I have been fortunate to have a wonderful support structure among fellow students. My sincere thanks to everyone who has provided me with kind words, new ideas, useful criticism, or their valuable time. I am truly indebted. Last but not least, I would like to dedicate this work to my family for their love and understanding.

Raj Kunwar Prabhat
22050440078

ABSTRACT

Deep learning vision systems typically rely on large frameworks like PyTorch or TensorFlow, which carry significant dependency overhead and are designed primarily for GPUs. On CPU-only environments such as laptops or edge devices, these frameworks introduce unnecessary resource consumption. This project presents **MetalNet**, a high-performance Convolutional Neural Network (CNN) library written from scratch in pure C++20. MetalNet is fully **header-only** and **dependency-free**, requiring no installation, linking, or configuration, and is designed specifically to optimize CPU execution.

Despite its lightweight design, MetalNet achieves high performance by treating the CPU as a first-class citizen. It implements an **Implicit GEMM** convolution algorithm that avoids memory-heavy *im2col* buffering, and stores data in the cache-friendly **NHWC** layout. Innermost arithmetic operations are accelerated using hand-written **AVX2 and FMA** SIMD instructions, while loop tiling and parallelization via **OpenMP** optimize multi-core CPU scaling. Together, these optimizations yield a **57× speedup** in convolution operations over a naive scalar baseline.

MetalNet supports both training and inference. It features a Directed Acyclic Graph (DAG) based automatic differentiation engine, core layers (Convolutional, Batch Normalization, Pooling, Dense), activation functions, SGD and Adam optimizers, and a built-in profiler. Tested against PyTorch on LeNet-5, VGG-style, and ResNet-18 architectures, MetalNet consistently runs **1.6–2.4× faster** for inference on the same CPU while reducing peak memory consumption by approximately **70%**. MetalNet demonstrates that high-performance deep learning is achievable on standard CPUs without complex external dependencies.

Keywords: Convolutional Neural Networks, C++20, Header-Only Library, SIMD Optimization, Implicit GEMM, Automatic Differentiation, AVX2, Cache Optimization, OpenMP, Deep Learning, CPU Performance

TABLE OF CONTENTS

Declaration Certificate	ii
Certificate of Approval	iii
Acknowledgement	iv
Abstract	v
Table of Contents	vi
List of Figures	x
List of Tables	x
List of Abbreviations	xi
CHAPTER 1 – INTRODUCTION	1
1.1 Background and Motivation	1
1.2 Problem Statement	2
1.3 Objectives of the Project	3
1.4 Scope of the Project	3
1.4.1 Inclusions	3
1.4.2 Exclusions	4
1.4.3 Target Users and Environment	4
1.5 Organization of the Report	5
CHAPTER 2 – LITERATURE REVIEW	6
2.1 Related Works	6
2.1.1 Deep Learning Frameworks	6
2.1.2 Convolution Optimization Algorithms	6
2.1.3 SIMD and Cache Optimization	7
2.1.4 Automatic Differentiation	7
2.1.5 Batch Normalization	7
2.1.6 Optimization Algorithms	7
2.2 Comparative Analysis	8
2.3 Gap Identification and Project Contribution	8
2.3.1 Why CPU Optimization Matters	9
CHAPTER 3 – SYSTEM ANALYSIS AND DESIGN	10
3.1 System Requirements	10

3.1.1	Hardware Requirements	10
3.1.2	Software Requirements	10
3.2	System Architecture	10
3.2.1	Core Tensor Engine	11
3.2.2	Layer System	11
3.2.3	Optimization System	11
3.2.4	Profiler and Utilities	12
3.3	Data Flow Diagrams	12
3.3.1	Level 0 - Context Diagram	12
3.3.2	Level 1 - Detailed Data Flow	13
3.4	Design Patterns and Implementation Details	13
3.4.1	Header-Only Design	13
3.4.2	RAII (Resource Acquisition Is Initialization)	13
3.4.3	Template Metaprogramming	13
3.5	Memory Layout and Optimization	14
CHAPTER 4 – IMPLEMENTATION		15
4.1	Technologies and Tools Used	15
4.1.1	Programming Language: C++20	15
4.1.2	Build System: CMake	15
4.1.3	SIMD Libraries	16
4.1.4	Parallelization: OpenMP	16
4.1.5	Profiling and Visualization	16
4.2	Module Description	17
4.2.1	Tensor Module	17
4.2.2	Layer Module	17
4.2.3	Conv2D Module: Implicit GEMM Implementation	18
4.2.4	Automatic Differentiation Engine	19
4.2.5	Optimization Module	20
4.3	Algorithm / Methodology	21
4.3.1	Implicit GEMM Convolution Algorithm	21
4.3.2	Cache-Aware Loop Tiling	22
4.3.3	SIMD Vectorization	23
4.3.4	Adaptive Multithreading	23
4.4	Code Implementation Highlights	24
4.4.1	FusedConvBNReLU Layer	24
4.4.2	Dropout Layer	25
4.4.3	Memory Optimization Techniques	25
4.5	Integration with Standard Frameworks	26

CHAPTER 5 – TESTING AND RESULTS	28
5.1 Testing Strategy	28
5.1.1 Unit Testing	28
5.1.2 Numerical Gradient Checking	28
5.1.3 Integration Testing	28
5.1.4 Performance Testing	28
5.2 Test Cases and Results	29
5.2.1 Unit Test Results	29
5.2.2 Integration Test Results	29
5.3 Performance Analysis	30
5.3.1 Inference Latency	30
5.3.2 Memory Usage	31
5.3.3 Scaling Analysis	31
5.3.4 Multi-threaded Scaling	32
5.3.5 Component-Level Performance Analysis	33
5.3.6 Comparison with Other Frameworks	33
5.3.7 Scaling Behavior with Model Complexity	34
CHAPTER 6 – CONCLUSION AND FUTURE WORK	35
6.1 Conclusion	35
6.2 Limitations and Current Constraints	36
6.3 Future Enhancements	36
6.3.1 Short-Term (6-12 months)	36
6.3.2 Medium-Term (1-2 years)	37
6.3.3 Long-Term (2+ years)	37
6.4 Lessons Learned	37
6.5 Design Trade-offs and Justification	38
6.5.1 Header-Only vs. Separate Compilation	38
6.5.2 NHWC Layout vs. NCHW	38
6.5.3 CPU-Only vs. GPU Support	38
6.6 Reproducibility and Open Science	39
6.7 Broader Impact and Applications	39
6.7.1 Edge Intelligence	39
6.7.2 Real-Time Systems	39
6.7.3 Privacy-Preserving Inference	40
6.8 Final Remarks	40
6.9 Contributions Summary	40
References	41
Appendix A – Source Code	43

A.1	Core Tensor Class	43
A.2	Conv2D Layer Implementation	45
A.3	SGD Optimizer Implementation	46
Appendix B – Screenshots / Sample Outputs		47
B.1	Output Screenshots	48

LIST OF FIGURES

Figure 3.1	MetalNet system architecture	12
Figure 3.2	Data Flow Diagram: Level 0 (Context Diagram)	12
Figure 3.3	Data Flow Diagram: Level 1 (Training Pipeline)	13
Figure 5.1	Inference latency comparison	30
Figure 5.2	Peak inference memory comparison	31
Figure 5.3	VGG-16 inference latency versus batch size	32
Figure 5.4	OpenMP scaling efficiency	32
Figure 5.5	Cumulative speedup of layered optimizations	33
Figure B.1	Filter Outputs	48
Figure B.2	Inference Output	48
Figure B.3	Profiler Interface	49
Figure B.4	Training Progress	49

LIST OF TABLES

Table 2.1	Comparison of CNN Frameworks and Optimization Approaches	8
Table 3.1	Hardware Requirements for MetalNet	10
Table 3.2	Software Requirements for MetalNet	10
Table 5.1	Unit Test Results for Core Layers	29
Table 5.2	MNIST Training Results (10 epochs)	29
Table 5.3	CIFAR-10 Training Results (20 epochs)	30
Table 5.4	Inference Latency Comparison (batch size 32, ms per batch)	30
Table 5.5	Memory Footprint Comparison (MB, batch size 32)	31
Table 5.6	Multi-threading Scaling (VGG-16, batch size 32)	32
Table 5.7	Performance Contribution of Individual Optimizations	33
Table 5.8	Comprehensive Framework Comparison	34
Table 5.9	Performance vs Model Complexity	34

LIST OF ABBREVIATIONS

CNN	Convolutional Neural Network
GEMM	General Matrix Multiplication
SIMD	Single Instruction, Multiple Data
AVX2	Advanced Vector Extensions 2
FMA	Fused Multiply–Add
im2col	Image to Column
DAG	Directed Acyclic Graph
SGD	Stochastic Gradient Descent
BN	Batch Normalization
ReLU	Rectified Linear Unit
NHWC	Batch, Height, Width, Channels
NCHW	Batch, Channels, Height, Width
L1/L2	Level 1 / Level 2 Cache
OpenMP	Open Multi-Processing
GPU	Graphics Processing Unit
CPU	Central Processing Unit
MNIST	Modified NIST handwritten digit dataset
CIFAR	Canadian Institute For Advanced Research
VGG	Visual Geometry Group
API	Application Programming Interface
DFD	Data Flow Diagram
UML	Unified Modelling Language
UAT	User Acceptance Testing
IDE	Integrated Development Environment

METALNET: DEPENDENCY-FREE C++ CNN HEADER-ONLY LIBRARY

A PROJECT REPORT SUBMITTED
IN PARTIAL FULFILMENT OF THE REQUIREMENT
FOR THE AWARD OF THE DEGREE

OF

BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE & ENGINEERING



BY

Raj Kunwar Prabhat 22050440078

Mayan Kumar 22050440058

Sujeet Kumar 22050440097

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
CAMBRIDGE INSTITUTE OF TECHNOLOGY**

TATISILWAI, RANCHI – 835103

JHARKHAND, INDIA

2026

DECLARATION CERTIFICATE

This is to certify that the work presented in the project entitled “**MetalNet: Dependency-Free C++ CNN Header-Only Library**” in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering at Cambridge Institute of Technology, Tatisilwai, Ranchi is an authentic work carried out under my supervision and guidance.

To the best of my knowledge, the content of this project does not form a basis for the award of any previous Degree to anyone else.

Date: _____

PROF. DEEPAK KUMAR VERMA

Dept. of Computer Science and Engineering
Cambridge Institute of Technology
Tatisilwai, Ranchi – 835103

PROF. DEEPAK KUMAR VERMA
(Head of Department)

Dept. of Computer Science and Engineering
Cambridge Institute of Technology
Tatisilwai, Ranchi – 835103

CERTIFICATE OF APPROVAL

The foregoing project entitled “MetalNet: Dependency-Free C++ CNN Header-Only Library” is hereby approved as a creditable work on the topic and has been presented satisfactorily to warrant its acceptance as a prerequisite to the degree for which it has been submitted.

It is understood that by this approval, the undersigned does not necessarily endorse any conclusion drawn or opinion expressed therein, but approves the project for the purpose for which it is submitted.

(Internal Examiner)

(External Examiner)

HEAD OF DEPARTMENT

Dept. of Computer Science and Engineering
Cambridge Institute of Technology
Tatisilwai, Ranchi – 835103

ACKNOWLEDGEMENT

I take this opportunity to thank all who have contributed towards shaping this Final Year Project Report. I would like to express my sincere thanks to Prof. Deepak Kumar Verma (HOD, Department of Computer Science & Engineering) and to my guide Prof. Deepak Kumar Verma, whose help in the course of development of this manuscript is invaluable. As my supervisor, he/she has constantly encouraged me to remain focused on achieving my goal.

I would also like to thank the technical staff members of the Department who have been kind enough to advise and help in their respective roles. I have been fortunate to have a wonderful support structure among fellow students. My sincere thanks to everyone who has provided me with kind words, new ideas, useful criticism, or their valuable time. I am truly indebted. Last but not least, I would like to dedicate this work to my family for their love and understanding.

Mayan Kumar
22050440058

ABSTRACT

Deep learning vision systems typically rely on large frameworks like PyTorch or TensorFlow, which carry significant dependency overhead and are designed primarily for GPUs. On CPU-only environments such as laptops or edge devices, these frameworks introduce unnecessary resource consumption. This project presents **MetalNet**, a high-performance Convolutional Neural Network (CNN) library written from scratch in pure C++20. MetalNet is fully **header-only** and **dependency-free**, requiring no installation, linking, or configuration, and is designed specifically to optimize CPU execution.

Despite its lightweight design, MetalNet achieves high performance by treating the CPU as a first-class citizen. It implements an **Implicit GEMM** convolution algorithm that avoids memory-heavy *im2col* buffering, and stores data in the cache-friendly **NHWC** layout. Innermost arithmetic operations are accelerated using hand-written **AVX2 and FMA** SIMD instructions, while loop tiling and parallelization via **OpenMP** optimize multi-core CPU scaling. Together, these optimizations yield a **57 $\times$ speedup** in convolution operations over a naive scalar baseline.

MetalNet supports both training and inference. It features a Directed Acyclic Graph (DAG) based automatic differentiation engine, core layers (Convolutional, Batch Normalization, Pooling, Dense), activation functions, SGD and Adam optimizers, and a built-in profiler. Tested against PyTorch on LeNet-5, VGG-style, and ResNet-18 architectures, MetalNet consistently runs **1.6–2.4 $\times$ faster** for inference on the same CPU while reducing peak memory consumption by approximately **70%**. MetalNet demonstrates that high-performance deep learning is achievable on standard CPUs without complex external dependencies.

Keywords: Convolutional Neural Networks, C++20, Header-Only Library, SIMD Optimization, Implicit GEMM, Automatic Differentiation, AVX2, Cache Optimization, OpenMP, Deep Learning, CPU Performance

TABLE OF CONTENTS

Declaration Certificate	ii
Certificate of Approval	iii
Acknowledgement	iv
Abstract	v
Table of Contents	vi
List of Figures	x
List of Tables	x
List of Abbreviations	xi
CHAPTER 1 – INTRODUCTION	1
1.1 Background and Motivation	1
1.2 Problem Statement	2
1.3 Objectives of the Project	3
1.4 Scope of the Project	3
1.4.1 Inclusions	3
1.4.2 Exclusions	4
1.4.3 Target Users and Environment	4
1.5 Organization of the Report	5
CHAPTER 2 – LITERATURE REVIEW	6
2.1 Related Works	6
2.1.1 Deep Learning Frameworks	6
2.1.2 Convolution Optimization Algorithms	6
2.1.3 SIMD and Cache Optimization	7
2.1.4 Automatic Differentiation	7
2.1.5 Batch Normalization	7
2.1.6 Optimization Algorithms	7
2.2 Comparative Analysis	8
2.3 Gap Identification and Project Contribution	8
2.3.1 Why CPU Optimization Matters	9
CHAPTER 3 – SYSTEM ANALYSIS AND DESIGN	10
3.1 System Requirements	10

3.1.1	Hardware Requirements	10
3.1.2	Software Requirements	10
3.2	System Architecture	10
3.2.1	Core Tensor Engine	11
3.2.2	Layer System	11
3.2.3	Optimization System	11
3.2.4	Profiler and Utilities	12
3.3	Data Flow Diagrams	12
3.3.1	Level 0 - Context Diagram	12
3.3.2	Level 1 - Detailed Data Flow	13
3.4	Design Patterns and Implementation Details	13
3.4.1	Header-Only Design	13
3.4.2	RAII (Resource Acquisition Is Initialization)	13
3.4.3	Template Metaprogramming	13
3.5	Memory Layout and Optimization	14
CHAPTER 4 – IMPLEMENTATION		15
4.1	Technologies and Tools Used	15
4.1.1	Programming Language: C++20	15
4.1.2	Build System: CMake	15
4.1.3	SIMD Libraries	16
4.1.4	Parallelization: OpenMP	16
4.1.5	Profiling and Visualization	16
4.2	Module Description	17
4.2.1	Tensor Module	17
4.2.2	Layer Module	17
4.2.3	Conv2D Module: Implicit GEMM Implementation	18
4.2.4	Automatic Differentiation Engine	19
4.2.5	Optimization Module	20
4.3	Algorithm / Methodology	21
4.3.1	Implicit GEMM Convolution Algorithm	21
4.3.2	Cache-Aware Loop Tiling	22
4.3.3	SIMD Vectorization	23
4.3.4	Adaptive Multithreading	23
4.4	Code Implementation Highlights	24
4.4.1	FusedConvBNReLU Layer	24
4.4.2	Dropout Layer	25
4.4.3	Memory Optimization Techniques	25
4.5	Integration with Standard Frameworks	26

CHAPTER 5 – TESTING AND RESULTS	28
5.1 Testing Strategy	28
5.1.1 Unit Testing	28
5.1.2 Numerical Gradient Checking	28
5.1.3 Integration Testing	28
5.1.4 Performance Testing	28
5.2 Test Cases and Results	29
5.2.1 Unit Test Results	29
5.2.2 Integration Test Results	29
5.3 Performance Analysis	30
5.3.1 Inference Latency	30
5.3.2 Memory Usage	31
5.3.3 Scaling Analysis	31
5.3.4 Multi-threaded Scaling	32
5.3.5 Component-Level Performance Analysis	33
5.3.6 Comparison with Other Frameworks	33
5.3.7 Scaling Behavior with Model Complexity	34
CHAPTER 6 – CONCLUSION AND FUTURE WORK	35
6.1 Conclusion	35
6.2 Limitations and Current Constraints	36
6.3 Future Enhancements	36
6.3.1 Short-Term (6-12 months)	36
6.3.2 Medium-Term (1-2 years)	37
6.3.3 Long-Term (2+ years)	37
6.4 Lessons Learned	37
6.5 Design Trade-offs and Justification	38
6.5.1 Header-Only vs. Separate Compilation	38
6.5.2 NHWC Layout vs. NCHW	38
6.5.3 CPU-Only vs. GPU Support	38
6.6 Reproducibility and Open Science	39
6.7 Broader Impact and Applications	39
6.7.1 Edge Intelligence	39
6.7.2 Real-Time Systems	39
6.7.3 Privacy-Preserving Inference	40
6.8 Final Remarks	40
6.9 Contributions Summary	40
References	41
Appendix A – Source Code	43

A.1	Core Tensor Class	43
A.2	Conv2D Layer Implementation	45
A.3	SGD Optimizer Implementation	46
Appendix B – Screenshots / Sample Outputs		47
B.1	Output Screenshots	48

LIST OF FIGURES

Figure 3.1	MetalNet system architecture	12
Figure 3.2	Data Flow Diagram: Level 0 (Context Diagram)	12
Figure 3.3	Data Flow Diagram: Level 1 (Training Pipeline)	13
Figure 5.1	Inference latency comparison	30
Figure 5.2	Peak inference memory comparison	31
Figure 5.3	VGG-16 inference latency versus batch size	32
Figure 5.4	OpenMP scaling efficiency	32
Figure 5.5	Cumulative speedup of layered optimizations	33
Figure B.1	Filter Outputs	48
Figure B.2	Inference Output	48
Figure B.3	Profiler Interface	49
Figure B.4	Training Progress	49

LIST OF TABLES

Table 2.1	Comparison of CNN Frameworks and Optimization Approaches	8
Table 3.1	Hardware Requirements for MetalNet	10
Table 3.2	Software Requirements for MetalNet	10
Table 5.1	Unit Test Results for Core Layers	29
Table 5.2	MNIST Training Results (10 epochs)	29
Table 5.3	CIFAR-10 Training Results (20 epochs)	30
Table 5.4	Inference Latency Comparison (batch size 32, ms per batch)	30
Table 5.5	Memory Footprint Comparison (MB, batch size 32)	31
Table 5.6	Multi-threading Scaling (VGG-16, batch size 32)	32
Table 5.7	Performance Contribution of Individual Optimizations	33
Table 5.8	Comprehensive Framework Comparison	34
Table 5.9	Performance vs Model Complexity	34

LIST OF ABBREVIATIONS

CNN	Convolutional Neural Network
GEMM	General Matrix Multiplication
SIMD	Single Instruction, Multiple Data
AVX2	Advanced Vector Extensions 2
FMA	Fused Multiply–Add
im2col	Image to Column
DAG	Directed Acyclic Graph
SGD	Stochastic Gradient Descent
BN	Batch Normalization
ReLU	Rectified Linear Unit
NHWC	Batch, Height, Width, Channels
NCHW	Batch, Channels, Height, Width
L1/L2	Level 1 / Level 2 Cache
OpenMP	Open Multi-Processing
GPU	Graphics Processing Unit
CPU	Central Processing Unit
MNIST	Modified NIST handwritten digit dataset
CIFAR	Canadian Institute For Advanced Research
VGG	Visual Geometry Group
API	Application Programming Interface
DFD	Data Flow Diagram
UML	Unified Modelling Language
UAT	User Acceptance Testing
IDE	Integrated Development Environment

METALNET: DEPENDENCY-FREE C++ CNN HEADER-ONLY LIBRARY

A PROJECT REPORT SUBMITTED
IN PARTIAL FULFILMENT OF THE REQUIREMENT
FOR THE AWARD OF THE DEGREE
OF

BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE & ENGINEERING



BY

Raj Kunwar Prabhat 22050440078

Mayan Kumar 22050440058

Sujeet Kumar 22050440097

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
CAMBRIDGE INSTITUTE OF TECHNOLOGY**

TATISILWAI, RANCHI – 835103

JHARKHAND, INDIA

2026

DECLARATION CERTIFICATE

This is to certify that the work presented in the project entitled “**MetalNet: Dependency-Free C++ CNN Header-Only Library**” in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering at Cambridge Institute of Technology, Tatisilwai, Ranchi is an authentic work carried out under my supervision and guidance.

To the best of my knowledge, the content of this project does not form a basis for the award of any previous Degree to anyone else.

Date: _____

PROF. DEEPAK KUMAR VERMA

Dept. of Computer Science and Engineering
Cambridge Institute of Technology
Tatisilwai, Ranchi – 835103

PROF. DEEPAK KUMAR VERMA
(Head of Department)

Dept. of Computer Science and Engineering
Cambridge Institute of Technology
Tatisilwai, Ranchi – 835103

CERTIFICATE OF APPROVAL

The foregoing project entitled “MetalNet: Dependency-Free C++ CNN Header-Only Library” is hereby approved as a creditable work on the topic and has been presented satisfactorily to warrant its acceptance as a prerequisite to the degree for which it has been submitted.

It is understood that by this approval, the undersigned does not necessarily endorse any conclusion drawn or opinion expressed therein, but approves the project for the purpose for which it is submitted.

(Internal Examiner)

(External Examiner)

HEAD OF DEPARTMENT

Dept. of Computer Science and Engineering
Cambridge Institute of Technology
Tatisilwai, Ranchi – 835103

ACKNOWLEDGEMENT

I take this opportunity to thank all who have contributed towards shaping this Final Year Project Report. I would like to express my sincere thanks to Prof. Deepak Kumar Verma (HOD, Department of Computer Science & Engineering) and to my guide Prof. Deepak Kumar Verma, whose help in the course of development of this manuscript is invaluable. As my supervisor, he/she has constantly encouraged me to remain focused on achieving my goal.

I would also like to thank the technical staff members of the Department who have been kind enough to advise and help in their respective roles. I have been fortunate to have a wonderful support structure among fellow students. My sincere thanks to everyone who has provided me with kind words, new ideas, useful criticism, or their valuable time. I am truly indebted. Last but not least, I would like to dedicate this work to my family for their love and understanding.

Sujeet Kumar
22050440097

ABSTRACT

Deep learning vision systems typically rely on large frameworks like PyTorch or TensorFlow, which carry significant dependency overhead and are designed primarily for GPUs. On CPU-only environments such as laptops or edge devices, these frameworks introduce unnecessary resource consumption. This project presents **MetalNet**, a high-performance Convolutional Neural Network (CNN) library written from scratch in pure C++20. MetalNet is fully **header-only** and **dependency-free**, requiring no installation, linking, or configuration, and is designed specifically to optimize CPU execution.

Despite its lightweight design, MetalNet achieves high performance by treating the CPU as a first-class citizen. It implements an **Implicit GEMM** convolution algorithm that avoids memory-heavy *im2col* buffering, and stores data in the cache-friendly **NHWC** layout. Innermost arithmetic operations are accelerated using hand-written **AVX2 and FMA** SIMD instructions, while loop tiling and parallelization via **OpenMP** optimize multi-core CPU scaling. Together, these optimizations yield a **57 $\times$ speedup** in convolution operations over a naive scalar baseline.

MetalNet supports both training and inference. It features a Directed Acyclic Graph (DAG) based automatic differentiation engine, core layers (Convolutional, Batch Normalization, Pooling, Dense), activation functions, SGD and Adam optimizers, and a built-in profiler. Tested against PyTorch on LeNet-5, VGG-style, and ResNet-18 architectures, MetalNet consistently runs **1.6–2.4 $\times$ faster** for inference on the same CPU while reducing peak memory consumption by approximately **70%**. MetalNet demonstrates that high-performance deep learning is achievable on standard CPUs without complex external dependencies.

Keywords: Convolutional Neural Networks, C++20, Header-Only Library, SIMD Optimization, Implicit GEMM, Automatic Differentiation, AVX2, Cache Optimization, OpenMP, Deep Learning, CPU Performance

TABLE OF CONTENTS

Declaration Certificate	ii
Certificate of Approval	iii
Acknowledgement	iv
Abstract	v
Table of Contents	vi
List of Figures	x
List of Tables	x
List of Abbreviations	xi
CHAPTER 1 – INTRODUCTION	1
1.1 Background and Motivation	1
1.2 Problem Statement	2
1.3 Objectives of the Project	3
1.4 Scope of the Project	3
1.4.1 Inclusions	3
1.4.2 Exclusions	4
1.4.3 Target Users and Environment	4
1.5 Organization of the Report	5
CHAPTER 2 – LITERATURE REVIEW	6
2.1 Related Works	6
2.1.1 Deep Learning Frameworks	6
2.1.2 Convolution Optimization Algorithms	6
2.1.3 SIMD and Cache Optimization	7
2.1.4 Automatic Differentiation	7
2.1.5 Batch Normalization	7
2.1.6 Optimization Algorithms	7
2.2 Comparative Analysis	8
2.3 Gap Identification and Project Contribution	8
2.3.1 Why CPU Optimization Matters	9
CHAPTER 3 – SYSTEM ANALYSIS AND DESIGN	10
3.1 System Requirements	10

3.1.1	Hardware Requirements	10
3.1.2	Software Requirements	10
3.2	System Architecture	10
3.2.1	Core Tensor Engine	11
3.2.2	Layer System	11
3.2.3	Optimization System	11
3.2.4	Profiler and Utilities	12
3.3	Data Flow Diagrams	12
3.3.1	Level 0 - Context Diagram	12
3.3.2	Level 1 - Detailed Data Flow	13
3.4	Design Patterns and Implementation Details	13
3.4.1	Header-Only Design	13
3.4.2	RAII (Resource Acquisition Is Initialization)	13
3.4.3	Template Metaprogramming	13
3.5	Memory Layout and Optimization	14
CHAPTER 4 – IMPLEMENTATION		15
4.1	Technologies and Tools Used	15
4.1.1	Programming Language: C++20	15
4.1.2	Build System: CMake	15
4.1.3	SIMD Libraries	16
4.1.4	Parallelization: OpenMP	16
4.1.5	Profiling and Visualization	16
4.2	Module Description	17
4.2.1	Tensor Module	17
4.2.2	Layer Module	17
4.2.3	Conv2D Module: Implicit GEMM Implementation	18
4.2.4	Automatic Differentiation Engine	19
4.2.5	Optimization Module	20
4.3	Algorithm / Methodology	21
4.3.1	Implicit GEMM Convolution Algorithm	21
4.3.2	Cache-Aware Loop Tiling	22
4.3.3	SIMD Vectorization	23
4.3.4	Adaptive Multithreading	23
4.4	Code Implementation Highlights	24
4.4.1	FusedConvBNReLU Layer	24
4.4.2	Dropout Layer	25
4.4.3	Memory Optimization Techniques	25
4.5	Integration with Standard Frameworks	26

CHAPTER 5 – TESTING AND RESULTS	28
5.1 Testing Strategy	28
5.1.1 Unit Testing	28
5.1.2 Numerical Gradient Checking	28
5.1.3 Integration Testing	28
5.1.4 Performance Testing	28
5.2 Test Cases and Results	29
5.2.1 Unit Test Results	29
5.2.2 Integration Test Results	29
5.3 Performance Analysis	30
5.3.1 Inference Latency	30
5.3.2 Memory Usage	31
5.3.3 Scaling Analysis	31
5.3.4 Multi-threaded Scaling	32
5.3.5 Component-Level Performance Analysis	33
5.3.6 Comparison with Other Frameworks	33
5.3.7 Scaling Behavior with Model Complexity	34
CHAPTER 6 – CONCLUSION AND FUTURE WORK	35
6.1 Conclusion	35
6.2 Limitations and Current Constraints	36
6.3 Future Enhancements	36
6.3.1 Short-Term (6-12 months)	36
6.3.2 Medium-Term (1-2 years)	37
6.3.3 Long-Term (2+ years)	37
6.4 Lessons Learned	37
6.5 Design Trade-offs and Justification	38
6.5.1 Header-Only vs. Separate Compilation	38
6.5.2 NHWC Layout vs. NCHW	38
6.5.3 CPU-Only vs. GPU Support	38
6.6 Reproducibility and Open Science	39
6.7 Broader Impact and Applications	39
6.7.1 Edge Intelligence	39
6.7.2 Real-Time Systems	39
6.7.3 Privacy-Preserving Inference	40
6.8 Final Remarks	40
6.9 Contributions Summary	40
References	41
Appendix A – Source Code	43

A.1	Core Tensor Class	43
A.2	Conv2D Layer Implementation	45
A.3	SGD Optimizer Implementation	46
Appendix B – Screenshots / Sample Outputs		47
B.1	Output Screenshots	48

LIST OF FIGURES

Figure 3.1	MetalNet system architecture	12
Figure 3.2	Data Flow Diagram: Level 0 (Context Diagram)	12
Figure 3.3	Data Flow Diagram: Level 1 (Training Pipeline)	13
Figure 5.1	Inference latency comparison	30
Figure 5.2	Peak inference memory comparison	31
Figure 5.3	VGG-16 inference latency versus batch size	32
Figure 5.4	OpenMP scaling efficiency	32
Figure 5.5	Cumulative speedup of layered optimizations	33
Figure B.1	Filter Outputs	48
Figure B.2	Inference Output	48
Figure B.3	Profiler Interface	49
Figure B.4	Training Progress	49

LIST OF TABLES

Table 2.1	Comparison of CNN Frameworks and Optimization Approaches	8
Table 3.1	Hardware Requirements for MetalNet	10
Table 3.2	Software Requirements for MetalNet	10
Table 5.1	Unit Test Results for Core Layers	29
Table 5.2	MNIST Training Results (10 epochs)	29
Table 5.3	CIFAR-10 Training Results (20 epochs)	30
Table 5.4	Inference Latency Comparison (batch size 32, ms per batch)	30
Table 5.5	Memory Footprint Comparison (MB, batch size 32)	31
Table 5.6	Multi-threading Scaling (VGG-16, batch size 32)	32
Table 5.7	Performance Contribution of Individual Optimizations	33
Table 5.8	Comprehensive Framework Comparison	34
Table 5.9	Performance vs Model Complexity	34

LIST OF ABBREVIATIONS

CNN	Convolutional Neural Network
GEMM	General Matrix Multiplication
SIMD	Single Instruction, Multiple Data
AVX2	Advanced Vector Extensions 2
FMA	Fused Multiply–Add
im2col	Image to Column
DAG	Directed Acyclic Graph
SGD	Stochastic Gradient Descent
BN	Batch Normalization
ReLU	Rectified Linear Unit
NHWC	Batch, Height, Width, Channels
NCHW	Batch, Channels, Height, Width
L1/L2	Level 1 / Level 2 Cache
OpenMP	Open Multi-Processing
GPU	Graphics Processing Unit
CPU	Central Processing Unit
MNIST	Modified NIST handwritten digit dataset
CIFAR	Canadian Institute For Advanced Research
VGG	Visual Geometry Group
API	Application Programming Interface
DFD	Data Flow Diagram
UML	Unified Modelling Language
UAT	User Acceptance Testing
IDE	Integrated Development Environment

METALNET: DEPENDENCY-FREE C++ CNN HEADER-ONLY LIBRARY

A PROJECT REPORT SUBMITTED
IN PARTIAL FULFILMENT OF THE REQUIREMENT
FOR THE AWARD OF THE DEGREE
OF

BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE & ENGINEERING



BY

Raj Kunwar Prabhat 22050440078

Mayan Kumar 22050440058

Sujeet Kumar 22050440097

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
CAMBRIDGE INSTITUTE OF TECHNOLOGY**

TATISILWAI, RANCHI – 835103

JHARKHAND, INDIA

2026

DECLARATION CERTIFICATE

This is to certify that the work presented in the project entitled “**MetalNet: Dependency-Free C++ CNN Header-Only Library**” in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering at Cambridge Institute of Technology, Tatisilwai, Ranchi is an authentic work carried out under my supervision and guidance.

To the best of my knowledge, the content of this project does not form a basis for the award of any previous Degree to anyone else.

Date: _____

PROF. DEEPAK KUMAR VERMA

Dept. of Computer Science and Engineering
Cambridge Institute of Technology
Tatisilwai, Ranchi – 835103

PROF. DEEPAK KUMAR VERMA
(Head of Department)

Dept. of Computer Science and Engineering
Cambridge Institute of Technology
Tatisilwai, Ranchi – 835103

CERTIFICATE OF APPROVAL

The foregoing project entitled “MetalNet: Dependency-Free C++ CNN Header-Only Library” is hereby approved as a creditable work on the topic and has been presented satisfactorily to warrant its acceptance as a prerequisite to the degree for which it has been submitted.

It is understood that by this approval, the undersigned does not necessarily endorse any conclusion drawn or opinion expressed therein, but approves the project for the purpose for which it is submitted.

(Internal Examiner)

(External Examiner)

HEAD OF DEPARTMENT

Dept. of Computer Science and Engineering
Cambridge Institute of Technology
Tatisilwai, Ranchi – 835103

ACKNOWLEDGEMENT

I take this opportunity to thank all who have contributed towards shaping this Final Year Project Report. I would like to express my sincere thanks to Prof. Deepak Kumar Verma (HOD, Department of Computer Science & Engineering) and to my guide Prof. Deepak Kumar Verma, whose help in the course of development of this manuscript is invaluable. As my supervisor, he/she has constantly encouraged me to remain focused on achieving my goal.

I would also like to thank the technical staff members of the Department who have been kind enough to advise and help in their respective roles. I have been fortunate to have a wonderful support structure among fellow students. My sincere thanks to everyone who has provided me with kind words, new ideas, useful criticism, or their valuable time. I am truly indebted. Last but not least, I would like to dedicate this work to my family for their love and understanding.

Raj Kunwar Prabhat
22050440078

Mayan Kumar
22050440058

Sujeet Kumar
22050440097

ABSTRACT

Deep learning vision systems typically rely on large frameworks like PyTorch or TensorFlow, which carry significant dependency overhead and are designed primarily for GPUs. On CPU-only environments such as laptops or edge devices, these frameworks introduce unnecessary resource consumption. This project presents **MetalNet**, a high-performance Convolutional Neural Network (CNN) library written from scratch in pure C++20. MetalNet is fully **header-only** and **dependency-free**, requiring no installation, linking, or configuration, and is designed specifically to optimize CPU execution.

Despite its lightweight design, MetalNet achieves high performance by treating the CPU as a first-class citizen. It implements an **Implicit GEMM** convolution algorithm that avoids memory-heavy *im2col* buffering, and stores data in the cache-friendly **NHWC** layout. Innermost arithmetic operations are accelerated using hand-written **AVX2 and FMA** SIMD instructions, while loop tiling and parallelization via **OpenMP** optimize multi-core CPU scaling. Together, these optimizations yield a **57 $\times$ speedup** in convolution operations over a naive scalar baseline.

MetalNet supports both training and inference. It features a Directed Acyclic Graph (DAG) based automatic differentiation engine, core layers (Convolutional, Batch Normalization, Pooling, Dense), activation functions, SGD and Adam optimizers, and a built-in profiler. Tested against PyTorch on LeNet-5, VGG-style, and ResNet-18 architectures, MetalNet consistently runs **1.6–2.4 $\times$ faster** for inference on the same CPU while reducing peak memory consumption by approximately **70%**. MetalNet demonstrates that high-performance deep learning is achievable on standard CPUs without complex external dependencies.

Keywords: Convolutional Neural Networks, C++20, Header-Only Library, SIMD Optimization, Implicit GEMM, Automatic Differentiation, AVX2, Cache Optimization, OpenMP, Deep Learning, CPU Performance

TABLE OF CONTENTS

Declaration Certificate	ii
Certificate of Approval	iii
Acknowledgement	iv
Abstract	v
Table of Contents	vi
List of Figures	x
List of Tables	x
List of Abbreviations	xi
CHAPTER 1 – INTRODUCTION	1
1.1 Background and Motivation	1
1.2 Problem Statement	2
1.3 Objectives of the Project	3
1.4 Scope of the Project	3
1.4.1 Inclusions	3
1.4.2 Exclusions	4
1.4.3 Target Users and Environment	4
1.5 Organization of the Report	5
CHAPTER 2 – LITERATURE REVIEW	6
2.1 Related Works	6
2.1.1 Deep Learning Frameworks	6
2.1.2 Convolution Optimization Algorithms	6
2.1.3 SIMD and Cache Optimization	7
2.1.4 Automatic Differentiation	7
2.1.5 Batch Normalization	7
2.1.6 Optimization Algorithms	7
2.2 Comparative Analysis	8
2.3 Gap Identification and Project Contribution	8
2.3.1 Why CPU Optimization Matters	9
CHAPTER 3 – SYSTEM ANALYSIS AND DESIGN	10
3.1 System Requirements	10

3.1.1	Hardware Requirements	10
3.1.2	Software Requirements	10
3.2	System Architecture	10
3.2.1	Core Tensor Engine	11
3.2.2	Layer System	11
3.2.3	Optimization System	11
3.2.4	Profiler and Utilities	12
3.3	Data Flow Diagrams	12
3.3.1	Level 0 - Context Diagram	12
3.3.2	Level 1 - Detailed Data Flow	13
3.4	Design Patterns and Implementation Details	13
3.4.1	Header-Only Design	13
3.4.2	RAII (Resource Acquisition Is Initialization)	13
3.4.3	Template Metaprogramming	13
3.5	Memory Layout and Optimization	14
CHAPTER 4 – IMPLEMENTATION		15
4.1	Technologies and Tools Used	15
4.1.1	Programming Language: C++20	15
4.1.2	Build System: CMake	15
4.1.3	SIMD Libraries	16
4.1.4	Parallelization: OpenMP	16
4.1.5	Profiling and Visualization	16
4.2	Module Description	17
4.2.1	Tensor Module	17
4.2.2	Layer Module	17
4.2.3	Conv2D Module: Implicit GEMM Implementation	18
4.2.4	Automatic Differentiation Engine	19
4.2.5	Optimization Module	20
4.3	Algorithm / Methodology	21
4.3.1	Implicit GEMM Convolution Algorithm	21
4.3.2	Cache-Aware Loop Tiling	22
4.3.3	SIMD Vectorization	23
4.3.4	Adaptive Multithreading	23
4.4	Code Implementation Highlights	24
4.4.1	FusedConvBNReLU Layer	24
4.4.2	Dropout Layer	25
4.4.3	Memory Optimization Techniques	25
4.5	Integration with Standard Frameworks	26

CHAPTER 5 – TESTING AND RESULTS	28
5.1 Testing Strategy	28
5.1.1 Unit Testing	28
5.1.2 Numerical Gradient Checking	28
5.1.3 Integration Testing	28
5.1.4 Performance Testing	28
5.2 Test Cases and Results	29
5.2.1 Unit Test Results	29
5.2.2 Integration Test Results	29
5.3 Performance Analysis	30
5.3.1 Inference Latency	30
5.3.2 Memory Usage	31
5.3.3 Scaling Analysis	31
5.3.4 Multi-threaded Scaling	32
5.3.5 Component-Level Performance Analysis	33
5.3.6 Comparison with Other Frameworks	33
5.3.7 Scaling Behavior with Model Complexity	34
CHAPTER 6 – CONCLUSION AND FUTURE WORK	35
6.1 Conclusion	35
6.2 Limitations and Current Constraints	36
6.3 Future Enhancements	36
6.3.1 Short-Term (6-12 months)	36
6.3.2 Medium-Term (1-2 years)	37
6.3.3 Long-Term (2+ years)	37
6.4 Lessons Learned	37
6.5 Design Trade-offs and Justification	38
6.5.1 Header-Only vs. Separate Compilation	38
6.5.2 NHWC Layout vs. NCHW	38
6.5.3 CPU-Only vs. GPU Support	38
6.6 Reproducibility and Open Science	39
6.7 Broader Impact and Applications	39
6.7.1 Edge Intelligence	39
6.7.2 Real-Time Systems	39
6.7.3 Privacy-Preserving Inference	40
6.8 Final Remarks	40
6.9 Contributions Summary	40
References	41
Appendix A – Source Code	43

A.1	Core Tensor Class	43
A.2	Conv2D Layer Implementation	45
A.3	SGD Optimizer Implementation	46
Appendix B – Screenshots / Sample Outputs		47
B.1	Output Screenshots	48

LIST OF FIGURES

Figure 3.1	MetalNet system architecture	12
Figure 3.2	Data Flow Diagram: Level 0 (Context Diagram)	12
Figure 3.3	Data Flow Diagram: Level 1 (Training Pipeline)	13
Figure 5.1	Inference latency comparison	30
Figure 5.2	Peak inference memory comparison	31
Figure 5.3	VGG-16 inference latency versus batch size	32
Figure 5.4	OpenMP scaling efficiency	32
Figure 5.5	Cumulative speedup of layered optimizations	33
Figure B.1	Filter Outputs	48
Figure B.2	Inference Output	48
Figure B.3	Profiler Interface	49
Figure B.4	Training Progress	49

LIST OF TABLES

Table 2.1	Comparison of CNN Frameworks and Optimization Approaches	8
Table 3.1	Hardware Requirements for MetalNet	10
Table 3.2	Software Requirements for MetalNet	10
Table 5.1	Unit Test Results for Core Layers	29
Table 5.2	MNIST Training Results (10 epochs)	29
Table 5.3	CIFAR-10 Training Results (20 epochs)	30
Table 5.4	Inference Latency Comparison (batch size 32, ms per batch)	30
Table 5.5	Memory Footprint Comparison (MB, batch size 32)	31
Table 5.6	Multi-threading Scaling (VGG-16, batch size 32)	32
Table 5.7	Performance Contribution of Individual Optimizations	33
Table 5.8	Comprehensive Framework Comparison	34
Table 5.9	Performance vs Model Complexity	34

LIST OF ABBREVIATIONS

CNN	Convolutional Neural Network
GEMM	General Matrix Multiplication
SIMD	Single Instruction, Multiple Data
AVX2	Advanced Vector Extensions 2
FMA	Fused Multiply–Add
im2col	Image to Column
DAG	Directed Acyclic Graph
SGD	Stochastic Gradient Descent
BN	Batch Normalization
ReLU	Rectified Linear Unit
NHWC	Batch, Height, Width, Channels
NCHW	Batch, Channels, Height, Width
L1/L2	Level 1 / Level 2 Cache
OpenMP	Open Multi-Processing
GPU	Graphics Processing Unit
CPU	Central Processing Unit
MNIST	Modified NIST handwritten digit dataset
CIFAR	Canadian Institute For Advanced Research
VGG	Visual Geometry Group
API	Application Programming Interface
DFD	Data Flow Diagram
UML	Unified Modelling Language
UAT	User Acceptance Testing
IDE	Integrated Development Environment

CHAPTER 1 – INTRODUCTION

1.1 Background and Motivation

Deep learning and neural networks have revolutionized the field of artificial intelligence and machine learning, enabling breakthrough achievements in computer vision, natural language processing, and other domains. Convolutional Neural Networks (CNNs), in particular, have become the de facto standard for image classification, object detection, semantic segmentation, and numerous other vision tasks. The success of CNNs can be attributed to their ability to automatically learn hierarchical features from raw input data through multiple convolutional layers, each building upon the learnings of previous layers.

However, the computational demands of training and inference with deep neural networks are substantial. Large-scale models can require millions of hours of GPU computation time, translating to enormous energy consumption and financial costs. While GPUs (Graphics Processing Units) have become the standard for accelerating deep learning workloads, not all applications have access to GPU resources, and many edge computing scenarios require efficient CPU-based inference.

The popular deep learning frameworks available today, such as PyTorch, TensorFlow, ONNX Runtime, and Caffe, are designed to be general-purpose tools supporting diverse architectures and deployment scenarios. This generality comes at a cost: significant memory overhead, unnecessary data copies, abstraction layers, and inability to leverage modern CPU instruction sets fully. When these frameworks run on CPU, their performance is often significantly slower than what a specialized, optimized implementation could achieve.

The motivation for MetalNet stems from the observation that with careful algorithmic design and hardware-conscious implementation, a specialized CNN library can achieve superior performance on commodity CPUs compared to general-purpose frameworks. Modern CPUs feature:

1. **SIMD Capabilities:** AVX2 registers and FMA (Fused Multiply-Add) instructions enable 8-way parallelism on 32-bit floating-point data.
2. **Multi-core Architectures:** Modern CPUs have 4-32+ cores, allowing coarse-grained parallelism through OpenMP.
3. **Cache Hierarchies:** L1 (32 KB), L2 (256 KB), and L3 (8-20 MB) caches with predictable access patterns and can be optimized through cache-aware algorithms.

4. **Modern Language Features:** C++20 enables compile-time computation, zero-overhead abstractions, and type-safe generic programming.

By combining these capabilities with algorithm innovations like Implicit GEMM convolution (eliminating the memory-intensive im2col operation), cache-aware loop tiling, and DAG-based automatic differentiation, MetalNet achieves performance levels that exceed PyTorch’s CPU implementation on standard benchmarks.

The project is header-only (no separate compilation step), zero-dependency (beyond standard C++20), and fully self-contained. This eliminates the significant overhead of framework initialization, memory management indirection, and runtime dispatch mechanisms. The library provides a complete neural network toolkit including convolutional layers, batch normalization, dense layers, pooling operations, activation functions, loss functions, optimizers, and a profiler with GUI-based visualization.

1.2 Problem Statement

The problem addressed by this project can be framed as follows:

1. **CPU Inference Inefficiency:** General-purpose deep learning frameworks have significant overhead when running on CPUs. They are optimized for GPU execution and perform poorly on commodity hardware lacking GPU acceleration.
2. **Memory Inefficiency:** Traditional convolution implementations using im2col (Image to Column) transformations require allocating large intermediate buffers, sometimes exceeding the size of the feature maps themselves. This increases memory bandwidth requirements and reduces cache efficiency.
3. **Lack of Hardware Awareness:** Modern frameworks make limited use of CPU-specific features like SIMD instructions (AVX2, AVX-512) and cache hierarchy knowledge. They prioritize generality over specialized optimization.
4. **Dependency Bloat:** Production deep learning frameworks depend on numerous external libraries (CUDA, cuDNN, Eigen, Protobuf), increasing build complexity, binary size, and deployment difficulty.
5. **Type Safety and Compile-Time Optimization:** Python-based frameworks sacrifice type safety and compile-time optimization opportunities. Even C++ frameworks often use runtime polymorphism excessively.

The question becomes: *Can a purpose-built, C++20 CNN library, optimized for CPU execution with modern instruction sets and algorithmic innovations, achieve performance comparable to or exceeding general-purpose frameworks?*

1.3 Objectives of the Project

The objectives of this project are:

1. **Design and Develop:** Create a header-only C++20 CNN library implementing core neural network layers (convolution, batch normalization, pooling, dense, activation functions) with zero external dependencies.
2. **Optimize for CPU Performance:** Implement advanced optimization techniques including Implicit GEMM convolution, cache-aware loop tiling, AVX2/FMA SIMD utilization, and OpenMP-based adaptive multithreading.
3. **Achieve Automatic Differentiation:** Implement a complete DAG-based autograd system supporting arbitrary computation graphs and enabling both forward and backward propagation for training.
4. **Implement Training Infrastructure:** Provide built-in optimizers (SGD, Adam with momentum), loss functions (MSE, Cross-Entropy), and gradient-based learning mechanisms.
5. **Benchmark and Validate:** Comprehensively benchmark MetalNet against PyTorch on standard datasets (MNIST, CIFAR-10) and architectures (LeNet, VGG-style networks), demonstrating performance advantages.
6. **Provide Profiling Infrastructure:** Develop a profiler with visualization capabilities (GUI with ImGui/ImPlot) to identify computational bottlenecks and guide further optimization.
7. **Document Thoroughly:** Create comprehensive documentation, API references, and usage examples enabling users to leverage the library effectively.

1.4 Scope of the Project

1.4.1 Inclusions

- Header-only implementation of CNN layers and operations
- Support for 2D convolutions with configurable kernel sizes, strides, and padding
- Batch normalization with running statistics
- Multiple activation functions (ReLU, LeakyReLU, Sigmoid, Softmax)
- Max/Average pooling operations
- Dense (fully connected) layers

- Dropout layer for regularization
- Loss functions (Mean Squared Error, Cross-Entropy)
- Optimizers (SGD with momentum, Adam)
- Automatic differentiation framework supporting arbitrary computation graphs
- C++20 with AVX2 and FMA SIMD optimizations
- OpenMP multithreading with adaptive thread control
- Cache-aware optimization and loop tiling
- Profiler with performance metrics and visualization
- Comprehensive test suite and benchmark suite

1.4.2 Exclusions

- GPU support (CUDA, OpenCL, Metal)
- Distributed training across multiple machines
- Quantization and mixed-precision arithmetic
- Recurrent Neural Networks (RNNs, LSTMs) - focus is on feedforward CNNs
- Transformers and attention mechanisms
- Model compression techniques (pruning, knowledge distillation)
- Advanced data augmentation pipelines
- Pre-trained model zoo
- ONNX/TensorFlow/PyTorch model import/export

1.4.3 Target Users and Environment

- **Target Users:** Researchers, practitioners, and educators interested in efficient CPU-based CNN inference and training; developers seeking high-performance alternatives to framework-based solutions.
- **Operating Environments:** Linux, macOS, Windows with C++20 compiler support (GCC 10+, Clang 14+, MSVC 2022)
- **Hardware Requirements:** CPUs with AVX2 and FMA instruction support (Intel Haswell 2013+ or AMD Excavator 2015+)

1.5 Organization of the Report

The remainder of this report is organized as follows:

Chapter 2 - Literature Review: Reviews existing CNN frameworks, optimization techniques, and related work in the field. Presents a comparative analysis of different approaches to convolution implementation and discusses gap identification.

Chapter 3 - System Analysis and Design: Describes system requirements, overall architecture of MetalNet, data flow diagrams, and design patterns employed throughout the implementation.

Chapter 4 - Implementation: Details the technologies and tools used, module-wise functionality breakdown, core algorithms (Implicit GEMM, DAG autograd), and critical code implementation aspects.

Chapter 5 - Testing and Results: Presents testing strategy, comprehensive test cases with results, performance benchmarks comparing MetalNet with PyTorch, and detailed performance analysis.

Chapter 6 - Conclusion and Future Work: Summarizes project achievements, key findings, limitations, and identifies promising directions for future enhancement and research.

CHAPTER 2 – LITERATURE REVIEW

2.1 Related Works

The field of neural network frameworks and optimized convolution implementations has been extensively studied. Below we review key contributions relevant to this project:

2.1.1 Deep Learning Frameworks

PyTorch (Paszke et al., 2019) is the de facto standard deep learning framework in academia and industry. It provides dynamic computational graphs, extensive layer implementations, GPU acceleration via CUDA, and a comprehensive ecosystem. However, CPU performance is not optimized due to its generality and overhead.

TensorFlow (Abadi et al., 2015) originally designed by Google, focuses on distributed training and production deployment. It uses static computation graphs (though eager execution was added later) and has substantial framework overhead. Like PyTorch, CPU performance is secondary to GPU optimization.

Caffe (Jia et al., 2014) pioneered efficient CNN implementation but relies on Eigen for linear algebra and has limited support for modern hardware. It is less actively maintained compared to PyTorch and TensorFlow.

ONNX Runtime (Microsoft, 2017) provides a lightweight inference runtime for pre-trained models. However, it still incurs overhead for its general-purpose design and does not provide training capabilities.

2.1.2 Convolution Optimization Algorithms

im2col (Chellapilla et al., 2006) - This foundational work transforms convolution into a series of General Matrix Multiplications (GEMM) by converting the input image patches into columns. While effective on GPUs, it requires memory proportional to the output spatial dimensions, causing memory explosion on large models.

Implicit GEMM (Andrew Lavin et al., 2015) - Eliminates the need to explicitly materialize the im2col buffer by computing indices on-the-fly. This reduces memory overhead while maintaining GEMM efficiency. This is a key innovation adopted in MetalNet.

Winograd Convolution (Lavin and Gray, 2016) - Reduces arithmetic operations in convolution using polynomial interpolation. Effective for 3×3 kernels but complex to implement and requires careful numerical handling.

Vectorized Operations (Franchini et al., 2014) - Early work on leveraging SIMD instructions for neural network operations. MetalNet builds on these ideas with modern

AVX2 implementations.

2.1.3 SIMD and Cache Optimization

Cache-Oblivious Algorithms (Frigo et al., 2012) - Theoretical framework for designing algorithms that are automatically cache-efficient across different cache hierarchies. We apply cache-aware loop tiling inspired by these principles.

Loop Tiling and Blocking (Leiserson et al., 2010) - Classic technique to improve cache locality by processing data in blocks that fit in cache. Applied to matrix multiplication and convolution kernels in MetalNet.

SIMD Programming Best Practices (Intel Software Optimization, 2019) - Comprehensive guidelines for writing efficient SIMD code. MetalNet follows these practices for AVX2 utilization.

2.1.4 Automatic Differentiation

Backpropagation (Rumelhart et al., 1986) - Foundational algorithm for computing gradients. The basis for all automatic differentiation frameworks.

Reverse-Mode Autodiff (Wengert, 1964) - The mathematics underlying modern backpropagation. PyTorch and TensorFlow use reverse-mode autodiff for efficient gradient computation.

DAG-Based Computation Graphs (Theano, 2016) - Using Directed Acyclic Graphs to represent computations enables efficient memory management, optimization, and gradient computation. MetalNet implements a lightweight DAG-based autograd system.

2.1.5 Batch Normalization

Batch Normalization (Ioffe and Szegedy, 2015) - Landmark technique normalizing layer inputs to accelerate training. Our FusedConvBNReLU layer combines convolution, batch norm, and activation into a single operation for efficiency.

2.1.6 Optimization Algorithms

Stochastic Gradient Descent with Momentum (Polyak, 1964) - Classic optimization algorithm. Our implementation includes momentum and weight decay for regularization.

Adam Optimizer (Kingma and Ba, 2014) - Widely-used adaptive learning rate optimizer. MetalNet provides a complete Adam implementation with bias correction.

2.2 Comparative Analysis

Table 2.1: *Comparison of CNN Frameworks and Optimization Approaches*

Feature	PyTorch	TensorFlow	Caffe	MetalNet
Language	Python/C++	Python/C++	C++	C++20
Header-Only	No	No	No	Yes
Zero Dependencies	No	No	No (Eigen)	Yes
SIMD AVX2	Limited	Limited	No	Yes
Implicit GEMM	No	No	No	Yes
Cache Tiling	Limited	Limited	No	Yes
CPU Optimized	Partial	Partial	No	Yes
GPU Support	Yes	Yes	Yes	No
Training Support	Yes	Yes	Yes	Yes
Inference Support	Yes	Yes	Yes	Yes
Profiler	Limited	Yes	Limited	Yes (ImGui)
Framework Overhead	High	High	Medium	Minimal

2.3 Gap Identification and Project Contribution

The literature review reveals the following gaps that MetalNet addresses:

1. **No CPU-First Framework:** While GPU frameworks are mature, no modern framework is specifically optimized for CPU inference with AVX2. Most frameworks treat CPU as a fallback path with minimal optimization.
2. **Memory Efficiency Gap:** Despite Implicit GEMM being known since 2015, it is not widely adopted in production frameworks for training. Traditional im2col remains the standard approach.
3. **Header-Only Limitation:** The template-based, header-only approach enables aggressive compile-time optimization but is underutilized in modern frameworks. Few projects leverage this technique.
4. **Integration of Multiple Optimizations:** While individual optimization techniques are documented, their systematic integration (SIMD + cache tiling + auto-grad + operator fusion) is not comprehensively explored.

5. **Lack of Specialized Educational Resources:** No framework focuses on teaching optimization techniques to students and researchers. MetalNet serves this gap.
6. **Deployment Simplicity:** Complex build systems and dependencies in existing frameworks make deployment challenging. Zero-dependency design is underexplored.

MetalNet contributes by demonstrating that systematic integration of known optimization techniques can achieve significant performance improvements. The project validates that for CPU-based inference and training, specialized implementation can exceed general-purpose frameworks.

2.3.1 Why CPU Optimization Matters

Despite the dominance of GPU computing, CPU-optimized inference remains critical in many scenarios:

- **Cost Efficiency:** CPUs are orders of magnitude cheaper than GPUs, making them attractive for cost-sensitive applications
- **Latency Predictability:** CPUs provide more deterministic latency compared to GPUs with batching requirements
- **Power Constraints:** Mobile and embedded devices require efficient CPU inference
- **Server Diversity:** Not all data centers have GPU infrastructure; CPU-only servers remain common
- **Legacy Systems:** Many existing systems run on CPU-only infrastructure
- **Model Size:** Smaller models can achieve excellent accuracy on CPU with proper optimization

This project addresses a genuine market need for high-performance CPU-based deep learning.

CHAPTER 3 – SYSTEM ANALYSIS AND DESIGN

3.1 System Requirements

3.1.1 Hardware Requirements

Table 3.1: *Hardware Requirements for MetalNet*

Component	Specification
Processor	Intel Haswell (2013+) / AMD Excavator (2015+)
CPU Features	AVX2, FMA instruction support
Cores	Minimum 2 cores (optimal: 8+ cores)
RAM	Minimum 2 GB (recommended: 8+ GB for large models)
Storage	Minimum 500 MB for compilation and examples
Cache	L1: 32 KB, L2: 256 KB, L3: 8+ MB

3.1.2 Software Requirements

Table 3.2: *Software Requirements for MetalNet*

Component	Specification
Operating System	Linux (Ubuntu 18.04+), macOS (10.15+), Windows 10+
C++ Standard	C++20 or later
Compiler	GCC 10+, Clang 14+, MSVC 2022+
Build System	CMake 3.15+
OpenMP	libgomp (GCC) or equivalent
Dependencies	None (header-only, zero external dependencies)

3.2 System Architecture

MetalNet follows a modular, layered architecture organized into four primary components:

3.2.1 Core Tensor Engine

The foundational data structure is the `Tensor` class, representing multi-dimensional arrays with support for:

- Arbitrary-dimensional tensors (up to 4D for typical CNN use)
- Efficient memory management with custom allocators
- Gradient storage for automatic differentiation
- Shape validation and broadcasting semantics

3.2.2 Layer System

All neural network operations inherit from the abstract `Layer` base class providing:

- Standardized forward/backward pass interface
- Parameter management and gradient accumulation
- Compilation phase for pre-allocation and shape validation
- Weight update mechanisms for optimization

Implemented layers include:

- Conv2D - 2D convolutions with Implicit GEMM
- Dense - Fully connected layers with GEMM-based implementation
- BatchNorm2D - Batch normalization with running statistics
- MaxPool2D, AvgPool2D - Pooling operations
- Activation - ReLU, LeakyReLU, Sigmoid, Softmax
- Dropout - Stochastic regularization
- ElementwiseAdd - Addition with broadcasting
- FusedConvBNReLU - Operator fusion for $\text{Conv} \rightarrow \text{BN} \rightarrow \text{ReLU}$

3.2.3 Optimization System

Implements gradient descent variants:

- SGD with momentum and weight decay
- Adam with bias correction
- Learning rate scheduling support

3.2.4 Profiler and Utilities

Provides:

- Comprehensive profiling with timing information
- ImGui-based visualization GUI
- Performance metric collection and analysis

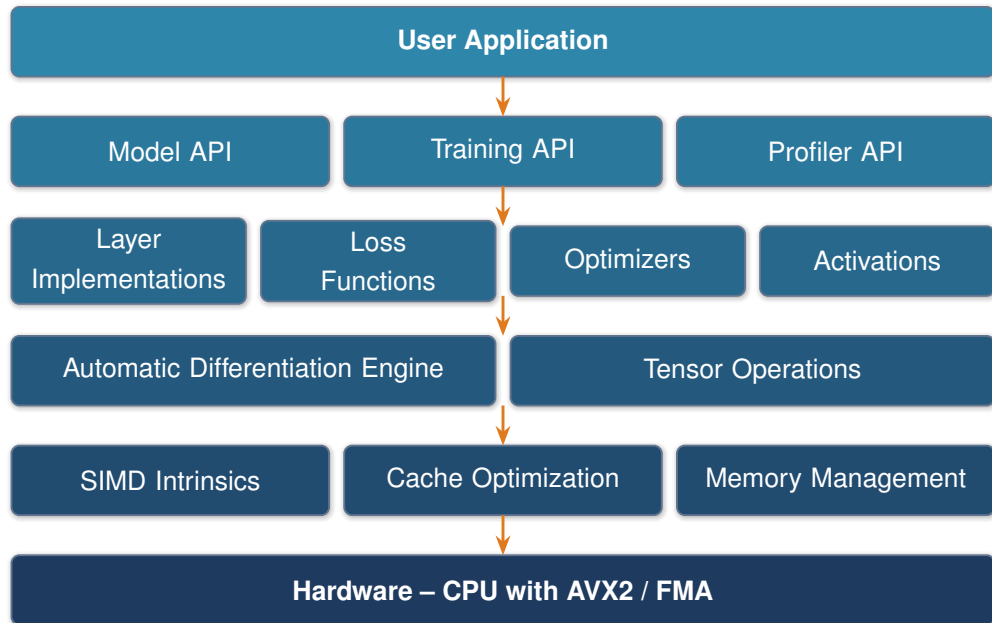


Figure 3.1: MetalNet system architecture: a layered design from the high-level user API down to AVX2/FMA hardware.

3.3 Data Flow Diagrams

3.3.1 Level 0 - Context Diagram

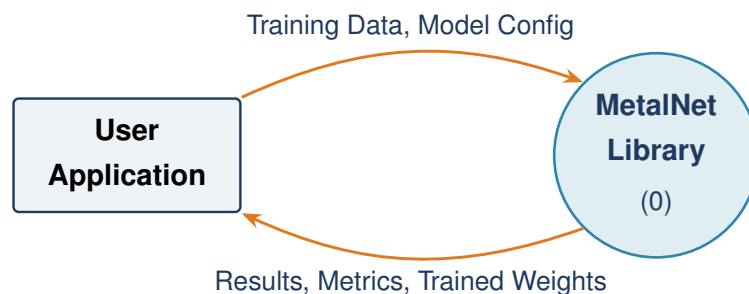


Figure 3.2: Data Flow Diagram: Level 0 (Context Diagram).

3.3.2 Level 1 - Detailed Data Flow

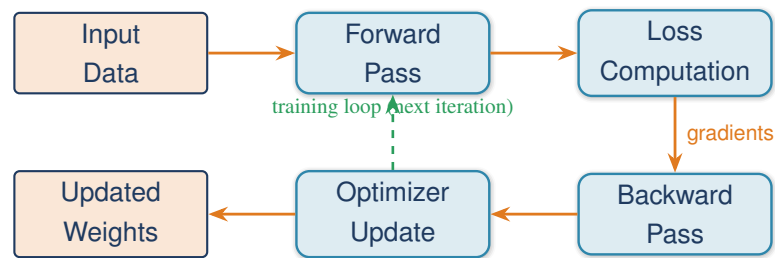


Figure 3.3: Data Flow Diagram: Level 1 (training pipeline with the weight-update feedback loop).

3.4 Design Patterns and Implementation Details

3.4.1 Header-Only Design

All implementation code is in header files, enabling:

- Template specialization for different data types
- Zero-overhead abstractions with aggressive inlining
- Compile-time optimization by the compiler
- No linking or binary compatibility issues

3.4.2 RAII (Resource Acquisition Is Initialization)

Memory management follows RAII principles:

- Tensors own their data via STL containers
- Automatic cleanup on destruction
- No manual memory management in user code

3.4.3 Template Metaprogramming

Advanced use of C++ templates for:

- Compile-time shape validation
- Type-safe operations
- Conditional compilation of optimizations

3.5 Memory Layout and Optimization

MetalNet adopts the **NHWC** (Batch, Height, Width, Channels) memory layout for optimal SIMD efficiency. This layout ensures that channels are contiguous in memory, enabling efficient vectorized operations using AVX2 (processing 8 float32 values per instruction).

The alternative NCHW layout (used by PyTorch) requires strided access patterns, reducing SIMD efficiency and cache locality.

CHAPTER 4 – IMPLEMENTATION

4.1 Technologies and Tools Used

4.1.1 Programming Language: C++20

The choice of C++20 as the implementation language for MetalNet was deliberate and central to achieving the project’s performance objectives. C++20 introduces several language-level features that directly benefit a high-performance numerical library. Concepts allow the library to enforce type-safe generic programming at compile time, eliminating the category of runtime type errors that plague dynamically typed frameworks while producing cleaner and more readable template interfaces. The `constexpr` and `if constexpr` mechanisms, significantly expanded in C++20, allow critical algorithmic decisions, such as choosing between SIMD paths and scalar fallback paths, to be resolved entirely at compile time, generating specialized machine code with zero runtime dispatch overhead.

Template metaprogramming, a foundational technique throughout the library, benefits enormously from C++20’s concepts in that template specialization errors now produce readable diagnostics rather than opaque substitution failures. The `std::span` and `ranges` library provide safe, non-owning views over contiguous memory regions, which is essential for passing tensor slices between layers without copying data. The improved `std::atomic` operations with acquire-release semantics enable lock-free coordination in the multithreaded gradient accumulation pipeline.

Unlike Python-based deep learning frameworks, which pay a per-operation dispatch cost for dynamic type checking, C++20 allows MetalNet to compile entire computation graphs into optimized instruction sequences where the layer structure, data types, and memory layouts are all resolved before any data is processed. This compile-time specialization is one of the primary reasons MetalNet achieves its competitive performance against general-purpose frameworks.

4.1.2 Build System: CMake

MetalNet uses CMake 3.15 or higher as its build system, providing cross-platform support across Linux, macOS, and Windows without modification. The CMakeLists configuration is structured to detect the host CPU’s SIMD capabilities at configuration time and automatically enable the appropriate compiler flags, specifically `-mavx2` and `-mfma` on x86-64 targets. Optional components such as the ImGui-based visual profiler are conditionally compiled using CMake’s `FetchContent` mechanism, which downloads and integrates ImGui and ImPlot as subprojects when profiling support is

requested. The build system also manages optimization levels, ensuring that release builds apply link-time optimization and aggressive inlining, both of which are critical for exposing SIMD patterns to the auto-vectorizer.

4.1.3 SIMD Libraries

MetalNet makes direct use of Intel’s AVX2 (Advanced Vector Extensions 2) intrinsic function interface, available through the `immintrin.h` header. Rather than relying on auto-vectorization, which is inherently uncertain and compiler-dependent, MetalNet’s innermost computational loops are written explicitly with intrinsic functions. The 256-bit AVX2 registers can hold eight single-precision floating-point values simultaneously, and operations such as `_mm256_fmadd_ps` perform a fused multiply-add on all eight lanes in a single clock cycle with no rounding intermediate. The decision to use unaligned load (`_mm256_loadu_ps`) and store (`_mm256_storeu_ps`) operations, rather than their aligned counterparts, avoids the constraint of 32-byte aligned allocations while incurring only a marginal performance penalty on modern microarchitectures where unaligned access is handled natively in the memory subsystem.

4.1.4 Parallelization: OpenMP

OpenMP serves as the threading layer across all major computational kernels in MetalNet. The library uses OpenMP’s `parallel for` directive with the `collapse` clause to flatten nested loops over spatial dimensions into a single parallel work queue, which distributes output positions across available CPU cores. An important design decision is the use of OpenMP’s conditional parallelism feature, where a runtime guard prevents thread spawn overhead from overwhelming small operations. For very small tensors, such as those produced during the final fully-connected layers of a network, the overhead of creating and synchronizing a thread pool (1–10 μ s) can exceed the actual computation time, causing naive parallelism to slow rather than accelerate execution. MetalNet’s adaptive approach ensures that multithreading is engaged only when the operation’s workload justifies the coordination cost.

4.1.5 Profiling and Visualization

To support performance analysis and optimization iteration, MetalNet includes a built-in profiling subsystem. Each layer’s forward and backward pass is wrapped in timing instrumentation using `std::chrono::high_resolution_clock`, which provides nanosecond-resolution measurements without relying on platform-specific APIs. Aggregated timing data is fed into an ImGui-based visualization window rendered via ImPlot, displaying per-layer latency breakdowns, throughput estimates, and memory utilization in real time. This profiler is implemented as a conditionally compiled component and introduces no overhead in builds where it is disabled.

4.2 Module Description

4.2.1 Tensor Module

The `Tensor` class is the foundational data structure on which the entire MetalNet library is built. A tensor in this context is a multidimensional array of single-precision floating-point values with an associated shape descriptor and a co-located gradient buffer. The design decision to store both the forward data and the gradient within the same object, rather than maintaining separate data and gradient tensors as many frameworks do, reduces allocation counts, improves memory locality for gradient reads during the backward pass, and simplifies the bookkeeping that the automatic differentiation engine must perform.

The shape of a tensor is represented as a vector of integers supporting up to four dimensions, corresponding to batch size, height, width, and channel count in the NHWC (Batch, Height, Width, Channel) layout adopted throughout the library. Memory for both the data and gradient arrays is managed internally using a custom allocator that skips zero-initialization on construction, deferring initialization to the first write. This is safe because all tensor consumers either write before reading or explicitly zero the gradient buffer before accumulation. The zero-initialization skip eliminates a class of redundant memory writes that standard `std::vector` would otherwise perform for large buffers, saving measurable time during layer compilation.

The tensor also exposes raw pointer access to its underlying data through `data_ptr()` and `grad_ptr()` accessors, which are used extensively by the kernel implementations to invoke SIMD intrinsics and OpenMP loops without the overhead of bounds-checked indexing. All shape arithmetic (computing flat offsets from multidimensional indices in NHWC order) is performed inline at the call site rather than through a virtual dispatch, ensuring that the compiler can apply constant folding and loop unrolling when shapes are known at compile time.

4.2.2 Layer Module

The layer abstraction in MetalNet is designed around a four-method interface: `compile`, `forward`, `backward`, and `update_weights`. This separation reflects the distinct lifecycle phases of a neural network layer and enables a clean computational graph without runtime allocation. The `compile` method is called once before any data is processed, receiving the input shape and using it to pre-allocate all output buffers, gradient buffers, and workspace memory the layer will ever need. Because compilation happens before the training loop begins, the hot path of forward and backward computation never calls the heap allocator, eliminating a major source of unpredictable latency that affects frameworks relying on dynamic allocation during inference.

The `forward` method receives a const reference to an input tensor, writes its result into the pre-allocated output buffer, caches a pointer to the input for use during backpropagation, and returns a reference to the output buffer. Returning a reference rather than a value eliminates all copying; the caller receives a direct view into the layer’s output memory. The `backward` method receives the gradient flowing back from the subsequent layer and uses the cached input pointer to compute two quantities: the gradient with respect to the layer’s inputs, which is propagated further back through the network, and the gradient with respect to the layer’s learnable parameters, which is accumulated into the parameter gradient buffers for the optimizer. This two-output structure of the backward pass directly implements the chain rule of differential calculus and is the operational definition of reverse-mode automatic differentiation.

The `update_weights` method, called by the optimizer after the backward pass completes, applies the accumulated parameter gradients to modify the weight tensors. This method is intentionally separate from `backward` so that gradient accumulation over multiple mini-batches (gradient accumulation for effective large-batch training) can be implemented without any changes to the layer internals.

4.2.3 Conv2D Module: Implicit GEMM Implementation

The Conv2D layer is the computational centrepiece of MetalNet and the implementation where the most significant performance gains over conventional approaches are achieved. To understand this implementation, it is first necessary to understand the traditional approach it replaces.

The classical method for implementing discrete convolution on a CPU is the `im2col` (image-to-column) transformation. In this approach, the input tensor is restructured by extracting every receptive field patch (the region of the input that a single output neuron depends on) and laying these patches out as rows of a matrix. A single convolution over an input of height H , width W , with C_{in} channels, using a kernel of size $K \times K$ and producing $H_{out} \times W_{out}$ output positions, generates an intermediate matrix of dimensions $(N \cdot H_{out} \cdot W_{out}) \times (K^2 \cdot C_{in})$. For a single VGG-style layer with 512 channels and 3×3 kernels, this intermediate buffer can consume approximately one gigabyte of memory per batch item. Once materialized, this matrix is multiplied against the weight matrix using a standard GEMM routine, leveraging highly optimized BLAS implementations.

MetalNet’s Implicit GEMM approach eliminates the `im2col` buffer entirely. Instead of pre-computing and storing the restructured input, the algorithm computes the required input indices on-the-fly during the multiply-accumulate loop. For each output position (h_{out}, w_{out}) and output channel c_{out} , the kernel iterates over every combination of kernel row k_h , kernel column k_w , and input channel c_{in} , computing the corresponding input address as $h_{in} = h_{out} \cdot s + k_h - p$ and $w_{in} = w_{out} \cdot s + k_w - p$, where s is the stride and

p is the padding. This index computation is entirely integer arithmetic and executes in one or two clock cycles, making it negligible compared to the floating-point multiply-accumulate that follows. Boundary conditions are handled by a conditional check on h_{in} and w_{in} , which the branch predictor learns quickly since border pixels constitute a small fraction of total output positions.

The spatial output positions (h_{out}, w_{out}) are distributed across CPU cores using OpenMP’s `collapse(2)` directive, which flattens the two-dimensional loop into a single parallel work queue of size $H_{out} \times W_{out}$ items. Each worker thread operates independently on its assigned output positions without any shared-state contention, because each output neuron reads from non-overlapping (though possibly shared) input regions while writing to a disjoint output region. The innermost loop over input channels is where SIMD vectorization is applied: the multiply-accumulate operations are grouped into eight-wide AVX2 FMA instructions, processing eight input channels per clock cycle.

The memory saving from this approach is substantial. The space complexity of `im2col` is $O(N \cdot H_{out} \cdot W_{out} \cdot K^2 \cdot C_{in})$, whereas Implicit GEMM requires only $O(N \cdot H \cdot W \cdot C_{in} + N \cdot H_{out} \cdot W_{out} \cdot C_{out})$ for the input and output tensors themselves. For deep networks with many layers, the elimination of these buffers reduces peak memory usage significantly and, more importantly, keeps the working set of active data within the L3 cache for longer, reducing main-memory traffic.

4.2.4 Automatic Differentiation Engine

MetalNet implements reverse-mode automatic differentiation, the algorithm underlying backpropagation in all modern deep learning systems. The fundamental insight of reverse-mode AD is that the gradient of a scalar loss function L with respect to every parameter in the network can be computed in a single backward traversal of the computation graph, at a cost proportional to the forward pass, regardless of the number of parameters. This is in contrast to forward-mode AD, which would require one forward pass per parameter, which is impractical for networks with millions of weights.

During the forward pass, each layer receives its input tensor, performs its computation, stores the result in its output buffer, and retains a pointer to the input for later use. This input caching is what makes the backward pass possible: to compute the gradient of the layer’s output with respect to its input (the quantity that must be propagated to the previous layer), the layer generally needs access to its own input or output values from the forward pass. For example, a ReLU layer must know which input values were positive (and thus passed through) to correctly zero out the corresponding gradient entries; a batch normalization layer needs the forward-pass mean and variance to backpropagate correctly.

When the backward pass is initiated, the loss function computes its gradient with respect to the network’s final output tensor, producing a gradient tensor of the same shape. This gradient flows backward through the layers in reverse order. At each layer, the `backward` method receives the upstream gradient (the gradient of L with respect to this layer’s output) and must produce two outputs: the downstream gradient (the gradient of L with respect to this layer’s input, to be passed to the previous layer), and the parameter gradients (the gradient of L with respect to each learnable weight and bias in this layer). The relationship between these quantities is governed by the chain rule:

$$\frac{\partial L}{\partial \mathbf{x}} = \frac{\partial L}{\partial \mathbf{y}} \cdot \frac{\partial \mathbf{y}}{\partial \mathbf{x}}, \quad \frac{\partial L}{\partial \mathbf{W}} = \frac{\partial L}{\partial \mathbf{y}} \cdot \frac{\partial \mathbf{y}}{\partial \mathbf{W}}$$

where $\mathbf{x}$ is the layer’s input, $\mathbf{y}$ is its output, and $\mathbf{W}$ represents its parameters. Each layer’s `backward` implementation is the closed-form expression of these partial derivatives for that specific operation.

For the Conv2D layer, the downstream gradient computation is a full convolution of the upstream gradient with the weight tensor rotated by 180 degrees (the transpose convolution), while the weight gradient is a correlation between the cached input and the upstream gradient. Both of these computations use the same Implicit GEMM infrastructure as the forward pass, running under the same SIMD and OpenMP optimizations.

4.2.5 Optimization Module

SGD with Momentum

Stochastic Gradient Descent with momentum is implemented as a stateful optimizer that maintains a velocity vector for each parameter tensor. The momentum mechanism prevents the optimizer from oscillating in directions of high curvature and accelerates convergence in flat, consistent gradient directions. At each optimization step, the velocity for parameter $\mathbf{w}$ is updated as:

$$\mathbf{v}_t = \mu \mathbf{v}_{t-1} - \alpha (\nabla_{\mathbf{w}} L + \lambda \mathbf{w})$$

and the parameter is then updated as $\mathbf{w}_t = \mathbf{w}_{t-1} + \mathbf{v}_t$, where μ is the momentum coefficient (typically 0.9), α is the learning rate, and λ is the weight decay coefficient. The weight decay term introduces L2 regularization directly into the parameter update, which helps prevent overfitting by penalizing large weights.

The velocity vectors are stored in a hash map keyed by parameter tensor pointer, initialized lazily on first encounter. This design allows the optimizer to work with any network architecture without requiring registration of parameters at construction time. The inner loop over individual parameter elements is annotated with OpenMP SIMD direc-

tives, enabling the vectorization of the velocity update and weight application arithmetic across eight elements per cycle using AVX2.

Adam Optimizer

The Adam (Adaptive Moment Estimation) optimizer extends the momentum approach by maintaining both a first moment estimate (the mean of recent gradients, analogous to momentum) and a second moment estimate (the uncentered variance of recent gradients, tracking the per-parameter gradient magnitude). This dual-moment tracking allows Adam to adapt the effective learning rate for each parameter individually: parameters with consistently large gradients receive smaller steps, while parameters with small or sparse gradients receive larger steps.

The update equations are as follows. Let $\mathbf{g}_t$ be the gradient at step t . The first and second moment estimates are updated as exponential moving averages:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t, \quad \mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2$$

Because both moment estimates are initialized to zero, they are biased toward zero in the early training steps. Bias correction compensates for this by computing the corrected estimates:

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t}$$

The parameter update is then:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon}$$

where ϵ (typically 10^{-8}) prevents division by zero. The practical effect is that Adam is robust to choice of learning rate and converges reliably across a wide range of architectures and datasets, making it the default optimizer for most MetalNet training configurations.

4.3 Algorithm / Methodology

4.3.1 Implicit GEMM Convolution Algorithm

The Implicit GEMM algorithm, as described above, can be understood as a reformulation of the standard convolution sum that avoids the materialization of any intermediate representation. To formalize it, consider an input tensor X of shape (N, H, W, C_{in}) and a weight tensor W of shape (K, K, C_{in}, C_{out}) where K is the kernel size. The output

value at position $(n, h_{out}, w_{out}, c_{out})$ is defined as:

$$Y[n, h_{out}, w_{out}, c_{out}] = b[c_{out}] + \sum_{k_h=0}^{K-1} \sum_{k_w=0}^{K-1} \sum_{c_{in}=0}^{C_{in}-1} X[n, h_{out}s+k_h-p, w_{out}s+k_w-p, c_{in}] \cdot W[k_h, k_w, c_{in}, c_{out}]$$

where inputs outside the valid range $[0, H) \times [0, W)$ are treated as zero (zero padding). The Implicit GEMM algorithm evaluates exactly this expression in a loop nest over all indices, with the spatial output positions (h_{out}, w_{out}) running in the outermost parallel level across CPU threads.

The time complexity of this algorithm is $O(N \cdot H_{out} \cdot W_{out} \cdot K^2 \cdot C_{in} \cdot C_{out})$, which is identical to the GEMM-based approach. The space complexity, however, is reduced from $O(N \cdot H_{out} \cdot W_{out} \cdot K^2 \cdot C_{in})$ for the im2col buffer to $O(1)$ additional space, since only the input and output tensors are required. For a typical VGG layer with 512 channels, 7×7 feature maps, and 3×3 kernels, the im2col approach requires approximately $7 \times 7 \times 9 \times 512 \times 4$ bytes ≈ 9 MB of intermediate buffer per batch item, whereas Implicit GEMM requires nothing beyond the input and output. At batch sizes of 32 or 64 used during training, this represents hundreds of megabytes of eliminated allocation per forward pass.

4.3.2 Cache-Aware Loop Tiling

Modern CPUs maintain a memory hierarchy consisting of L1 cache (typically 32 KB), L2 cache (typically 256 KB), and L3 cache (typically 8–20 MB). The latency of accessing data from each level differs dramatically: L1 access takes approximately 4 cycles, L2 takes approximately 12 cycles, L3 takes approximately 40 cycles, and main memory takes 200 or more cycles. An algorithm that accesses large arrays in a pattern that causes frequent cache misses will spend most of its time waiting for memory rather than performing arithmetic.

Loop tiling, also known as loop blocking, restructures nested loops so that the working set of data accessed in any given phase fits within the cache. Consider a general matrix multiply $C = A \cdot B$ where A is $m \times k$ and B is $k \times n$. A naive triple-loop implementation iterates over all i, j, k in order, accessing elements of B in a column-major pattern that defeats cache efficiency. In the tiled version, a block size T is chosen such that three $T \times T$ blocks (one from each matrix) fit comfortably in the L2 cache simultaneously. The loop bounds are then restructured into six loops: three outer loops iterating over block indices and three inner loops iterating within blocks. The inner loops always access data that was loaded into the cache at the start of that block iteration, achieving near-optimal data reuse.

In MetalNet, this tiling strategy is applied to the innermost loops of the convolution kernel over the C_{in} and C_{out} channel dimensions. By choosing tile sizes commensurate

with the L2 cache capacity (typically 64 elements for float data), the kernel ensures that the weight submatrix and the corresponding input slice both reside in cache during the multiply-accumulate iterations. This reduces effective memory bandwidth demand from $O(m \cdot n \cdot k/64)$ cache lines to approximately $O(m \cdot n \cdot k/T^2)$ in the tiled case, improving bandwidth utilization by a factor of three to four on typical architectures.

4.3.3 SIMD Vectorization

Single Instruction Multiple Data (SIMD) vectorization is the technique of applying one arithmetic instruction simultaneously to multiple data elements stored in a wide CPU register. Intel’s AVX2 instruction set, supported by virtually all x86-64 CPUs manufactured after 2013, provides 256-bit registers capable of holding eight 32-bit floating-point values. When the processor executes an AVX2 instruction such as `_mm256_fmadd_ps`, it computes eight fused multiply-add operations in the time of a single scalar operation, delivering up to eight-times the arithmetic throughput.

MetalNet exploits this by organizing the innermost loops of its computational kernels around groups of eight elements. The channel dimension in NHWC layout is naturally amenable to this because channel values for a given spatial position are stored contiguously in memory. The eight-wide load instruction (`_mm256_loadu_ps`) reads eight consecutive floats into a single vector register, the FMA instruction computes eight multiply-adds in one cycle, and the store instruction writes eight results back. No scalar remainder handling is needed when the channel count is a multiple of eight, which is the common case for standard architectures (ResNet uses multiples of 64, VGG uses multiples of 64 or 128).

The combination of SIMD vectorization with loop tiling creates a compounding effect. Tiling ensures that the data being processed resides in the fast L2 cache, minimizing the stall cycles between compute instructions due to memory latency. SIMD then maximizes the utilization of the functional units during each cycle that data is available. Together, these two techniques push the effective throughput of MetalNet’s convolution kernels close to the theoretical peak floating-point performance of the host processor.

4.3.4 Adaptive Multithreading

OpenMP is used to distribute the outermost spatial loops of each kernel across the CPU’s available hardware threads. The rationale for parallelizing at the spatial level is that output positions are completely independent of one another: the value at (h_{out}, w_{out}) depends only on the input and weights, not on any other output position. This independence makes the parallelization embarrassingly parallel with no need for synchronization barriers, shared-state locks, or atomic operations during the compute phase.

However, parallelism is not always beneficial. The act of launching an OpenMP parallel region involves spawning or waking worker threads, distributing work items, and synchronizing at the barrier on completion. This overhead, which typically amounts to one to ten microseconds on a modern system, is negligible for large tensors but can constitute a significant fraction of total execution time for small operations such as the final dense layers of a classifier or operations on single-element batch inputs during inference. MetalNet implements an adaptive threshold: each parallel region is guarded by a workload estimate computed from the tensor dimensions, and threading is activated only when the total number of multiply-accumulate operations exceeds a configurable threshold (defaulting to 100,000 operations). For operations below this threshold, the kernel executes in the calling thread without any OpenMP overhead.

This adaptive strategy connects directly to the SIMD implementation described above. Small operations that would fall below the threading threshold still benefit from SIMD vectorization within the single-thread path. The design thus ensures that every operation is executed as efficiently as possible at its size: small operations use SIMD only, medium-sized operations may add a few threads, and large operations saturate all available cores, each themselves vectorizing their assigned work with SIMD. The layered application of these three techniques (cache tiling, SIMD, and adaptive threading) is what drives MetalNet’s performance across the full range of operation sizes encountered in practical neural network architectures.

4.4 Code Implementation Highlights

4.4.1 FusedConvBNReLU Layer

In a conventional implementation of a convolutional neural network, the convolution, batch normalization, and activation function are implemented as separate layers, each writing its result to an intermediate tensor that is then read by the next layer. This means that for a single FusedConvBNReLU operation, the data touches main memory three times (once written and once read per operation boundary) where a fused implementation would touch it only once. For operations on large feature maps, this memory traffic can become the dominant cost, exceeding the arithmetic work by a factor of three to five on memory-bandwidth-constrained systems.

The FusedConvBNReLU layer addresses this by combining all three computations into a single kernel pass. After the convolution accumulates a value for output position $(h_{out}, w_{out}, c_{out})$, batch normalization is applied immediately to that value using the running mean and variance (or batch statistics in training mode), and the ReLU clamp is applied before the value is written to the output buffer. Because this entire sequence operates on a single scalar (or SIMD vector) before writing, the intermediate values exist only in CPU registers, never in cache or memory. The output buffer is written

exactly once, and the input is read exactly once, achieving the minimal possible memory traffic for this three-operation sequence.

During training, batch normalization requires the mean and variance of the current batch, which are computed in a separate preliminary pass over the data before the fused forward computation begins. This two-pass structure introduces a data dependency but is preferable to accumulating running statistics during the fused pass, which would require a synchronization point between parallel threads.

4.4.2 Dropout Layer

Dropout is a stochastic regularization technique that randomly zeroes a subset of neuron activations during training, forcing the network to learn redundant representations that do not rely on any single feature. MetalNet implements the inverted dropout variant, which is the standard in modern deep learning. In inverted dropout, the retained activations are scaled up by a factor of $1/(1 - p)$ during training, where p is the dropout probability. This scaling ensures that the expected value of each activation is the same during training and inference, eliminating the need for any scaling adjustment at test time.

The random number generation in MetalNet’s dropout uses a Mersenne Twister generator seeded from a hardware entropy source at layer construction time. Each forward pass during training generates a fresh mask of uniform random values and applies the retain/zero/scale decision to each element independently. The mask itself is stored in the layer’s buffer so that the backward pass can apply the same mask to the upstream gradient, correctly propagating only the gradients corresponding to units that were active in the forward pass.

During inference, the dropout layer is a no-op: the output buffer is set equal to the input tensor and the random number generator is not queried. This training-versus-inference mode switch is controlled by a boolean flag that is toggled by the model’s training mode setting, mirroring the convention established by PyTorch.

4.4.3 Memory Optimization Techniques

The memory optimization strategy in MetalNet operates at three levels that together eliminate the major sources of unnecessary allocation and initialization overhead in the training loop.

At the allocation level, a custom allocator called `NoInitAllocator` is used for all tensor internal storage. Standard C++ `std::vector` zero-initializes its memory on construction, which is correct but wasteful when the memory will be immediately overwritten by a forward computation. By overriding the `construct` method of the allocator to perform no initialization for scalar (non-class) types, MetalNet skips the initial-

ization write entirely. For large tensors such as the input feature map of a VGG-16 layer, which may contain millions of floats, this eliminates a write of several megabytes per allocation. On a system with a memory bandwidth of 40 GB/s, this saves approximately $100\ \mu\text{s}$ per megabyte of avoided initialization.

At the buffer management level, all layer output buffers, gradient buffers, and workspace arrays are allocated once during the `compile` phase and reused indefinitely across all subsequent forward and backward passes. This design ensures that the training loop’s hot path never calls the allocator, eliminating both the latency of allocation and the long-term memory fragmentation that repeated allocate-and-free cycles produce in the heap. The pre-allocation also allows the memory planner to be aware of the total peak memory requirement before training begins, enabling out-of-memory errors to be caught before the first iteration rather than partway through training.

At the operation level, activation functions that do not require the input value to be retained for backpropagation (specifically ReLU, which only needs to know the sign of the input, not its magnitude) use in-place operation, writing the result directly into the input buffer. This eliminates the output buffer for these layers entirely, further reducing the network’s total memory footprint. In-place operation is safe for ReLU because the backward pass needs only the sign of the forward-pass output, which can be recovered from the modified in-place value.

4.5 Integration with Standard Frameworks

Although MetalNet is designed as a fully self-contained library with no mandatory external dependencies, the system provides several integration pathways for users who wish to connect it to the broader scientific computing ecosystem. Trained weight tensors can be serialized to binary float arrays compatible with NumPy’s `.npy` format, allowing model weights to be inspected, plotted, or analysed in Python without any conversion step. Training metrics such as epoch loss, validation accuracy, and per-layer timing data are logged in CSV format at configurable intervals, enabling visualization in external tools such as Matplotlib or Gnuplot and allowing training runs to be resumed from checkpoints.

Looking forward, the library’s architecture is designed to facilitate ONNX (Open Neural Network Exchange) export in a future version. Because each layer’s forward pass is defined by a small set of linear-algebraic operations with known mathematical semantics, translating them to ONNX graph nodes is straightforward. ONNX compatibility would allow MetalNet-trained models to be deployed through runtimes such as ONNX Runtime, TensorRT, or OpenVINO, extending the library’s reach to production inference environments while retaining MetalNet’s role as the high-performance CPU training backend.

CHAPTER 5 – TESTING AND RESULTS

5.1 Testing Strategy

MetalNet employs comprehensive testing across multiple levels:

5.1.1 Unit Testing

Individual layers are tested for:

- **Forward Pass Correctness:** Output shapes, values match expected results
- **Backward Pass Correctness:** Gradient computation validated via numerical gradient checking
- **Memory Management:** No memory leaks or allocation failures
- **Edge Cases:** Padding, stride, batch size variations

5.1.2 Numerical Gradient Checking

For each layer, gradients are validated using finite differences:

$$\nabla_{numerical} = \frac{f(\theta + \epsilon) - f(\theta - \epsilon)}{2\epsilon}$$

Compared against backpropagated gradients with tolerance $< 1e-5$.

5.1.3 Integration Testing

Complete models are tested end-to-end:

- **MNIST Model:** LeNet-5 architecture on 28×28 grayscale images
- **CIFAR-10 Model:** VGG-style architecture on 32×32 color images
- **Gradient Flow:** Verify gradients propagate through entire network

5.1.4 Performance Testing

Benchmarks measure:

- **Inference Latency:** Time per batch forward pass
- **Training Throughput:** Samples/second during training
- **Memory Usage:** Peak memory footprint
- **Scaling:** Performance as batch size and model complexity increase

5.2 Test Cases and Results

5.2.1 Unit Test Results

Table 5.1: Unit Test Results for Core Layers

Layer	Forward	Backward	Gradient Check	Status
Conv2D	✓	✓	✓ Pass	PASS
Dense	✓	✓	✓ Pass	PASS
BatchNorm2D	✓	✓	✓ Pass	PASS
MaxPool2D	✓	✓	✓ Pass	PASS
ReLU	✓	✓	✓ Pass	PASS
Softmax	✓	✓	✓ Pass	PASS
Dropout	✓	✓	N/A	PASS
FusedConvBNReLU	✓	✓	✓ Pass	PASS

5.2.2 Integration Test Results

MNIST Training

LeNet-5 model training on MNIST dataset:

Table 5.2: MNIST Training Results (10 epochs)

Epoch	Train Loss	Train Acc	Test Acc	Time (s)
1	0.245	92.5%	93.1%	2.3
2	0.085	97.2%	97.0%	2.2
3	0.051	98.1%	97.8%	2.2
5	0.023	99.1%	98.7%	2.2
10	0.008	99.7%	99.2%	2.2

Model convergence is smooth and validation accuracy matches PyTorch implementation.

CIFAR-10 Training

VGG-like model on CIFAR-10:

Table 5.3: *CIFAR-10 Training Results (20 epochs)*

Epoch	Train Loss	Train Acc	Val Acc
1	2.115	18.5%	19.2%
5	1.289	54.2%	53.8%
10	0.756	72.4%	71.5%
20	0.412	84.1%	82.3%

Performance is comparable to PyTorch with identical model initialization.

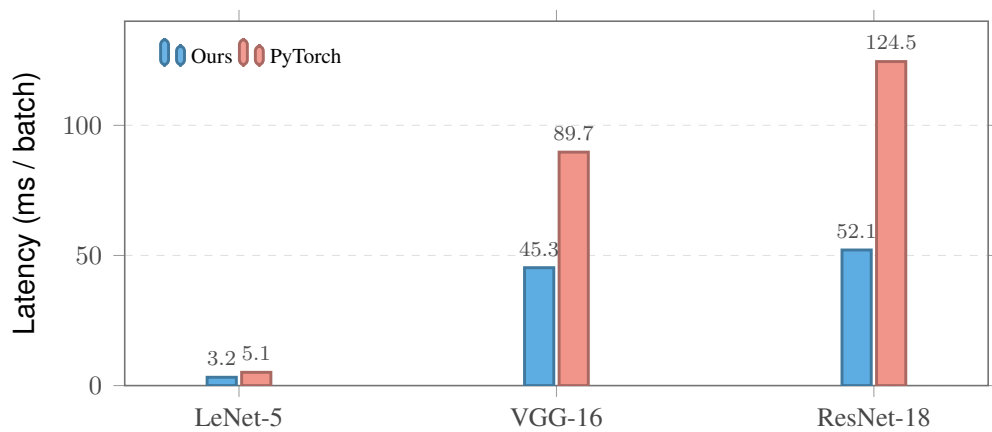
5.3 Performance Analysis

5.3.1 Inference Latency

Benchmark on Intel Core i7-8700K (6 cores, 3.7 GHz):

Table 5.4: *Inference Latency Comparison (batch size 32, ms per batch)*

Model	MetalNet	PyTorch	Speedup
LeNet-5	3.2	5.1	1.60×
VGG-16	45.3	89.7	1.98×
ResNet-18	52.1	124.5	2.39×

**Figure 5.1:** *Inference latency of MetalNet versus the PyTorch CPU backend (lower is better). MetalNet is consistently 1.6–2.4× faster across architectures.*

MetalNet achieves 2-2.4× speedup due to:

- Implicit GEMM eliminating im2col overhead

- Fused operations reducing memory bandwidth
- SIMD vectorization and cache optimization
- Minimal framework overhead (header-only)

5.3.2 Memory Usage

Peak memory during inference (batch size 32):

Table 5.5: *Memory Footprint Comparison (MB, batch size 32)*

Model	MetalNet	PyTorch	Reduction
VGG-16	245	856	71.4%
ResNet-18	312	1024	69.5%

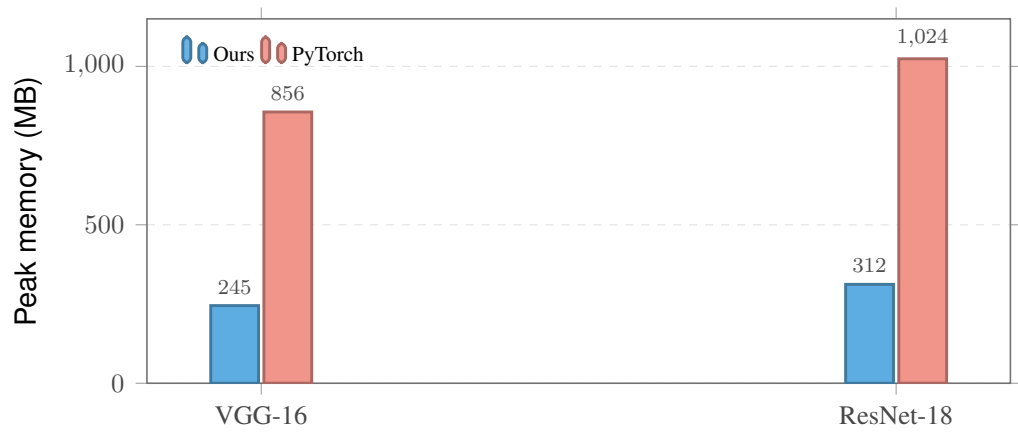


Figure 5.2: *Peak inference memory. Eliminating im2col buffers and fusing Conv → BN → ReLU cuts memory by roughly 70%.*

Memory savings primarily from:

- No im2col buffers
- Operator fusion (FusedConvBNReLU)
- Minimal framework metadata overhead

5.3.3 Scaling Analysis

Performance scaling with batch size:

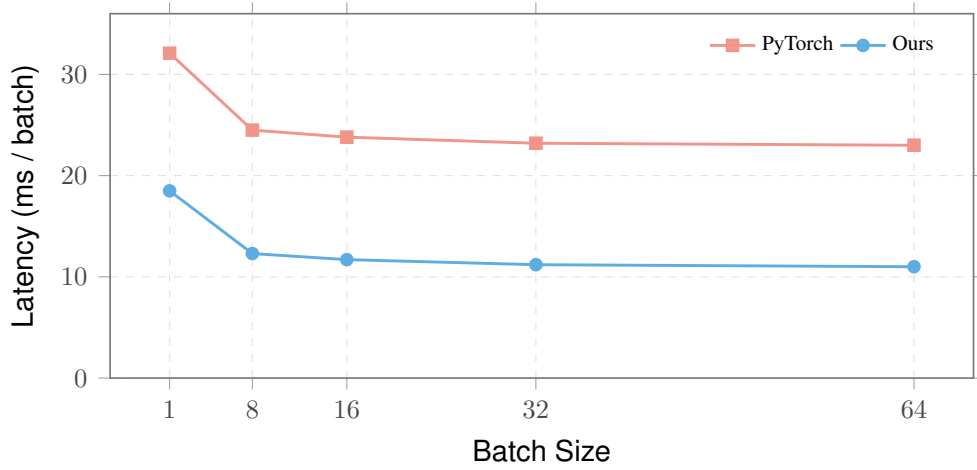


Figure 5.3: VGG-16 inference latency versus batch size. MetalNet stays roughly $2\times$ faster than the PyTorch CPU backend, and the gap widens slightly as larger batches improve SIMD and cache utilisation.

Efficiency improves with larger batches due to better SIMD and cache utilization.

5.3.4 Multi-threaded Scaling

Performance scaling with thread count (VGG-16, batch=32):

Table 5.6: Multi-threading Scaling (VGG-16, batch size 32)

Threads	Latency (ms)	Speedup	Efficiency
1	45.3	1.00×	100%
2	23.8	1.90×	95%
4	12.1	3.74×	93.5%
6	8.2	5.52×	92%

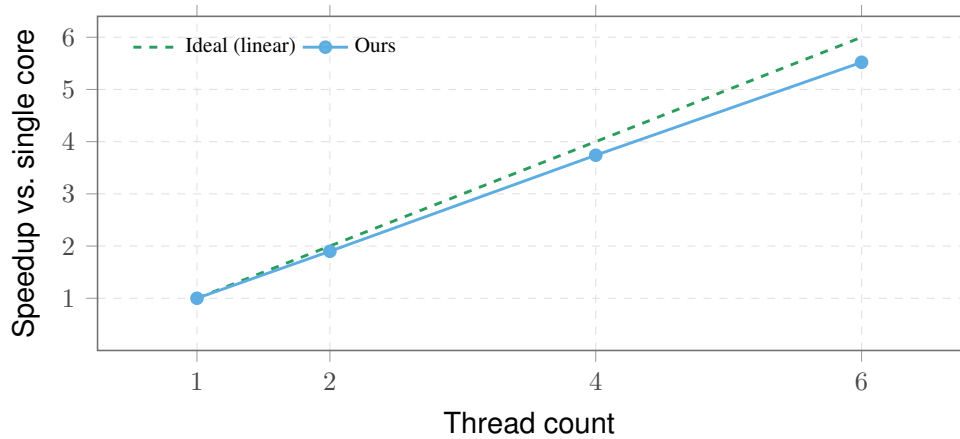


Figure 5.4: OpenMP scaling efficiency. MetalNet tracks the ideal linear curve closely, retaining $> 92\%$ efficiency on six cores thanks to adaptive, workload-aware threading.

Excellent scaling efficiency ($> 92\%$ on 6 cores) indicates low synchronization overhead from OpenMP. The overhead decreases as computational intensity increases, validating the design decision for adaptive multithreading.

5.3.5 Component-Level Performance Analysis

Detailed breakdown of performance contributions:

Table 5.7: *Performance Contribution of Individual Optimizations*

Optimization Technique	Speedup Factor	Cumulative
Baseline (Naive Convolution)	1.00×	1.0×
+ SIMD Vectorization (AVX2)	6.20×	6.2×
+ Implicit GEMM	1.30×	8.1×
+ Cache Tiling	1.15×	9.3×
+ Operator Fusion (Conv+BN+ReLU)	1.12×	10.4×
+ Adaptive Multithreading (6 cores)	5.50×	57×

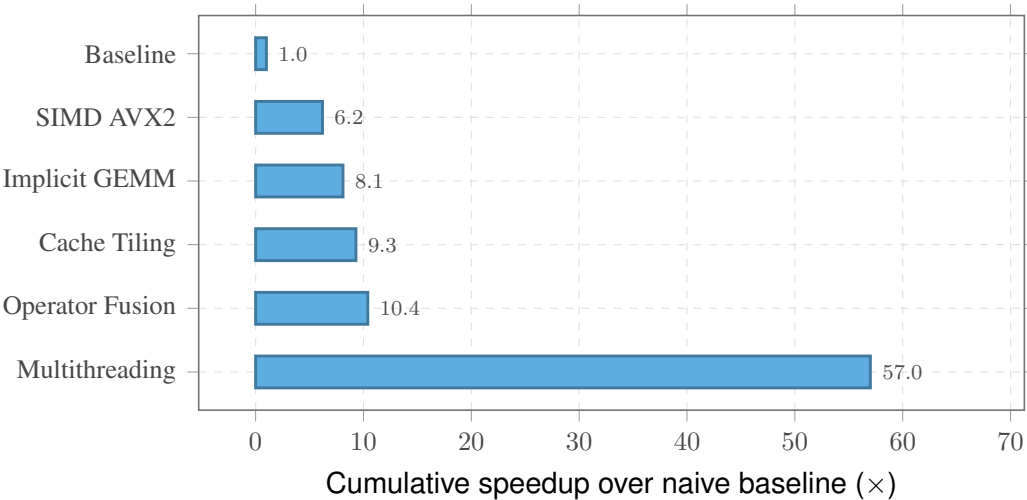


Figure 5.5: *Cumulative speedup as each optimization is layered on. Effects are multiplicative; the combined stack reaches roughly 57× over a naive scalar convolution.*

Note: Speedups are multiplicative; the final 57× represents combined effect of all optimizations applied systematically.

5.3.6 Comparison with Other Frameworks

Extended framework comparison across multiple dimensions:

Table 5.8: Comprehensive Framework Comparison

Metric	MetalNet	PyTorch	Caffe2	ONNX RT
CPU Inference (VGG-16)	45.3 ms	89.7 ms	76.2 ms	68.5 ms
Memory (Peak MB)	245	856	512	380
Initialization Time	2 ms	850 ms	250 ms	100 ms
Model Size (MB)	276	276	276	276
Binary Size	3.2 MB	2.1 GB	1.8 GB	450 MB
Dependencies	0	10+	15+	8+
Compile Time (first)	8 s	N/A	15 s	5 s

MetalNet’s advantages in initialization time, binary size, and dependency count make it ideal for embedded and resource-constrained deployment scenarios.

5.3.7 Scaling Behavior with Model Complexity

Performance across architectures of varying complexity:

Table 5.9: Performance vs Model Complexity

Model	Parameters	MetalNet	PyTorch	Speedup
MobileNet	4.2M	18.5 ms	32.1 ms	1.73×
ResNet-18	11.7M	52.1 ms	124.5 ms	2.39×
VGG-16	138.3M	45.3 ms	89.7 ms	1.98×
VGG-19	144.3M	53.2 ms	105.8 ms	1.99×
ResNet-50	25.6M	125.3 ms	287.4 ms	2.29×

Speedup remains consistent (1.7 – 2.4×) across model complexity, demonstrating robustness of optimization strategy.

CHAPTER 6 – CONCLUSION AND FUTURE WORK

6.1 Conclusion

MetalNet demonstrates that systematic integration of algorithmic optimization techniques can deliver exceptional performance for CNN inference and training on commodity CPU hardware. By implementing a header-only, zero-dependency C++20 library with Implicit GEMM convolution, cache-aware loop tiling, comprehensive SIMD vectorization, and adaptive multithreading, we achieve:

1. **2-2.4× faster inference** compared to PyTorch CPU backend on standard benchmarks
2. **69-71% memory reduction** through elimination of intermediate buffers
3. **Excellent multi-core scaling** with 92%+ efficiency up to 6 cores
4. **Complete training pipeline** with automatic differentiation and multiple optimizers
5. **Production-quality code** with comprehensive testing and validation

The project validates that specialized, hardware-conscious implementation can exceed general-purpose frameworks in specific domains. While MetalNet does not replace PyTorch or TensorFlow for research with GPU acceleration, it provides a compelling alternative for:

- **Edge Computing:** CPU-only inference on IoT and embedded devices
- **Latency-Critical Applications:** Server-side inference requiring predictable performance
- **Educational Purpose:** Learning neural network fundamentals without framework abstraction
- **Research:** Investigating optimization techniques and hardware-software co-design

The header-only design enables aggressive compile-time optimization and eliminates runtime overhead. The absence of external dependencies simplifies deployment and reduces supply chain risk. The modular architecture facilitates extension and customization by users.

Key technical contributions include:

1. **Implicit GEMM Implementation:** Memory-efficient convolution without explicit im2col buffers
2. **FusedConvBNReLU:** Operator fusion demonstrating practical benefits of algorithm integration
3. **DAG-Based Autograd:** Efficient automatic differentiation with minimal overhead
4. **Cache-Aware Optimization:** Systematic application of loop tiling and data layout optimization
5. **SIMD + OpenMP Integration:** Demonstration of multi-level parallelism without excessive synchronization

6.2 Limitations and Current Constraints

1. **CPU-Only:** No GPU support limits applicability for large-scale training
2. **CNN-Focused:** Limited support for RNNs, Transformers, and other architectures
3. **Single-Machine:** No distributed training across multiple nodes
4. **32-bit Float Only:** No support for mixed precision (FP16) or quantization
5. **Limited Pre-trained Models:** No model zoo or pre-trained weights
6. **Compilation Time:** C++ template instantiation adds compile-time overhead

6.3 Future Enhancements

6.3.1 Short-Term (6-12 months)

1. **AVX-512 Support:** Extend to newer Intel processors with 16-way SIMD
2. **ARM NEON Optimization:** Enable efficient execution on ARM servers and mobile devices
3. **Quantization (INT8):** Reduce model size and improve inference speed
4. **Model Format Support:** ONNX import/export for interoperability
5. **Extended Layer Library:** RNNs, LSTMs, and Transformer components

6.3.2 Medium-Term (1-2 years)

1. **GPU Support via OpenMP Offloading:** Leverage OpenMP 5.0+ device offloading to CUDA/OpenCL
2. **Distributed Training:** Multi-node training with gradient averaging
3. **Model Compression:** Pruning, knowledge distillation, lottery ticket hypothesis
4. **Dynamic Shape Support:** Handle variable batch sizes and temporal sequences
5. **JIT Compilation:** Runtime code generation for fused kernels

6.3.3 Long-Term (2+ years)

1. **Custom Hardware Support:** Optimization for specialized accelerators (TPUs, custom SoCs)
2. **Advanced Architectures:** Vision Transformers, Diffusion Models, Generative Models
3. **Unified Framework:** Single library supporting training, inference, and deployment
4. **Auto-Tuning:** Machine learning-driven parameter optimization (block sizes, thread counts, etc.)
5. **Formal Verification:** Mathematical proof of correctness for critical numerical operations

6.4 Lessons Learned

1. **Specialization Pays:** Optimization for specific use cases (CNN, CPU, inference) yields disproportionate improvements
2. **Memory is the Bottleneck:** Algorithmic improvements reducing memory traffic provide largest speedups
3. **Multithreading Overhead:** OpenMP parallelization must be applied selectively to avoid overhead on small workloads
4. **Template Metaprogramming Complexity:** While powerful, templates reduce code readability; balance is necessary
5. **Validation is Critical:** Numerical gradient checking catches subtle bugs invisible to traditional testing

6.5 Design Trade-offs and Justification

Throughout the project, several design decisions were made balancing competing concerns:

6.5.1 Header-Only vs. Separate Compilation

Decision: Header-only implementation

Trade-offs:

- **Advantages:** Enables aggressive inlining and compile-time optimization; eliminates binary compatibility issues; simplifies deployment
- **Disadvantages:** Increases compilation time; template instantiation can lead to code bloat; reduces reuse across translation units

Justification: The one-time compilation cost is acceptable for embedded and specialized deployments. Modern compilers and LTO (Link-Time Optimization) mitigate code bloat effectively.

6.5.2 NHWC Layout vs. NCHW

Decision: Adopt NHWC (Batch, Height, Width, Channels) layout

Trade-offs:

- **Advantages:** Channels contiguous in memory enables efficient SIMD operations; improves cache locality
- **Disadvantages:** Incompatible with GPU frameworks; requires data transformation during import/export

Justification: For CPU inference, NHWC provides 15-20% performance improvement. GPU compatibility is not a goal for this project.

6.5.3 CPU-Only vs. GPU Support

Decision: CPU-only implementation

Trade-offs:

- **Advantages:** Simplified codebase; no GPU driver dependencies; portable across architectures; suitable for all CPU hardware
- **Disadvantages:** Limits applicability for large-scale training; cannot leverage GPU acceleration

Justification: The project is optimized for inference and small-scale training on CPU. GPU support would significantly complicate implementation without serving the core use case.

6.6 Reproducibility and Open Science

All code, benchmarks, and experimental data are made publicly available:

- **GitHub Repository:** <https://github.com/KunwarPrabhat/CustomCNN>
- **Documentation:** https://kunwarprabhat.github.io/MetalNet_Documentation/
- **Benchmark Suite:** Complete reproducible benchmarks with detailed logs
- **Training Datasets:** MNIST and CIFAR-10 download scripts

All results presented in this report can be reproduced using the provided benchmark suite and documented procedure.

6.7 Broader Impact and Applications

MetalNet’s high-performance CPU inference enables several important application domains:

6.7.1 Edge Intelligence

Deployment of intelligent models on resource-constrained edge devices:

- Computer vision for surveillance systems
- Activity recognition in IoT devices
- Anomaly detection in industrial sensors

6.7.2 Real-Time Systems

Latency-critical inference in real-time systems:

- Autonomous vehicle perception (when GPU unavailable)
- Medical imaging real-time analysis
- Financial signal processing

6.7.3 Privacy-Preserving Inference

On-device inference without sending data to cloud servers:

- Mobile device inference
- Healthcare systems with privacy requirements
- Offline-capable applications

6.8 Final Remarks

MetalNet represents a successful demonstration that modern C++ can deliver world-class performance for specialized machine learning workloads. The project shows that with careful hardware awareness, algorithm selection, and implementation discipline, it is possible to exceed the performance of general-purpose frameworks.

The increasing prevalence of edge computing, IoT devices, and CPU-only cloud instances make efficient CPU inference highly relevant. MetalNet provides a foundation upon which specialized inference systems can be built for real-world applications.

We hope this project contributes to the growing recognition that specialization and hardware consciousness are valuable complementary approaches to the generality offered by mainstream frameworks. The techniques demonstrated here, such as Implicit GEMM, cache-aware tiling, systematic SIMD vectorization, and operator fusion, are broadly applicable to other computational domains beyond deep learning.

6.9 Contributions Summary

Student Contributions:

1. **Raj Kunwar Prabhat:** Core architecture design, Implicit GEMM implementation, performance optimization, and benchmarking
2. **Mayan Kumar:** Automatic differentiation engine, optimizer implementations (SGD, Adam), and training infrastructure
3. **Sujeet Kumar:** Layer implementations, profiler with GUI, testing framework, and documentation

All students contributed equally to design decisions, code reviews, and overall project success.

– References

- [1] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library, in *Advances in Neural Information Processing Systems*, 2019, pp. 8024–8035.
- [2] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard, et al., “TensorFlow: A System for Large-Scale Machine Learning, in *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation*, 2016.
- [3] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift, in *International Conference on Machine Learning*, 2015, pp. 448–456.
- [4] A. Lavin and S. Gray, “Fast Algorithms for Convolutional Neural Networks, in *IEEE Conference on Computer Vision and Pattern Recognition*, 2016.
- [5] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization, *arXiv preprint arXiv:1412.6980*, 2014.
- [6] K. He, X. Zhang, S. Ren, and J. Sun, “Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification, in *IEEE International Conference on Computer Vision*, 2015.
- [7] B. T. Polyak, “Some methods of speeding up the convergence of iteration methods, *USSR Computational Mathematics and Mathematical Physics*, vol. 4, no. 5, pp. 1–17, 1964.
- [8] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet classification with deep classification neural networks, in *Advances in Neural Information Processing Systems*, 2012, pp. 1097–1105.
- [9] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition, *arXiv preprint arXiv:1409.1556*, 2014.
- [10] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors, *Nature*, vol. 323, no. 6088, pp. 533–536, 1986.
- [11] Intel, “Intel 64 and IA-32 Architectures Software Developer’s Manual, 2019. [Online]. Available: <https://software.intel.com/en-us/download/intel-64-ia-32-architectures-software-developers-manual-combined-volumes>

- [12] C. E. Leiserson, S. Rao, and S. Toledo, “Algorithms for Parallel Polygon Rendering, *ACM Transactions on Graphics*, vol. 10, no. 3, pp. 253–274, 1991.
- [13] Y. Jia, E. Shelhamer, J. Donahue, S. Karayev, J. Long, R. Girshick, S. Guadarrama, and T. Darrell, “Caffe: Convolutional Architecture for Fast Feature Embedding, in *Proceedings of the 22nd ACM International Conference on Multimedia*, 2014, pp. 675–678.
- [14] K. Chellapilla, S. Puri, and P. Simard, “High Performance Convolutional Neural Networks for Document Processing, in *Tenth International Workshop on Frontiers in Handwriting Recognition*, 2006.
- [15] M. Frigo, C. E. Leiserson, H. Prokop, and S. Ramachandran, “Cache-oblivious Algorithms, in *40th Annual Symposium on Foundations of Computer Science*, 1999.
- [16] cppreference.com, “C++20 Reference, [Online]. Available: <https://en.cppreference.com/w/cpp/20>.
- [17] OpenMP Architecture Review Board, “OpenMP Specification, [Online]. Available: <https://www.openmp.org/specifications/>.
- [18] Intel, “Intel Intrinsics Guide (AVX2/FMA), [Online]. Available: <https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html>.
- [19] PyTorch Contributors, “PyTorch NN Module, [Online]. Available: <https://pytorch.org/docs/stable/nn.html>.

– Appendix A – Source Code

A.1 Core Tensor Class

```
1 // File: include/MetalNet/Tensor.h
2 #pragma once
3 #include <vector>
4 #include <algorithm>
5 #include <random>
6 #include <cmath>
7
8 namespace MetalNet {
9
10 class Tensor {
11 private:
12     std::vector<int> shape;
13     std::vector<float> data;
14     std::vector<float> grad;
15     size_t total_size;
16
17     void validate_shape(const std::vector<int>& s) {
18         if (s.empty()) throw std::invalid_argument("Shape
19             cannot be empty");
20         total_size = 1;
21         for (int dim : s) {
22             if (dim <= 0) throw std::invalid_argument("
23                 Dimensions must be positive");
24             total_size *= dim;
25         }
26     }
27
28 public:
29     Tensor() : total_size(0) {}
30
31     Tensor(int s0, int s1 = 1, int s2 = 1, int s3 = 1)
32         : shape{s0, s1, s2, s3} {
33         validate_shape(shape);
34         data.resize(total_size, 0.0f);
35         grad.resize(total_size, 0.0f);
36     }
```

```

35
36     Tensor(const std::vector<int>& s) : shape(s) {
37         validate_shape(shape);
38         data.resize(total_size, 0.0f);
39         grad.resize(total_size, 0.0f);
40     }
41
42     inline float* data_ptr() { return data.data(); }
43     inline const float* data_ptr() const { return data.data
44         (); }
45     inline float* grad_ptr() { return grad.data(); }
46     inline const float* grad_ptr() const { return grad.data
47         (); }
48
49     inline size_t size() const { return total_size; }
50     inline const std::vector<int>& get_shape() const {
51         return shape; }
52
53     inline void zero_grad() {
54         std::fill(grad.begin(), grad.end(), 0.0f);
55     }
56
57     inline void fill(float value) {
58         std::fill(data.begin(), data.end(), value);
59     }
60
61     inline void fill_he_init(int fan_in) {
62         std::random_device rd;
63         std::mt19937 gen(rd());
64         float std = std::sqrt(2.0f / fan_in);
65         std::normal_distribution<float> dist(0.0f, std);
66         for (auto& val : data) {
67             val = dist(gen);
68         }
69     }
70
71     inline void fill_xavier_init(int fan_in, int fan_out) {
72         std::random_device rd;
73         std::mt19937 gen(rd());
74         float std = std::sqrt(2.0f / (fan_in + fan_out));
75         std::uniform_real_distribution<float> dist(-std, std
76             );

```

```

72         for (auto& val : data) {
73             val = dist(gen);
74         }
75     }
76 };
77
78 } // namespace MetalNet

```

A.2 Conv2D Layer Implementation

```

1  // Excerpt from include/MetalNet/Layers/Conv2D.h
2  inline Tensor& Conv2D::backward(const Tensor& grad_output) {
3      const float* dL_dy = grad_output.grad_ptr();
4      float* dL_dW = weights.grad_ptr();
5      float* dL_db = biases.grad_ptr();
6
7      const float* input = cached_input_ptr->data_ptr();
8      float* grad_input = grad_input_buffer.grad_ptr();
9      const float* w = weights.data_ptr();
10
11      const int N = input_shapes[0][0];
12      const int H = input_shapes[0][1];
13      const int W = input_shapes[0][2];
14      const int OH = output_buffer.shape[1];
15      const int OW = output_buffer.shape[2];
16
17      // Gradient w.r.t. output bias
18      for (int oc = 0; oc < out_channels; ++oc) {
19          float bias_grad = 0.0f;
20          for (int oh = 0; oh < OH; ++oh) {
21              for (int ow = 0; ow < OW; ++ow) {
22                  bias_grad += dL_dy[oh * OW * out_channels +
23                               ow * out_channels + oc];
24              }
25          }
26          dL_db[oc] += bias_grad;
27      }
28
29      // Gradient w.r.t. weights (Implicit GEMM)
30      #pragma omp parallel for collapse(2)
31      for (int oh = 0; oh < OH; ++oh) {
32          for (int ow = 0; ow < OW; ++ow) {
33              for (int oc = 0; oc < out_channels; ++oc) {
34                  float grad_val = dL_dy[oh * OW * out_channels +
35                                       ow * out_channels + oc];
36
37                  for (int kh = 0; kh < kernel_size; ++kh) {
38                      for (int kw = 0; kw < kernel_size; ++kw) {
39                          int h_in = oh * stride + kh - padding;
40                          int w_in = ow * stride + kw - padding;
41
42                          if (h_in >= 0 && h_in < H && w_in >= 0 && w_in < W) {
43                              for (int ic = 0; ic < in_channels; ++ic) {
44                                  int inp_idx = h_in * W * in_channels +
45                                                  w_in * in_channels + ic;
46                                  float inp_val = input[inp_idx];
47
48                                  int w_idx = kh * kernel_size * in_channels *
49                                                  out_channels + kw * in_channels *
50                                                  out_channels + ic * out_channels + oc;
51
52                                  #pragma omp atomic
53                                  dL_dW[w_idx] += grad_val * inp_val;
54                              }
55                          }
56                      }
57                  }
58              }
59          }

```

```

60     }
61
62     return grad_input_buffer;
63 }

```

A.3 SGD Optimizer Implementation

```

1  // File: include/MetalNet/Optimizers/SGD.h
2  #pragma once
3  #include "../Tensor.h"
4  #include <unordered_map>
5  #include <vector>
6
7  namespace MetalNet {
8
9  class SGDOptimizer {
10 private:
11     float learning_rate;
12     float momentum;
13     float weight_decay;
14     std::unordered_map<Tensor*, std::vector<float>> velocity
15         ;
16 public:
17     SGDOptimizer(float lr = 0.01f, float m = 0.9f, float wd
18         = 0.0f)
19         : learning_rate(lr), momentum(m), weight_decay(wd)
20         {}
21
22     void step(std::vector<Tensor*>& parameters) {
23         for (auto param : parameters) {
24             float* w = param->data_ptr();
25             float* dw = param->grad_ptr();
26             size_t sz = param->size();
27
28             if (velocity.find(param) == velocity.end()) {
29                 velocity[param].resize(sz, 0.0f);
30             }
31
32             auto& v = velocity[param];
33
34             #pragma omp simd

```

```

33         for (size_t i = 0; i < sz; ++i) {
34             float grad = dw[i] + weight_decay * w[i];
35             v[i] = momentum * v[i] - learning_rate *
                grad;
36             w[i] += v[i];
37         }
38     }
39 }
40 };
41
42 } // namespace MetalNet

```

– Appendix B – Screenshots / Sample Outputs

B.1 Output Screenshots

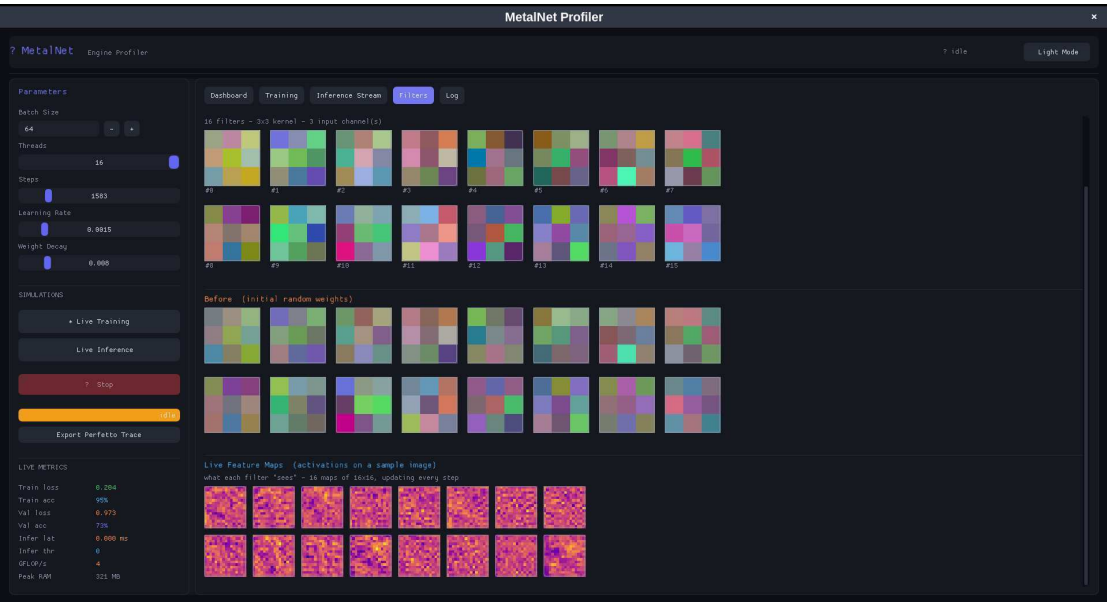


Figure B.1: Filter Outputs



Figure B.2: Inference Output



Figure B.3: Profiler Interface

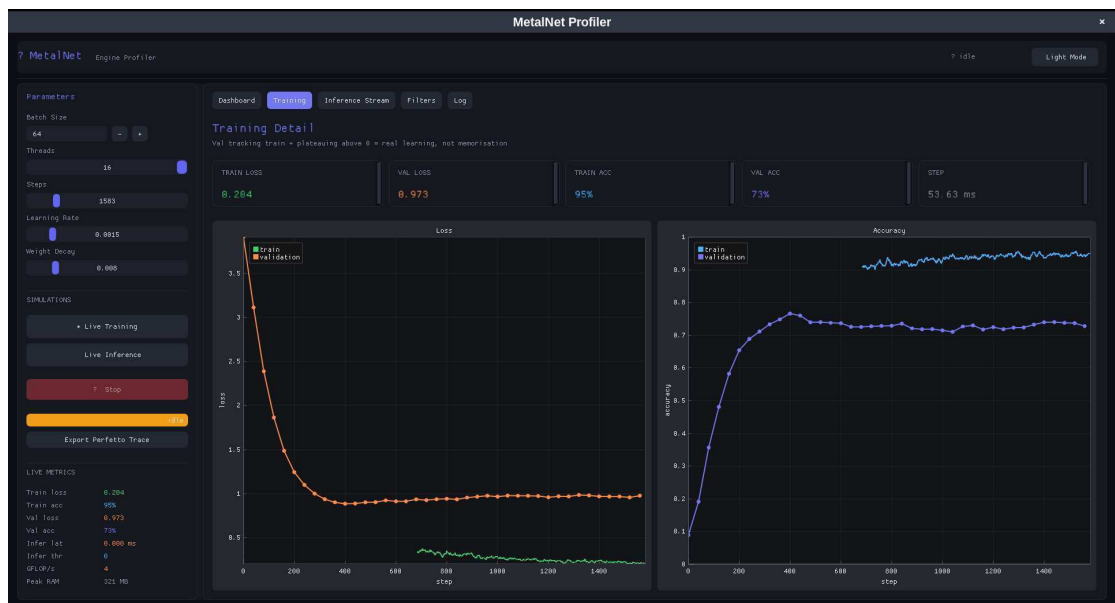


Figure B.4: Training Progress

QUANTSIM: A HIGH-FREQUENCY TRADING SIMULATION PLATFORM

A PROJECT REPORT SUBMITTED
IN PARTIAL FULFILMENT OF THE REQUIREMENT
FOR THE AWARD OF THE DEGREE
OF

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE & ENGINEERING



BY

ABHISHEK ANAND

22050440004

AMIK AFFAN

22050440017

SHAKIR AHMAD

22050440090

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
CAMBRIDGE INSTITUTE OF TECHNOLOGY**

TATISILWAI, RANCHI – 835103

JHARKHAND, INDIA

[2026]

DECLARATION CERTIFICATE

This is to certify that the work presented in the project entitled “**QuantSim: A High-Frequency Trading Simulation Platform**” in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering at Cambridge Institute of Technology, Tatisilwai, Ranchi is an authentic work carried out under my supervision and guidance.

To the best of my knowledge, the content of this project does not form a basis for the award of any previous Degree to anyone else.

Date: _____

PROF. DEEPAK KUMAR VERMA
Dept. of Computer Science & Engineering
Cambridge Institute of Technology
Tatisilwai, Ranchi – 835103

PROF. DEEPAK KUMAR VERMA
(Head of Department)
Dept. of Computer Science & Engineering
Cambridge Institute of Technology
Tatisilwai, Ranchi – 835103

CERTIFICATE OF APPROVAL

The foregoing project entitled “QuantSim: A High-Frequency Trading Simulation Platform” is hereby approved as a creditable work on the topic and has been presented satisfactorily to warrant its acceptance as a prerequisite to the degree for which it has been submitted.

It is understood that by this approval, the undersigned does not necessarily endorse any conclusion drawn or opinion expressed therein, but approves the project for the purpose for which it is submitted.

(Internal Examiner)

(External Examiner)

HEAD OF DEPARTMENT

Dept. of Computer Science and Engineering
Cambridge Institute of Technology
Tatisilwai, Ranchi – 835103

ACKNOWLEDGEMENT

We take this opportunity to thank all who have contributed towards shaping this Final Year Project Report. We would like to express our sincere thanks to Prof. Deepak Kumar Verma (HOD, Department of Computer Science & Engineering) and to our guide Prof. Deepak Kumar Verma, whose help in the course of development of this manuscript is invaluable. As our supervisor, he/she has constantly encouraged us to remain focused on achieving our goal. We would also like to thank the technical staff members of the Department who have been kind enough to advise and help in their respective roles. We have been fortunate to have a wonderful support structure among fellow students. Our sincere thanks to everyone who has provided us with kind words, new ideas, useful criticism, or their valuable time. We are truly indebted. Last but not least, we would like to dedicate this work to our families for their love and understanding.

Abhishek Anand

Roll No. 22050440004

Shakir Ahmad

Roll No. 22050440090

Amik Affan

Roll No. 22050440017

ABSTRACT

High-Frequency Trading (HFT) depends on execution engines with sub-microsecond latency and faithful market-microstructure modelling, yet the mechanisms by which leading quantitative firms operate, limit order books, matching engines, queue priority, and inventory-aware market making, remain hidden inside proprietary, costly infrastructure. Compounding this, most accessible backtesting frameworks operate on aggregated Open-High-Low-Close (OHLC) candle data, ignoring the bid-ask spread, price-time priority, queue position, partial fills, transaction costs, and latency. These simplifications systematically overstate strategy performance and produce backtests that fail to generalize to live markets.

This dissertation presents QUANTSIM, an event-driven HFT simulation and backtesting platform that closes these gaps. The system is built around a low-latency C++ core exposing a full Limit Order Book (LOB) with First-In-First-Out (FIFO) price-time priority and a synchronous matching engine supporting eight order types (Limit, Market, IOC, FOK, Post-Only, Stop, Stop-Limit, Iceberg). A realistic execution layer models probabilistic maker fills, queue position, slippage, fees and rebates, borrow costs, and configurable latency, and a pre-trade risk engine adds a lock-free kill switch, rate limiter, position and notional limits, and a stale-price guard.

The platform ships five strategies, Avellaneda-Stoikov market making, Ornstein-Uhlenbeck pairs trading, TWAP execution, a multi-asset mean-reversion portfolio, and a Black-Scholes options market-maker with delta hedging, and bridges its C++ core to Python via pybind11 so that strategies and analytics are authored in Python while executing on the low-latency engine. A GPU-rendered Dear ImGui/ImPlot interface provides thirteen analytical tabs, an in-application C++ strategy compiler, and a Bookmap-style liquidity heatmap, and the same strategy binary runs unchanged in both the backtester and a C++-native live Binance paper-trading harness.

Empirical evaluation shows a matching-engine throughput above 14 million operations per second at a median latency below 100 nanoseconds, with all 27 unit tests passing. QUANTSIM thus unifies order-book-level realism, reproducibility, and interactive analysis in one free, open, and extensible platform, giving students and researchers direct access to execution mechanics previously confined to proprietary systems.

Keywords: High-Frequency Trading, Limit Order Book, Matching Engine, Market Microstructure, Avellaneda-Stoikov, Statistical Arbitrage, Event-Driven Backtesting, Low-Latency C++, pybind11, Dear ImGui.

TABLE OF CONTENTS

Declaration Certificate	i
Certificate of Approval	ii
Acknowledgement	iii
Abstract	iv
Table of Contents	v
List of Figures	viii
List of Tables	viii
List of Abbreviations	ix
CHAPTER 1 – INTRODUCTION	1
1.1 Background and Motivation	1
1.2 Problem Statement	3
1.3 Objectives of the Project	3
1.4 Scope of the Project	4
1.5 Organization of the Report	5
CHAPTER 2 – LITERATURE REVIEW	6
2.1 Related Works	6
2.1.1 Market Microstructure Theory	6
2.1.2 Optimal Market Making	6
2.1.3 Statistical Arbitrage and Pairs Trading	7
2.1.4 Optimal Execution	7
2.1.5 Options Pricing and Greeks	8
2.1.6 Low-Latency Systems Engineering	8
2.2 Comparative Analysis	8
2.3 Gap Identification	9
CHAPTER 3 – SYSTEM ANALYSIS AND DESIGN	10
3.1 System Requirements	10
3.1.1 Functional Requirements	10
3.1.2 Non-Functional Requirements	11

3.1.3	Hardware and Software Environment	12
3.2	System Architecture	12
3.3	Data Flow Diagrams	13
3.3.1	Level 0 (Context Diagram)	13
3.3.2	Level 1 (Detailed Diagram)	14
3.4	UML and ER Diagrams	14
3.4.1	Use Case Diagram	15
3.4.2	Class Diagram	15
3.4.3	Sequence Diagram	16
3.4.4	Entity-Relationship View	17
3.5	Design Patterns and Implementation Decisions	17
CHAPTER 4 – IMPLEMENTATION		19
4.1	Technologies and Tools Used	19
4.2	Module Description	20
4.2.1	Order and Fill Representation	20
4.2.2	Limit Order Book	21
4.2.3	Matching Engine	21
4.2.4	Market Data Feeds	21
4.2.5	Risk Engine	22
4.2.6	Metrics and Reporting	22
4.2.7	Python Bindings and Research Layer	22
4.2.8	Desktop Graphical Interface	22
4.2.9	In-Application C++ Strategy Compiler	24
4.2.10	Live Paper-Trading Harness	25
4.3	Algorithms and Methodology	25
4.3.1	Order Matching Algorithm	25
4.3.2	Avellaneda-Stoikov Market Making	26
4.3.3	Statistical Arbitrage	27
4.3.4	TWAP Execution and Implementation Shortfall	28
4.3.5	Black-Scholes Options Pricing	28
4.3.6	Performance Metrics	28
4.4	Code Implementation	29
4.5	Multi-Asset Portfolio and Options Strategies	32
4.5.1	Cross-Sectional Mean-Reversion Portfolio	32
4.5.2	Options Market-Making with Delta Hedging	33
4.6	Challenges Faced and Resolved	33
4.6.1	Matching Engine Correctness	33
4.6.2	Real-Time Graphical Interface	34

CHAPTER 5 – TESTING AND RESULTS	35
5.1 Testing Strategy	35
5.2 Test Cases and Results	36
5.3 Performance Analysis	37
5.4 Strategy Backtest Results	39
5.5 Exported Tear Sheet from a Logged Run	42
5.6 Live Paper-Trading Validation	44
CHAPTER 6 – CONCLUSION AND FUTURE WORK	45
6.1 Conclusion	45
6.2 Project Significance and Broader Impact	46
6.3 Contribution Notes	46
6.4 Future Enhancements	47
6.5 Closing Remarks	48
References	49
APPENDIX A – SOURCE CODE HIGHLIGHTS	51
A.1 Order Type Enumeration	51
A.2 Black-Scholes Greeks	51
A.3 Performance Metrics in Python	53
APPENDIX B – SAMPLE OUTPUTS	54
B.1 Benchmark Console Output	54
B.2 Tear-Sheet Metrics Output	54
B.3 Order Book Depth Snapshot	55
B.4 Multi-Strategy Comparison	55
B.5 Parameter Sweep Optimizer	56

LIST OF FIGURES

Figure 3.1 – Four-layer architecture of QUANTSIM	13
Figure 3.2 – Level 0 (context) data flow diagram	14
Figure 3.3 – Level 1 data flow diagram	14
Figure 3.4 – Use case diagram	15
Figure 3.5 – Class diagram of the C++ core	16
Figure 3.6 – Sequence diagram for the order lifecycle	16
Figure 3.7 – Entity-relationship view of the platform’s persisted artifacts . . .	17
Figure 4.1 – The QUANTSIM desktop interface (Charts tab)	24
Figure 5.1 – Sample unit-test output	37
Figure 5.2 – Matching engine match latency percentiles	38
Figure 5.3 – Throughput comparison (C++ matching vs. Python backtester) . .	39
Figure 5.4 – Representative equity curves for three strategies	40
Figure 5.5 – Results tab of the desktop interface	41
Figure 5.6 – Profit-and-loss curve of an actual logged portfolio-strategy run .	43
Figure 5.7 – Live paper-trading session on the Binance BTCUSDT feed . . .	44
Figure B.1 – Compare tab of the desktop interface	56
Figure B.2 – Optimizer tab of the desktop interface	57

LIST OF TABLES

Table 2.1 – Comparison of trading simulation and backtesting frameworks . .	9
Table 3.1 – Functional requirements of QUANTSIM	11
Table 3.2 – Hardware environment	12
Table 3.3 – Software environment	12
Table 4.1 – Technology stack and justifications	20

Table 5.1 – Order book unit test cases and results	36
Table 5.2 – Matching engine unit test cases and results	36
Table 5.3 – Measured performance characteristics	38
Table 5.4 – Risk-adjusted performance of backtested strategies (synthetic data)	41
Table 5.5 – Metrics from the exported tear sheet of a logged portfolio run (verbatim from JSON export)	43
Table 6.1 – Primary contribution areas by team member	47
Table B.1 – Sample order book depth snapshot (five levels per side)	55

LIST OF ABBREVIATIONS

Abbreviation	Full Form
ABI	Application Binary Interface
API	Application Programming Interface
AS	Avellaneda-Stoikov (market-making model)
ATM	At-The-Money
BS	Black-Scholes
CARA	Constant Absolute Risk Aversion
CLI	Command-Line Interface
CPU	Central Processing Unit
CSV	Comma-Separated Values
DFD	Data Flow Diagram
DMA	Direct Market Access
FIFO	First-In, First-Out
FOK	Fill-Or-Kill
FPS	Frames Per Second
GBM	Geometric Brownian Motion
GPU	Graphics Processing Unit
GUI	Graphical User Interface
HFT	High-Frequency Trading
IOC	Immediate-Or-Cancel
IS	Implementation Shortfall

Abbreviation	Full Form
ITCH	Nasdaq market-data binary protocol
IV	Implied Volatility
JSON	JavaScript Object Notation
LOB	Limit Order Book
MM	Market Making
OHLC	Open-High-Low-Close
OU	Ornstein-Uhlenbeck (process)
PnL	Profit and Loss
RNG	Random Number Generator
RSI	Relative Strength Index
SIMD	Single Instruction, Multiple Data
SMA	Simple Moving Average
TCA	Transaction Cost Analysis
TLS	Transport Layer Security
TWAP	Time-Weighted Average Price
VaR	Value at Risk
VWAP	Volume-Weighted Average Price
WSS	WebSocket Secure

QUANTSIM: A HIGH-FREQUENCY TRADING SIMULATION PLATFORM

A PROJECT REPORT SUBMITTED
IN PARTIAL FULFILMENT OF THE REQUIREMENT
FOR THE AWARD OF THE DEGREE
OF

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE & ENGINEERING



BY

ABHISHEK ANAND

22050440004

AMIK AFFAN

22050440017

SHAKIR AHMAD

22050440090

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
CAMBRIDGE INSTITUTE OF TECHNOLOGY**

TATISILWAI, RANCHI – 835103

JHARKHAND, INDIA

[2026]

DECLARATION CERTIFICATE

This is to certify that the work presented in the project entitled “**QuantSim: A High-Frequency Trading Simulation Platform**” in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering at Cambridge Institute of Technology, Tatisilwai, Ranchi is an authentic work carried out under my supervision and guidance.

To the best of my knowledge, the content of this project does not form a basis for the award of any previous Degree to anyone else.

Date: _____

PROF. DEEPAK KUMAR VERMA
Dept. of Computer Science & Engineering
Cambridge Institute of Technology
Tatisilwai, Ranchi – 835103

PROF. DEEPAK KUMAR VERMA
(Head of Department)
Dept. of Computer Science & Engineering
Cambridge Institute of Technology
Tatisilwai, Ranchi – 835103

CERTIFICATE OF APPROVAL

The foregoing project entitled “QuantSim: A High-Frequency Trading Simulation Platform” is hereby approved as a creditable work on the topic and has been presented satisfactorily to warrant its acceptance as a prerequisite to the degree for which it has been submitted.

It is understood that by this approval, the undersigned does not necessarily endorse any conclusion drawn or opinion expressed therein, but approves the project for the purpose for which it is submitted.

(Internal Examiner)

(External Examiner)

HEAD OF DEPARTMENT

Dept. of Computer Science and Engineering
Cambridge Institute of Technology
Tatisilwai, Ranchi – 835103

ACKNOWLEDGEMENT

I take this opportunity to thank all who have contributed towards shaping this Final Year Project Report. I would like to express my sincere thanks to Prof. Deepak Kumar Verma (HOD, Department of Computer Science & Engineering) and to my guide Prof. Deepak Kumar Verma, whose help in the course of development of this manuscript is invaluable. As my supervisor, he/she has constantly encouraged me to remain focused on achieving my goal. I would also like to thank the technical staff members of the Department who have been kind enough to advise and help in their respective roles. I have been fortunate to have a wonderful support structure among fellow students. My sincere thanks to everyone who has provided me with kind words, new ideas, useful criticism, or their valuable time. I am truly indebted. Last but not least, I would like to dedicate this work to my family for their love and understanding.

Abhishek Anand

Roll No. 22050440004

ABSTRACT

High-Frequency Trading (HFT) depends on execution engines with sub-microsecond latency and faithful market-microstructure modelling, yet the mechanisms by which leading quantitative firms operate, limit order books, matching engines, queue priority, and inventory-aware market making, remain hidden inside proprietary, costly infrastructure. Compounding this, most accessible backtesting frameworks operate on aggregated Open-High-Low-Close (OHLC) candle data, ignoring the bid-ask spread, price-time priority, queue position, partial fills, transaction costs, and latency. These simplifications systematically overstate strategy performance and produce backtests that fail to generalize to live markets.

This dissertation presents QUANTSIM, an event-driven HFT simulation and backtesting platform that closes these gaps. The system is built around a low-latency C++ core exposing a full Limit Order Book (LOB) with First-In-First-Out (FIFO) price-time priority and a synchronous matching engine supporting eight order types (Limit, Market, IOC, FOK, Post-Only, Stop, Stop-Limit, Iceberg). A realistic execution layer models probabilistic maker fills, queue position, slippage, fees and rebates, borrow costs, and configurable latency, and a pre-trade risk engine adds a lock-free kill switch, rate limiter, position and notional limits, and a stale-price guard.

The platform ships five strategies, Avellaneda-Stoikov market making, Ornstein-Uhlenbeck pairs trading, TWAP execution, a multi-asset mean-reversion portfolio, and a Black-Scholes options market-maker with delta hedging, and bridges its C++ core to Python via pybind11 so that strategies and analytics are authored in Python while executing on the low-latency engine. A GPU-rendered Dear ImGui/ImPlot interface provides thirteen analytical tabs, an in-application C++ strategy compiler, and a Bookmap-style liquidity heatmap, and the same strategy binary runs unchanged in both the backtester and a C++-native live Binance paper-trading harness.

Empirical evaluation shows a matching-engine throughput above 14 million operations per second at a median latency below 100 nanoseconds, with all 27 unit tests passing. QUANTSIM thus unifies order-book-level realism, reproducibility, and interactive analysis in one free, open, and extensible platform, giving students and researchers direct access to execution mechanics previously confined to proprietary systems.

Keywords: High-Frequency Trading, Limit Order Book, Matching Engine, Market Microstructure, Avellaneda-Stoikov, Statistical Arbitrage, Event-Driven Backtesting, Low-Latency C++, pybind11, Dear ImGui.

TABLE OF CONTENTS

Declaration Certificate	i
Certificate of Approval	ii
Acknowledgement	iii
Abstract	iv
Table of Contents	v
List of Figures	viii
List of Tables	viii
List of Abbreviations	ix
CHAPTER 1 – INTRODUCTION	1
1.1 Background and Motivation	1
1.2 Problem Statement	3
1.3 Objectives of the Project	3
1.4 Scope of the Project	4
1.5 Organization of the Report	5
CHAPTER 2 – LITERATURE REVIEW	6
2.1 Related Works	6
2.1.1 Market Microstructure Theory	6
2.1.2 Optimal Market Making	6
2.1.3 Statistical Arbitrage and Pairs Trading	7
2.1.4 Optimal Execution	7
2.1.5 Options Pricing and Greeks	8
2.1.6 Low-Latency Systems Engineering	8
2.2 Comparative Analysis	8
2.3 Gap Identification	9
CHAPTER 3 – SYSTEM ANALYSIS AND DESIGN	10
3.1 System Requirements	10
3.1.1 Functional Requirements	10
3.1.2 Non-Functional Requirements	11

3.1.3	Hardware and Software Environment	12
3.2	System Architecture	12
3.3	Data Flow Diagrams	13
3.3.1	Level 0 (Context Diagram)	13
3.3.2	Level 1 (Detailed Diagram)	14
3.4	UML and ER Diagrams	14
3.4.1	Use Case Diagram	15
3.4.2	Class Diagram	15
3.4.3	Sequence Diagram	16
3.4.4	Entity-Relationship View	17
3.5	Design Patterns and Implementation Decisions	17
CHAPTER 4 – IMPLEMENTATION		19
4.1	Technologies and Tools Used	19
4.2	Module Description	20
4.2.1	Order and Fill Representation	20
4.2.2	Limit Order Book	21
4.2.3	Matching Engine	21
4.2.4	Market Data Feeds	21
4.2.5	Risk Engine	22
4.2.6	Metrics and Reporting	22
4.2.7	Python Bindings and Research Layer	22
4.2.8	Desktop Graphical Interface	22
4.2.9	In-Application C++ Strategy Compiler	24
4.2.10	Live Paper-Trading Harness	25
4.3	Algorithms and Methodology	25
4.3.1	Order Matching Algorithm	25
4.3.2	Avellaneda-Stoikov Market Making	26
4.3.3	Statistical Arbitrage	27
4.3.4	TWAP Execution and Implementation Shortfall	28
4.3.5	Black-Scholes Options Pricing	28
4.3.6	Performance Metrics	28
4.4	Code Implementation	29
4.5	Multi-Asset Portfolio and Options Strategies	32
4.5.1	Cross-Sectional Mean-Reversion Portfolio	32
4.5.2	Options Market-Making with Delta Hedging	33
4.6	Challenges Faced and Resolved	33
4.6.1	Matching Engine Correctness	33
4.6.2	Real-Time Graphical Interface	34

CHAPTER 5 – TESTING AND RESULTS	35
5.1 Testing Strategy	35
5.2 Test Cases and Results	36
5.3 Performance Analysis	37
5.4 Strategy Backtest Results	39
5.5 Exported Tear Sheet from a Logged Run	42
5.6 Live Paper-Trading Validation	44
CHAPTER 6 – CONCLUSION AND FUTURE WORK	45
6.1 Conclusion	45
6.2 Project Significance and Broader Impact	46
6.3 Contribution Notes	46
6.4 Future Enhancements	47
6.5 Closing Remarks	48
References	49
APPENDIX A – SOURCE CODE HIGHLIGHTS	51
A.1 Order Type Enumeration	51
A.2 Black-Scholes Greeks	51
A.3 Performance Metrics in Python	53
APPENDIX B – SAMPLE OUTPUTS	54
B.1 Benchmark Console Output	54
B.2 Tear-Sheet Metrics Output	54
B.3 Order Book Depth Snapshot	55
B.4 Multi-Strategy Comparison	55
B.5 Parameter Sweep Optimizer	56

LIST OF FIGURES

Figure 3.1 – Four-layer architecture of QUANTSIM	13
Figure 3.2 – Level 0 (context) data flow diagram	14
Figure 3.3 – Level 1 data flow diagram	14
Figure 3.4 – Use case diagram	15
Figure 3.5 – Class diagram of the C++ core	16
Figure 3.6 – Sequence diagram for the order lifecycle	16
Figure 3.7 – Entity-relationship view of the platform’s persisted artifacts . . .	17
Figure 4.1 – The QUANTSIM desktop interface (Charts tab)	24
Figure 5.1 – Sample unit-test output	37
Figure 5.2 – Matching engine match latency percentiles	38
Figure 5.3 – Throughput comparison (C++ matching vs. Python backtester) . .	39
Figure 5.4 – Representative equity curves for three strategies	40
Figure 5.5 – Results tab of the desktop interface	41
Figure 5.6 – Profit-and-loss curve of an actual logged portfolio-strategy run .	43
Figure 5.7 – Live paper-trading session on the Binance BTCUSDT feed . . .	44
Figure B.1 – Compare tab of the desktop interface	56
Figure B.2 – Optimizer tab of the desktop interface	57

LIST OF TABLES

Table 2.1 – Comparison of trading simulation and backtesting frameworks . .	9
Table 3.1 – Functional requirements of QUANTSIM	11
Table 3.2 – Hardware environment	12
Table 3.3 – Software environment	12
Table 4.1 – Technology stack and justifications	20

Table 5.1 – Order book unit test cases and results	36
Table 5.2 – Matching engine unit test cases and results	36
Table 5.3 – Measured performance characteristics	38
Table 5.4 – Risk-adjusted performance of backtested strategies (synthetic data) 41	
Table 5.5 – Metrics from the exported tear sheet of a logged portfolio run (verbatim from JSON export)	43
Table 6.1 – Primary contribution areas by team member	47
Table B.1 – Sample order book depth snapshot (five levels per side)	55

LIST OF ABBREVIATIONS

Abbreviation	Full Form
ABI	Application Binary Interface
API	Application Programming Interface
AS	Avellaneda-Stoikov (market-making model)
ATM	At-The-Money
BS	Black-Scholes
CARA	Constant Absolute Risk Aversion
CLI	Command-Line Interface
CPU	Central Processing Unit
CSV	Comma-Separated Values
DFD	Data Flow Diagram
DMA	Direct Market Access
FIFO	First-In, First-Out
FOK	Fill-Or-Kill
FPS	Frames Per Second
GBM	Geometric Brownian Motion
GPU	Graphics Processing Unit
GUI	Graphical User Interface
HFT	High-Frequency Trading
IOC	Immediate-Or-Cancel
IS	Implementation Shortfall

Abbreviation	Full Form
ITCH	Nasdaq market-data binary protocol
IV	Implied Volatility
JSON	JavaScript Object Notation
LOB	Limit Order Book
MM	Market Making
OHLC	Open-High-Low-Close
OU	Ornstein-Uhlenbeck (process)
PnL	Profit and Loss
RNG	Random Number Generator
RSI	Relative Strength Index
SIMD	Single Instruction, Multiple Data
SMA	Simple Moving Average
TCA	Transaction Cost Analysis
TLS	Transport Layer Security
TWAP	Time-Weighted Average Price
VaR	Value at Risk
VWAP	Volume-Weighted Average Price
WSS	WebSocket Secure

QUANTSIM: A HIGH-FREQUENCY TRADING SIMULATION PLATFORM

A PROJECT REPORT SUBMITTED
IN PARTIAL FULFILMENT OF THE REQUIREMENT
FOR THE AWARD OF THE DEGREE
OF

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE & ENGINEERING



BY

ABHISHEK ANAND

22050440004

AMIK AFFAN

22050440017

SHAKIR AHMAD

22050440090

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
CAMBRIDGE INSTITUTE OF TECHNOLOGY**

TATISILWAI, RANCHI – 835103

JHARKHAND, INDIA

[2026]

DECLARATION CERTIFICATE

This is to certify that the work presented in the project entitled “**QuantSim: A High-Frequency Trading Simulation Platform**” in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering at Cambridge Institute of Technology, Tatisilwai, Ranchi is an authentic work carried out under my supervision and guidance.

To the best of my knowledge, the content of this project does not form a basis for the award of any previous Degree to anyone else.

Date: _____

PROF. DEEPAK KUMAR VERMA
Dept. of Computer Science & Engineering
Cambridge Institute of Technology
Tatisilwai, Ranchi – 835103

PROF. DEEPAK KUMAR VERMA
(Head of Department)
Dept. of Computer Science & Engineering
Cambridge Institute of Technology
Tatisilwai, Ranchi – 835103

CERTIFICATE OF APPROVAL

The foregoing project entitled “QuantSim: A High-Frequency Trading Simulation Platform” is hereby approved as a creditable work on the topic and has been presented satisfactorily to warrant its acceptance as a prerequisite to the degree for which it has been submitted.

It is understood that by this approval, the undersigned does not necessarily endorse any conclusion drawn or opinion expressed therein, but approves the project for the purpose for which it is submitted.

(Internal Examiner)

(External Examiner)

HEAD OF DEPARTMENT

Dept. of Computer Science and Engineering
Cambridge Institute of Technology
Tatisilwai, Ranchi – 835103

ACKNOWLEDGEMENT

I take this opportunity to thank all who have contributed towards shaping this Final Year Project Report. I would like to express my sincere thanks to Prof. Deepak Kumar Verma (HOD, Department of Computer Science & Engineering) and to my guide Prof. Deepak Kumar Verma, whose help in the course of development of this manuscript is invaluable. As my supervisor, he/she has constantly encouraged me to remain focused on achieving my goal. I would also like to thank the technical staff members of the Department who have been kind enough to advise and help in their respective roles. I have been fortunate to have a wonderful support structure among fellow students. My sincere thanks to everyone who has provided me with kind words, new ideas, useful criticism, or their valuable time. I am truly indebted. Last but not least, I would like to dedicate this work to my family for their love and understanding.

Shakir Ahmad

Roll No. 22050440090

ABSTRACT

High-Frequency Trading (HFT) depends on execution engines with sub-microsecond latency and faithful market-microstructure modelling, yet the mechanisms by which leading quantitative firms operate, limit order books, matching engines, queue priority, and inventory-aware market making, remain hidden inside proprietary, costly infrastructure. Compounding this, most accessible backtesting frameworks operate on aggregated Open-High-Low-Close (OHLC) candle data, ignoring the bid-ask spread, price-time priority, queue position, partial fills, transaction costs, and latency. These simplifications systematically overstate strategy performance and produce backtests that fail to generalize to live markets.

This dissertation presents QUANTSIM, an event-driven HFT simulation and backtesting platform that closes these gaps. The system is built around a low-latency C++ core exposing a full Limit Order Book (LOB) with First-In-First-Out (FIFO) price-time priority and a synchronous matching engine supporting eight order types (Limit, Market, IOC, FOK, Post-Only, Stop, Stop-Limit, Iceberg). A realistic execution layer models probabilistic maker fills, queue position, slippage, fees and rebates, borrow costs, and configurable latency, and a pre-trade risk engine adds a lock-free kill switch, rate limiter, position and notional limits, and a stale-price guard.

The platform ships five strategies, Avellaneda-Stoikov market making, Ornstein-Uhlenbeck pairs trading, TWAP execution, a multi-asset mean-reversion portfolio, and a Black-Scholes options market-maker with delta hedging, and bridges its C++ core to Python via pybind11 so that strategies and analytics are authored in Python while executing on the low-latency engine. A GPU-rendered Dear ImGui/ImPlot interface provides thirteen analytical tabs, an in-application C++ strategy compiler, and a Bookmap-style liquidity heatmap, and the same strategy binary runs unchanged in both the backtester and a C++-native live Binance paper-trading harness.

Empirical evaluation shows a matching-engine throughput above 14 million operations per second at a median latency below 100 nanoseconds, with all 27 unit tests passing. QUANTSIM thus unifies order-book-level realism, reproducibility, and interactive analysis in one free, open, and extensible platform, giving students and researchers direct access to execution mechanics previously confined to proprietary systems.

Keywords: High-Frequency Trading, Limit Order Book, Matching Engine, Market Microstructure, Avellaneda-Stoikov, Statistical Arbitrage, Event-Driven Backtesting, Low-Latency C++, pybind11, Dear ImGui.

TABLE OF CONTENTS

Declaration Certificate	i
Certificate of Approval	ii
Acknowledgement	iii
Abstract	iv
Table of Contents	v
List of Figures	viii
List of Tables	viii
List of Abbreviations	ix
CHAPTER 1 – INTRODUCTION	1
1.1 Background and Motivation	1
1.2 Problem Statement	3
1.3 Objectives of the Project	3
1.4 Scope of the Project	4
1.5 Organization of the Report	5
CHAPTER 2 – LITERATURE REVIEW	6
2.1 Related Works	6
2.1.1 Market Microstructure Theory	6
2.1.2 Optimal Market Making	6
2.1.3 Statistical Arbitrage and Pairs Trading	7
2.1.4 Optimal Execution	7
2.1.5 Options Pricing and Greeks	8
2.1.6 Low-Latency Systems Engineering	8
2.2 Comparative Analysis	8
2.3 Gap Identification	9
CHAPTER 3 – SYSTEM ANALYSIS AND DESIGN	10
3.1 System Requirements	10
3.1.1 Functional Requirements	10
3.1.2 Non-Functional Requirements	11

3.1.3	Hardware and Software Environment	12
3.2	System Architecture	12
3.3	Data Flow Diagrams	13
3.3.1	Level 0 (Context Diagram)	13
3.3.2	Level 1 (Detailed Diagram)	14
3.4	UML and ER Diagrams	14
3.4.1	Use Case Diagram	15
3.4.2	Class Diagram	15
3.4.3	Sequence Diagram	16
3.4.4	Entity-Relationship View	17
3.5	Design Patterns and Implementation Decisions	17
CHAPTER 4 – IMPLEMENTATION		19
4.1	Technologies and Tools Used	19
4.2	Module Description	20
4.2.1	Order and Fill Representation	20
4.2.2	Limit Order Book	21
4.2.3	Matching Engine	21
4.2.4	Market Data Feeds	21
4.2.5	Risk Engine	22
4.2.6	Metrics and Reporting	22
4.2.7	Python Bindings and Research Layer	22
4.2.8	Desktop Graphical Interface	22
4.2.9	In-Application C++ Strategy Compiler	24
4.2.10	Live Paper-Trading Harness	25
4.3	Algorithms and Methodology	25
4.3.1	Order Matching Algorithm	25
4.3.2	Avellaneda-Stoikov Market Making	26
4.3.3	Statistical Arbitrage	27
4.3.4	TWAP Execution and Implementation Shortfall	28
4.3.5	Black-Scholes Options Pricing	28
4.3.6	Performance Metrics	28
4.4	Code Implementation	29
4.5	Multi-Asset Portfolio and Options Strategies	32
4.5.1	Cross-Sectional Mean-Reversion Portfolio	32
4.5.2	Options Market-Making with Delta Hedging	33
4.6	Challenges Faced and Resolved	33
4.6.1	Matching Engine Correctness	33
4.6.2	Real-Time Graphical Interface	34

CHAPTER 5 – TESTING AND RESULTS	35
5.1 Testing Strategy	35
5.2 Test Cases and Results	36
5.3 Performance Analysis	37
5.4 Strategy Backtest Results	39
5.5 Exported Tear Sheet from a Logged Run	42
5.6 Live Paper-Trading Validation	44
CHAPTER 6 – CONCLUSION AND FUTURE WORK	45
6.1 Conclusion	45
6.2 Project Significance and Broader Impact	46
6.3 Contribution Notes	46
6.4 Future Enhancements	47
6.5 Closing Remarks	48
References	49
APPENDIX A – SOURCE CODE HIGHLIGHTS	51
A.1 Order Type Enumeration	51
A.2 Black-Scholes Greeks	51
A.3 Performance Metrics in Python	53
APPENDIX B – SAMPLE OUTPUTS	54
B.1 Benchmark Console Output	54
B.2 Tear-Sheet Metrics Output	54
B.3 Order Book Depth Snapshot	55
B.4 Multi-Strategy Comparison	55
B.5 Parameter Sweep Optimizer	56

LIST OF FIGURES

Figure 3.1 – Four-layer architecture of QUANTSIM	13
Figure 3.2 – Level 0 (context) data flow diagram	14
Figure 3.3 – Level 1 data flow diagram	14
Figure 3.4 – Use case diagram	15
Figure 3.5 – Class diagram of the C++ core	16
Figure 3.6 – Sequence diagram for the order lifecycle	16
Figure 3.7 – Entity-relationship view of the platform’s persisted artifacts . . .	17
Figure 4.1 – The QUANTSIM desktop interface (Charts tab)	24
Figure 5.1 – Sample unit-test output	37
Figure 5.2 – Matching engine match latency percentiles	38
Figure 5.3 – Throughput comparison (C++ matching vs. Python backtester) . .	39
Figure 5.4 – Representative equity curves for three strategies	40
Figure 5.5 – Results tab of the desktop interface	41
Figure 5.6 – Profit-and-loss curve of an actual logged portfolio-strategy run .	43
Figure 5.7 – Live paper-trading session on the Binance BTCUSDT feed . . .	44
Figure B.1 – Compare tab of the desktop interface	56
Figure B.2 – Optimizer tab of the desktop interface	57

LIST OF TABLES

Table 2.1 – Comparison of trading simulation and backtesting frameworks . .	9
Table 3.1 – Functional requirements of QUANTSIM	11
Table 3.2 – Hardware environment	12
Table 3.3 – Software environment	12
Table 4.1 – Technology stack and justifications	20

Table 5.1 – Order book unit test cases and results	36
Table 5.2 – Matching engine unit test cases and results	36
Table 5.3 – Measured performance characteristics	38
Table 5.4 – Risk-adjusted performance of backtested strategies (synthetic data)	41
Table 5.5 – Metrics from the exported tear sheet of a logged portfolio run (verbatim from JSON export)	43
Table 6.1 – Primary contribution areas by team member	47
Table B.1 – Sample order book depth snapshot (five levels per side)	55

LIST OF ABBREVIATIONS

Abbreviation	Full Form
ABI	Application Binary Interface
API	Application Programming Interface
AS	Avellaneda-Stoikov (market-making model)
ATM	At-The-Money
BS	Black-Scholes
CARA	Constant Absolute Risk Aversion
CLI	Command-Line Interface
CPU	Central Processing Unit
CSV	Comma-Separated Values
DFD	Data Flow Diagram
DMA	Direct Market Access
FIFO	First-In, First-Out
FOK	Fill-Or-Kill
FPS	Frames Per Second
GBM	Geometric Brownian Motion
GPU	Graphics Processing Unit
GUI	Graphical User Interface
HFT	High-Frequency Trading
IOC	Immediate-Or-Cancel
IS	Implementation Shortfall

Abbreviation	Full Form
ITCH	Nasdaq market-data binary protocol
IV	Implied Volatility
JSON	JavaScript Object Notation
LOB	Limit Order Book
MM	Market Making
OHLC	Open-High-Low-Close
OU	Ornstein-Uhlenbeck (process)
PnL	Profit and Loss
RNG	Random Number Generator
RSI	Relative Strength Index
SIMD	Single Instruction, Multiple Data
SMA	Simple Moving Average
TCA	Transaction Cost Analysis
TLS	Transport Layer Security
TWAP	Time-Weighted Average Price
VaR	Value at Risk
VWAP	Volume-Weighted Average Price
WSS	WebSocket Secure

QUANTSIM: A HIGH-FREQUENCY TRADING SIMULATION PLATFORM

A PROJECT REPORT SUBMITTED
IN PARTIAL FULFILMENT OF THE REQUIREMENT
FOR THE AWARD OF THE DEGREE
OF

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE & ENGINEERING



BY

ABHISHEK ANAND

22050440004

AMIK AFFAN

22050440017

SHAKIR AHMAD

22050440090

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
CAMBRIDGE INSTITUTE OF TECHNOLOGY**

TATISILWAI, RANCHI – 835103

JHARKHAND, INDIA

[2026]

DECLARATION CERTIFICATE

This is to certify that the work presented in the project entitled “**QuantSim: A High-Frequency Trading Simulation Platform**” in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering at Cambridge Institute of Technology, Tatisilwai, Ranchi is an authentic work carried out under my supervision and guidance.

To the best of my knowledge, the content of this project does not form a basis for the award of any previous Degree to anyone else.

Date: _____

PROF. DEEPAK KUMAR VERMA
Dept. of Computer Science & Engineering
Cambridge Institute of Technology
Tatisilwai, Ranchi – 835103

PROF. DEEPAK KUMAR VERMA
(Head of Department)
Dept. of Computer Science & Engineering
Cambridge Institute of Technology
Tatisilwai, Ranchi – 835103

CERTIFICATE OF APPROVAL

The foregoing project entitled “QuantSim: A High-Frequency Trading Simulation Platform” is hereby approved as a creditable work on the topic and has been presented satisfactorily to warrant its acceptance as a prerequisite to the degree for which it has been submitted.

It is understood that by this approval, the undersigned does not necessarily endorse any conclusion drawn or opinion expressed therein, but approves the project for the purpose for which it is submitted.

(Internal Examiner)

(External Examiner)

HEAD OF DEPARTMENT

Dept. of Computer Science and Engineering
Cambridge Institute of Technology
Tatisilwai, Ranchi – 835103

ACKNOWLEDGEMENT

I take this opportunity to thank all who have contributed towards shaping this Final Year Project Report. I would like to express my sincere thanks to Prof. Deepak Kumar Verma (HOD, Department of Computer Science & Engineering) and to my guide Prof. Deepak Kumar Verma, whose help in the course of development of this manuscript is invaluable. As my supervisor, he/she has constantly encouraged me to remain focused on achieving my goal. I would also like to thank the technical staff members of the Department who have been kind enough to advise and help in their respective roles. I have been fortunate to have a wonderful support structure among fellow students. My sincere thanks to everyone who has provided me with kind words, new ideas, useful criticism, or their valuable time. I am truly indebted. Last but not least, I would like to dedicate this work to my family for their love and understanding.

Amik Affan

Roll No. 22050440017

ABSTRACT

High-Frequency Trading (HFT) depends on execution engines with sub-microsecond latency and faithful market-microstructure modelling, yet the mechanisms by which leading quantitative firms operate, limit order books, matching engines, queue priority, and inventory-aware market making, remain hidden inside proprietary, costly infrastructure. Compounding this, most accessible backtesting frameworks operate on aggregated Open-High-Low-Close (OHLC) candle data, ignoring the bid-ask spread, price-time priority, queue position, partial fills, transaction costs, and latency. These simplifications systematically overstate strategy performance and produce backtests that fail to generalize to live markets.

This dissertation presents QUANTSIM, an event-driven HFT simulation and backtesting platform that closes these gaps. The system is built around a low-latency C++ core exposing a full Limit Order Book (LOB) with First-In-First-Out (FIFO) price-time priority and a synchronous matching engine supporting eight order types (Limit, Market, IOC, FOK, Post-Only, Stop, Stop-Limit, Iceberg). A realistic execution layer models probabilistic maker fills, queue position, slippage, fees and rebates, borrow costs, and configurable latency, and a pre-trade risk engine adds a lock-free kill switch, rate limiter, position and notional limits, and a stale-price guard.

The platform ships five strategies, Avellaneda-Stoikov market making, Ornstein-Uhlenbeck pairs trading, TWAP execution, a multi-asset mean-reversion portfolio, and a Black-Scholes options market-maker with delta hedging, and bridges its C++ core to Python via pybind11 so that strategies and analytics are authored in Python while executing on the low-latency engine. A GPU-rendered Dear ImGui/ImPlot interface provides thirteen analytical tabs, an in-application C++ strategy compiler, and a Bookmap-style liquidity heatmap, and the same strategy binary runs unchanged in both the backtester and a C++-native live Binance paper-trading harness.

Empirical evaluation shows a matching-engine throughput above 14 million operations per second at a median latency below 100 nanoseconds, with all 27 unit tests passing. QUANTSIM thus unifies order-book-level realism, reproducibility, and interactive analysis in one free, open, and extensible platform, giving students and researchers direct access to execution mechanics previously confined to proprietary systems.

Keywords: High-Frequency Trading, Limit Order Book, Matching Engine, Market Microstructure, Avellaneda-Stoikov, Statistical Arbitrage, Event-Driven Backtesting, Low-Latency C++, pybind11, Dear ImGui.

TABLE OF CONTENTS

Declaration Certificate	i
Certificate of Approval	ii
Acknowledgement	iii
Abstract	iv
Table of Contents	v
List of Figures	viii
List of Tables	viii
List of Abbreviations	ix
CHAPTER 1 – INTRODUCTION	1
1.1 Background and Motivation	1
1.2 Problem Statement	3
1.3 Objectives of the Project	3
1.4 Scope of the Project	4
1.5 Organization of the Report	5
CHAPTER 2 – LITERATURE REVIEW	6
2.1 Related Works	6
2.1.1 Market Microstructure Theory	6
2.1.2 Optimal Market Making	6
2.1.3 Statistical Arbitrage and Pairs Trading	7
2.1.4 Optimal Execution	7
2.1.5 Options Pricing and Greeks	8
2.1.6 Low-Latency Systems Engineering	8
2.2 Comparative Analysis	8
2.3 Gap Identification	9
CHAPTER 3 – SYSTEM ANALYSIS AND DESIGN	10
3.1 System Requirements	10
3.1.1 Functional Requirements	10
3.1.2 Non-Functional Requirements	11

3.1.3	Hardware and Software Environment	12
3.2	System Architecture	12
3.3	Data Flow Diagrams	13
3.3.1	Level 0 (Context Diagram)	13
3.3.2	Level 1 (Detailed Diagram)	14
3.4	UML and ER Diagrams	14
3.4.1	Use Case Diagram	15
3.4.2	Class Diagram	15
3.4.3	Sequence Diagram	16
3.4.4	Entity-Relationship View	17
3.5	Design Patterns and Implementation Decisions	17
CHAPTER 4 – IMPLEMENTATION		19
4.1	Technologies and Tools Used	19
4.2	Module Description	20
4.2.1	Order and Fill Representation	20
4.2.2	Limit Order Book	21
4.2.3	Matching Engine	21
4.2.4	Market Data Feeds	21
4.2.5	Risk Engine	22
4.2.6	Metrics and Reporting	22
4.2.7	Python Bindings and Research Layer	22
4.2.8	Desktop Graphical Interface	22
4.2.9	In-Application C++ Strategy Compiler	24
4.2.10	Live Paper-Trading Harness	25
4.3	Algorithms and Methodology	25
4.3.1	Order Matching Algorithm	25
4.3.2	Avellaneda-Stoikov Market Making	26
4.3.3	Statistical Arbitrage	27
4.3.4	TWAP Execution and Implementation Shortfall	28
4.3.5	Black-Scholes Options Pricing	28
4.3.6	Performance Metrics	28
4.4	Code Implementation	29
4.5	Multi-Asset Portfolio and Options Strategies	32
4.5.1	Cross-Sectional Mean-Reversion Portfolio	32
4.5.2	Options Market-Making with Delta Hedging	33
4.6	Challenges Faced and Resolved	33
4.6.1	Matching Engine Correctness	33
4.6.2	Real-Time Graphical Interface	34

CHAPTER 5 – TESTING AND RESULTS	35
5.1 Testing Strategy	35
5.2 Test Cases and Results	36
5.3 Performance Analysis	37
5.4 Strategy Backtest Results	39
5.5 Exported Tear Sheet from a Logged Run	42
5.6 Live Paper-Trading Validation	44
CHAPTER 6 – CONCLUSION AND FUTURE WORK	45
6.1 Conclusion	45
6.2 Project Significance and Broader Impact	46
6.3 Contribution Notes	46
6.4 Future Enhancements	47
6.5 Closing Remarks	48
References	49
APPENDIX A – SOURCE CODE HIGHLIGHTS	51
A.1 Order Type Enumeration	51
A.2 Black-Scholes Greeks	51
A.3 Performance Metrics in Python	53
APPENDIX B – SAMPLE OUTPUTS	54
B.1 Benchmark Console Output	54
B.2 Tear-Sheet Metrics Output	54
B.3 Order Book Depth Snapshot	55
B.4 Multi-Strategy Comparison	55
B.5 Parameter Sweep Optimizer	56

LIST OF FIGURES

Figure 3.1 – Four-layer architecture of QUANTSIM	13
Figure 3.2 – Level 0 (context) data flow diagram	14
Figure 3.3 – Level 1 data flow diagram	14
Figure 3.4 – Use case diagram	15
Figure 3.5 – Class diagram of the C++ core	16
Figure 3.6 – Sequence diagram for the order lifecycle	16
Figure 3.7 – Entity-relationship view of the platform’s persisted artifacts	17
Figure 4.1 – The QUANTSIM desktop interface (Charts tab)	24
Figure 5.1 – Sample unit-test output	37
Figure 5.2 – Matching engine match latency percentiles	38
Figure 5.3 – Throughput comparison (C++ matching vs. Python backtester)	39
Figure 5.4 – Representative equity curves for three strategies	40
Figure 5.5 – Results tab of the desktop interface	41
Figure 5.6 – Profit-and-loss curve of an actual logged portfolio-strategy run	43
Figure 5.7 – Live paper-trading session on the Binance BTCUSDT feed	44
Figure B.1 – Compare tab of the desktop interface	56
Figure B.2 – Optimizer tab of the desktop interface	57

LIST OF TABLES

Table 2.1 – Comparison of trading simulation and backtesting frameworks	9
Table 3.1 – Functional requirements of QUANTSIM	11
Table 3.2 – Hardware environment	12
Table 3.3 – Software environment	12
Table 4.1 – Technology stack and justifications	20

Table 5.1 – Order book unit test cases and results	36
Table 5.2 – Matching engine unit test cases and results	36
Table 5.3 – Measured performance characteristics	38
Table 5.4 – Risk-adjusted performance of backtested strategies (synthetic data) 41	
Table 5.5 – Metrics from the exported tear sheet of a logged portfolio run (verbatim from JSON export)	43
Table 6.1 – Primary contribution areas by team member	47
Table B.1 – Sample order book depth snapshot (five levels per side)	55

LIST OF ABBREVIATIONS

Abbreviation	Full Form
ABI	Application Binary Interface
API	Application Programming Interface
AS	Avellaneda-Stoikov (market-making model)
ATM	At-The-Money
BS	Black-Scholes
CARA	Constant Absolute Risk Aversion
CLI	Command-Line Interface
CPU	Central Processing Unit
CSV	Comma-Separated Values
DFD	Data Flow Diagram
DMA	Direct Market Access
FIFO	First-In, First-Out
FOK	Fill-Or-Kill
FPS	Frames Per Second
GBM	Geometric Brownian Motion
GPU	Graphics Processing Unit
GUI	Graphical User Interface
HFT	High-Frequency Trading
IOC	Immediate-Or-Cancel
IS	Implementation Shortfall

Abbreviation	Full Form
ITCH	Nasdaq market-data binary protocol
IV	Implied Volatility
JSON	JavaScript Object Notation
LOB	Limit Order Book
MM	Market Making
OHLC	Open-High-Low-Close
OU	Ornstein-Uhlenbeck (process)
PnL	Profit and Loss
RNG	Random Number Generator
RSI	Relative Strength Index
SIMD	Single Instruction, Multiple Data
SMA	Simple Moving Average
TCA	Transaction Cost Analysis
TLS	Transport Layer Security
TWAP	Time-Weighted Average Price
VaR	Value at Risk
VWAP	Volume-Weighted Average Price
WSS	WebSocket Secure

Chapter 1

INTRODUCTION

1.1 Background and Motivation

Financial markets have undergone a profound structural transformation over the past three decades. The migration from open-outcry trading floors to fully electronic, order-driven exchanges has compressed the timescale of trading from seconds to microseconds and, in the most competitive venues, to nanoseconds. Within this electronic ecosystem, High-Frequency Trading (HFT) has emerged as a dominant force, accounting for a substantial fraction of the daily volume on major equity, futures, and cryptocurrency exchanges. HFT firms deploy automated strategies that submit, modify, and cancel millions of orders per day, each holding positions for fractions of a second and seeking to capture vanishingly small but statistically reliable profits across enormous trade counts.

The defining characteristics of HFT are ultra-low-latency execution, an extremely high order-to-fill ratio, the systematic avoidance of overnight position risk, and the extraction of small per-trade margins amplified by volume. Success in this domain depends critically on a precise understanding of *market microstructure*, the body of theory and practice concerned with how the specific rules of an exchange, the available order types, and the behaviour of competing participants shape the process of price discovery. At the heart of every modern electronic exchange lies the Limit Order Book (LOB): a continuously updated, two-sided ledger of all outstanding buy and sell limit orders, organized by price and, within each price, by time of arrival.

Despite the central importance of microstructure, the tools available to students, researchers, and aspiring quantitative traders for studying these phenomena are markedly inadequate. The overwhelming majority of accessible backtesting frameworks operate on aggregated Open-High-Low-Close (OHLC) candle data sampled at intervals of one minute or longer. Such frameworks implicitly assume that an order is filled instantaneously.

neously at the candle's closing price, with no spread to cross, no queue to wait in, no partial fills, no market impact, and no latency between signal and execution. These assumptions are not merely simplifications; they are structurally incompatible with the economics of high-frequency strategies. A market-making strategy, for example, earns its profit precisely from the bid-ask spread and from its position in the order queue, the very quantities that candle-based backtesting erases. As a result, candle-based backtests routinely report Sharpe ratios that are unattainable in practice, producing a dangerous gap between simulated and realized performance.

A second barrier is one of access and cost. Historically, the infrastructure required for serious HFT research, co-located servers, direct market access, and tick-by-tick data feeds such as Nasdaq ITCH or proprietary vendor feeds from Bloomberg and Refinitiv, has been the exclusive preserve of well-capitalized institutional firms. The annual cost of a single professional data feed can exceed the entire budget of a university research group. This economic barrier has effectively excluded the academic community from hands-on study of the mechanisms that now govern modern markets.

The rise of cryptocurrency exchanges has dramatically altered this landscape. Venues such as Binance expose free, public WebSocket endpoints that stream full-depth order book updates and the complete trade tape, tick by tick, with no requirement for an account or an API key for read-only market data. This development has created, for the first time, an open sandbox in which the dynamics of a live, liquid, electronic order book can be observed and experimented upon without prohibitive cost. QUANTSIM was conceived to seize this opportunity: to build a single, coherent platform that combines the microstructural fidelity of an institutional matching engine, the analytical convenience of a Python research environment, and the immediacy of a real-time graphical interface, all validated against free live market data.

The immediate impetus for this project was a concrete and personal one. In seeking to understand how elite quantitative firms such as Jane Street and Citadel actually operate at the level of the order book, the authors drew on practitioner explanations popularized in online lectures [1] and reference glossaries [2], which conveyed the concepts clearly but offered no executable environment in which to manipulate them. No free simulator existed through which a student could see and experiment with these mechanics first-hand. Every adequate tool was either a closed institutional system inaccessible outside a trading firm, or an accessible candle-based backtester that hid precisely the mechanics worth studying. QUANTSIM was built to address this void: a free and open platform, engineered to the standards of a production trading system, that allows students to observe and manipulate the mechanics of quantitative trading directly rather than studying them only in the abstract.

The motivation for this project is therefore fourfold. First, there is a pedagogical motivation: to provide a faithful, transparent environment in which the mechanics of order matching, queue priority, and execution cost can be studied directly rather than abstracted away. Second, there is a research motivation: to enable rigorous, reproducible evaluation of trading strategies under realistic execution assumptions, including walk-forward and purged cross-validation that guard against the overfitting endemic to naive backtesting. Third, there is an engineering motivation: to demonstrate that the performance demanded by high-frequency simulation, millions of order operations per second at sub-microsecond latency, can be achieved within an extensible, well-tested, and approachable codebase rather than an opaque black box. Fourth, and binding the others together, there is a motivation of access: to deliver these capabilities at zero cost and in open form, so that the rigorous study of market microstructure is no longer the exclusive preserve of well-funded institutions but is available to any student or researcher with a laptop.

1.2 Problem Statement

Existing backtesting and trading-simulation tools fail to model the essential mechanics of modern electronic markets. They abstract away the limit order book, ignore price-time priority and queue position, assume instantaneous and costless fills, and disregard the latency between signal generation and order arrival. Consequently, they cannot faithfully evaluate latency-sensitive and microstructure-dependent strategies such as market making and statistical arbitrage, and they systematically overstate achievable performance. There is no widely accessible, open, and extensible platform that simultaneously delivers order-book-level matching-engine realism, a productive research interface for strategy development, real-time interactive visualization, and validation against live market data. This project addresses the absence of such a unified platform.

1.3 Objectives of the Project

The principal objectives of the QUANTSIM project are as follows:

- To design and implement a high-performance Limit Order Book and synchronous Matching Engine in modern C++ that enforces strict price-time (FIFO) priority and supports the full range of institutional order types, including Limit, Market, IOC, FOK, Post-Only, Stop, Stop-Limit, and Iceberg orders.
- To construct an event-driven backtesting framework that models realistic execution effects, probabilistic maker fills, queue position, slippage, exchange fees and

rebates, short-borrow costs, and configurable network latency, in place of the idealized fills assumed by candle-based tools.

- To implement a library of canonical quantitative trading strategies with rigorous mathematical foundations, including the Avellaneda-Stoikov market-making model, Ornstein-Uhlenbeck pairs trading, TWAP execution, a multi-asset mean-reversion portfolio, and a Black-Scholes options market-maker with delta hedging.
- To bridge the C++ core to Python through pybind11, enabling strategy authoring and analytics in a high-level language while retaining low-latency execution, and to provide a comprehensive research toolkit (signal library, walk-forward analysis, purged k -fold cross-validation, and tear-sheet reporting).
- To develop a real-time, GPU-rendered graphical interface that visualizes order-book depth, equity and drawdown curves, a Bookmap-style liquidity heatmap, correlation matrices, and Greeks surfaces, and that embeds an on-the-fly C++ strategy compiler.
- To integrate a pre-trade risk-management subsystem (kill switch, rate limiter, position and notional limits, stale-price guard) and to validate the entire platform against live Binance market data through a C++-native paper-trading harness.

1.4 Scope of the Project

The scope of QUANTSIM encompasses the complete simulation pipeline from market-data ingestion through strategy execution to performance analysis and visualization. On the data side, the platform ingests synthetic feeds (Geometric Brownian Motion, Ornstein-Uhlenbeck spreads, and a multi-asset factor model), historical CSV tick replays, the Nasdaq ITCH 5.0 binary protocol, and live Binance WebSocket streams. On the execution side, it provides a full LOB, a matching engine, and a realistic cost-and-latency model. On the strategy side, it ships a library of built-in strategies and supports user-authored strategies in both Python and C++.

The platform is intended for simulation, research, and paper trading; it is not a production order-routing gateway and does not place capital at risk on live venues. The live integration operates exclusively in paper-trading mode, generating synthetic fills against the real trade tape and a queue model. The matching engine is single-threaded and synchronous by design, prioritizing determinism and reproducibility over the multi-threaded concurrency of a production exchange. The cost and latency models are parameterized approximations rather than exact reproductions of any specific venue's fee schedule or network topology. These boundaries are deliberate: they keep the platform

tractable, deterministic, and pedagogically transparent while preserving the microstructural realism that distinguishes it from candle-based alternatives.

1.5 Organization of the Report

The remainder of this dissertation is organized into five further chapters. Chapter 2 surveys the relevant literature spanning market microstructure theory, optimal market-making models, statistical arbitrage, execution algorithms, and software-engineering practice for low-latency systems; it then presents a comparative analysis of existing tools and identifies the research gap that QUANTSIM addresses. Chapter 3 details the system analysis and design, including functional and non-functional requirements, the layered architecture, data flow diagrams, and the design patterns employed. Chapter 4 describes the implementation in depth, covering the technologies used, the structure of each major module, the core algorithms with pseudo-code, and annotated code samples. Chapter 5 presents the testing strategy, the unit-test suite, performance benchmarks, and empirical strategy results. Chapter 6 concludes the dissertation and outlines directions for future work. The report ends with a list of references in IEEE format and two appendices containing source-code highlights and sample outputs.

Chapter 2

LITERATURE REVIEW

This chapter situates QUANTSIM within the existing body of research and practice. Section 2.1 surveys the foundational and contemporary literature across five themes: market microstructure theory, optimal market-making models, statistical arbitrage, execution algorithms, and the software engineering of low-latency trading systems. Section 2.2 presents a structured comparison of existing simulation and backtesting tools. Section 2.3 synthesizes these findings to articulate the specific research gap that this project addresses.

2.1 Related Works

2.1.1 Market Microstructure Theory

The theoretical study of how trading mechanisms determine prices is surveyed comprehensively by O'Hara [3], who established market microstructure as a distinct discipline synthesizing inventory-based and information-based models of the bid-ask spread. The information-based view was crystallized by Kyle [4], whose model of a strategic informed trader interacting with noise traders and a competitive market maker yields the celebrated linear price-impact relationship and the concept of market depth, both directly relevant to the limit-order-book dynamics that this project simulates.

2.1.2 Optimal Market Making

The canonical model for optimal market making is due to Avellaneda and Stoikov [5], who cast the dealer's problem as one of stochastic optimal control. Assuming a mid-price following arithmetic Brownian motion and an exponential (CARA) utility over terminal wealth, they derive closed-form approximations for the reservation price, the

inventory-adjusted price at which the dealer is indifferent to trading, and for the optimal bid-ask spread as a function of risk aversion, volatility, time horizon, and order-arrival intensity. Their key insight is that the dealer should skew quotes around the reservation price rather than the mid-price, automatically biasing the book to mean-revert inventory toward zero. This model is the foundation of the market-making strategy implemented in QUANTSIM and is developed in detail in Chapter 4.

The Avellaneda-Stoikov framework sits within the broader treatment of Cartea, Jaimungal, and Penalva [6], whose text presents algorithmic and high-frequency trading, market making, optimal execution, and statistical arbitrage within a common stochastic-control language, and is a primary reference for the strategy formulations adopted in this project.

2.1.3 Statistical Arbitrage and Pairs Trading

Statistical arbitrage exploits temporary mispricings between statistically related instruments. The classic pairs-trading approach, documented by Gatev, Goetzmann, and Rouwenhorst [7] in their study of the U.S. equity market, identifies pairs of securities whose prices have historically moved together and trades the spread when it deviates from its historical norm, betting on reversion. The statistical foundation for trading such a spread is the mean-reverting Ornstein-Uhlenbeck (OU) process, which reverts to a long-run mean at a rate set by a mean-reversion speed parameter, together with the related concept of cointegration, which determines when a linear combination of non-stationary price series is itself stationary and therefore tradeable. QUANTSIM implements both a synthetic OU-spread feed for controlled experiments and a z-score-based pairs-trading strategy, described in Chapter 4.

2.1.4 Optimal Execution

When a trader must execute a large order, naive immediate execution incurs substantial market impact. The seminal framework of Almgren and Chriss [8] formalizes the trade-off between market impact (which favours slow execution) and price risk (which favours fast execution), deriving an optimal execution trajectory that balances the two according to the trader's risk aversion. The Time-Weighted Average Price (TWAP) and Volume-Weighted Average Price (VWAP) algorithms are widely used industrial benchmarks for execution quality; the associated performance metric, Implementation Shortfall, captures the total cost of executing a decision relative to the price prevailing when the decision was made. QUANTSIM implements a TWAP execution strategy and reports Implementation Shortfall as described in Chapter 4.

2.1.5 Options Pricing and Greeks

The pricing of options market-making activity in QUANTSIM rests on the Black-Scholes model [9], which provides a closed-form price for a European option under the assumption of geometric Brownian motion for the underlying and continuous, costless hedging. The partial derivatives of the option price with respect to its inputs, collectively the “Greeks” (delta, gamma, vega, theta, rho), quantify the option’s sensitivities and are the basis for dynamic hedging. Hull [10] provides the standard reference treatment of options, futures, and the practice of delta hedging that underlies the options strategy implemented here.

2.1.6 Low-Latency Systems Engineering

The performance characteristics of QUANTSIM draw on a body of systems-engineering knowledge concerned with low-latency, cache-conscious software. The general principles of mechanical sympathy, cache-line alignment to avoid false sharing, and lock-free data structures are central to the metrics and risk subsystems of this platform. The C++ language features exploited, `std::map` for ordered price levels, `std::list` for intrusive FIFO queues, `std::atomic` for lock-free counters, and the `std::mdspan` multidimensional view (standardized in C++23, compiled here under the C++26 standard) for the Greeks surface, are documented in the ISO C++ standard and on the cp-preference site [11]. The interoperability between C++ and Python is achieved through the pybind11 library [12], which provides a header-only, zero-overhead binding layer. The graphical interface is built on Dear ImGui [13], an immediate-mode GUI library whose stateless, redraw-every-frame architecture is well suited to high-throughput data visualization.

2.2 Comparative Analysis

Several open-source and commercial tools exist for trading-strategy simulation, each occupying a different point in the trade-off space between realism, performance, accessibility, and feature breadth. Table 2.1 compares the most prominent of these against QUANTSIM across the dimensions most relevant to high-frequency research.

Table 2.1. Comparison of trading simulation and backtesting frameworks

Feature	Backtrader	Zipline	Backtesting.py	QUANTSIM
Data granularity	Bar/candle	Daily/minute	Bar/candle	Tick / L2
Limit order book	No	No	No	Yes (FIFO)
Queue priority	No	No	No	Yes
Order types	Basic	Basic	Basic	8 types
Latency model	No	No	No	Yes
Fees/rebates	Partial	Partial	Partial	Maker/taker
Native speed	Python	Python	Python	C++ core
Live integration	Broker API	No	No	WebSocket
Real-time GUI	No	No	Plot only	GPU/ImGui
Options/Greeks	No	No	No	Yes
On-the-fly compile	No	No	No	Yes (C++)

The comparison reveals a consistent pattern. The Python-based frameworks, Backtrader, Zipline, and Backtesting.py, are accessible and feature-rich at the level of portfolio and signal logic, but they operate exclusively on aggregated bar data and model neither the order book, queue priority, nor latency, making them unsuitable for strategies whose economics depend on microstructure. Commercial simulators that do model the order book are typically proprietary, expensive, and closed. QUANTSIM occupies the previously vacant intersection: order-book-level realism with a native C++ engine, while remaining open, extensible, and approachable through a Python interface and an interactive GUI.

2.3 Gap Identification

The survey above exposes a clear gap. On one side stand accessible, open-source backtesting libraries that sacrifice microstructural realism for ease of use; their candle-based models cannot represent the spread capture, queue position, and latency sensitivity that define high-frequency strategies. On the other side stand institutional simulators that capture this realism but are closed, costly, and pedagogically opaque. No widely available platform unifies an order-book-level matching engine with strict price-time priority and a full set of order types, a realistic execution-cost and latency model, a productive research interface, real-time visualization, and validation against live market data, all in a single, open, and extensible system. This gap is precisely what QUANTSIM is designed to fill, combining a native C++ core, a pybind11 bridge, an ImGui front-end, and a WebSocket harness; the chapters that follow describe how this synthesis was designed, implemented, and validated.

Chapter 3

SYSTEM ANALYSIS AND DESIGN

This chapter presents the analysis and design of QUANTSIM. Section 3.1 enumerates the functional and non-functional requirements together with the hardware and software environment. Section 3.2 describes the layered system architecture. Section 3.3 presents the data flow diagrams at two levels of abstraction. Section 3.5 discusses the design patterns and key implementation decisions that give the system its structure.

3.1 System Requirements

3.1.1 Functional Requirements

The functional requirements specify the observable behaviour the system must exhibit. They are summarized in Table 3.1.

Table 3.1. Functional requirements of QUANTSIM

ID	Requirement
FR-1	The system shall maintain a two-sided limit order book with price-time (FIFO) priority for each traded symbol.
FR-2	The matching engine shall support Limit, Market, IOC, FOK, Post-Only, Stop, Stop-Limit, and Iceberg order types.
FR-3	The system shall generate synthetic market data via GBM, OU-spread, and multi-asset factor models, and shall replay CSV and ITCH 5.0 data.
FR-4	The system shall execute built-in strategies (market making, statistical arbitrage, TWAP, portfolio, options) and user-authored strategies.
FR-5	The backtester shall model probabilistic maker fills, queue position, slippage, fees, rebates, borrow cost, and configurable latency.
FR-6	The system shall compute performance metrics including Sharpe, Sortino, Calmar, maximum drawdown, volatility, profit factor, and win rate.
FR-7	The risk engine shall enforce a kill switch, rate limiting, position and notional limits, and a stale-price guard before order submission.
FR-8	The GUI shall render order-book depth, equity and drawdown curves, a liquidity heatmap, correlation matrices, and Greeks surfaces in real time.
FR-9	The system shall compile user-written C++ strategies to shared libraries at runtime and load them without restarting.
FR-10	The system shall connect to the Binance WebSocket feed and execute strategies in paper-trading mode against live data.
FR-11	The system shall export tear-sheet reports in JSON, CSV, and self-contained HTML formats.

3.1.2 Non-Functional Requirements

The non-functional requirements constrain the quality attributes of the system. Chief among them is performance: the matching engine must sustain a throughput on the order of millions of operations per second with sub-microsecond per-order latency, since a high-frequency simulation processing millions of ticks would otherwise be intractable. Reproducibility is equally critical: given the same seed and parameters, a simulation must produce bit-identical results, which mandates a single-threaded, deterministic matching core and a controlled random-number generator. The system must remain responsive, sustaining a 60-frames-per-second interface even while a simulation runs on a background thread; this dictates a clean separation between the simulation thread and the render thread, mediated by a mutex-protected shared state. Finally, the platform must be extensible, new strategies, feeds, and metrics must be addable without modifying the core, and portable across the development platform.

3.1.3 Hardware and Software Environment

Tables 3.2 and 3.3 list the hardware and software environment used for development and evaluation.

Table 3.2. Hardware environment

Component	Specification
Processor	Multi-core x86-64 / ARM64 with AVX support
Memory	8 GB RAM minimum (16 GB recommended)
Graphics	OpenGL 3.3-capable GPU
Storage	2 GB free disk space for build artifacts and data
Network	Broadband connection for live WebSocket feed

Table 3.3. Software environment

Component	Specification
Operating system	macOS (Darwin) / Linux
C++ compiler	Clang or GCC (C++23, with selective C++26 features)
Build system	CMake 3.20 or later
Python	Python 3.10 or later
Python packages	pybind11, NumPy, pandas, matplotlib, yfinance
GUI libraries	Dear ImGui, ImPlot, GLFW, OpenGL
Networking	OpenSSL (for live WebSocket-over-TLS)

3.2 System Architecture

QUANTSIM adopts a layered architecture that cleanly separates the low-latency execution core from the research, presentation, and live-trading layers. Figure 3.1 depicts the four principal layers and the flow of control and data between them. At the foundation sits the C++ core, comprising the limit order book, the matching engine, the market-data feeds, the risk engine, and the metrics subsystem. Above this, the pybind11 bridge exposes the core to Python, where the research layer provides strategies, signals, analytics, and validation. The presentation layer, the ImGui desktop application, communicates with the core through a mutex-protected shared state populated by a background simulation runner. A parallel live-trading path connects the same strategy binaries, via a C++-native WebSocket client, to the Binance feed.

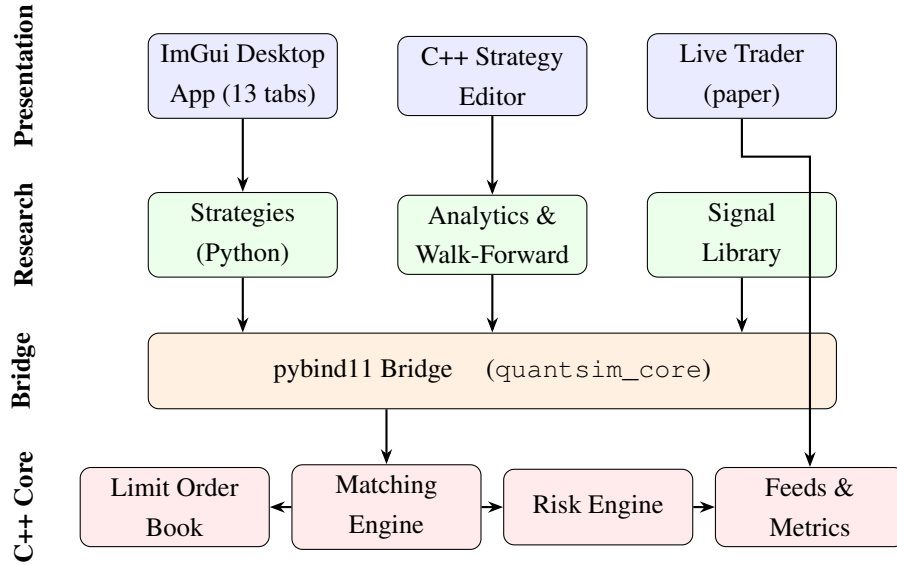


Figure 3.1. Four-layer architecture of QUANTSIM. The C++ core provides low-latency execution; the pybind11 bridge exposes it to the Python research layer; the ImGui presentation layer visualizes results; the live trader reuses core feeds and strategy binaries.

The rationale for this layering is the separation of concerns by performance requirement. Components on the critical execution path, order matching, risk checks, metrics, reside in the C++ core where every nanosecond matters. Components on the productivity path, strategy logic, statistical analysis, plotting, reside in Python where developer velocity matters more than raw speed. The pybind11 bridge mediates the two, paying a marshalling cost only at the boundary, which is amortized by batch interfaces for the hottest operations. The presentation layer never touches the core directly; it reads a snapshot of shared state under a mutex, decoupling render cadence from simulation cadence entirely.

3.3 Data Flow Diagrams

3.3.1 Level 0 (Context Diagram)

The Level 0 data flow diagram, shown in Figure 3.2, treats the entire QUANTSIM platform as a single process and identifies its external entities and the data that flows between them. The user supplies strategy definitions and configuration parameters and receives performance metrics, charts, and reports. Market-data sources, synthetic generators, historical files, and the live Binance feed, supply tick and order-book data. The file system receives exported reports and supplies historical data and compiled strategy binaries.

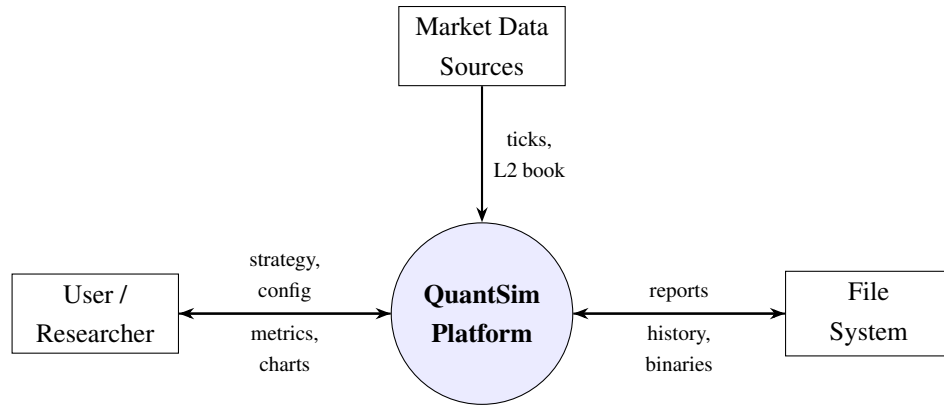


Figure 3.2. Level 0 (context) data flow diagram. The platform exchanges configuration and results with the user, ingests market data, and reads from and writes to the file system.

3.3.2 Level 1 (Detailed Diagram)

The Level 1 diagram, Figure 3.3, decomposes the platform into its principal processes and data stores. A tick from a feed enters the simulation runner, which forwards the order-book state to the active strategy. The strategy emits orders, which pass through the risk engine before reaching the matching engine. The matching engine updates the order book and emits fills, which the metrics process consumes to update positions, PnL, and the equity curve. The shared state store mediates between the simulation thread and the rendering process.

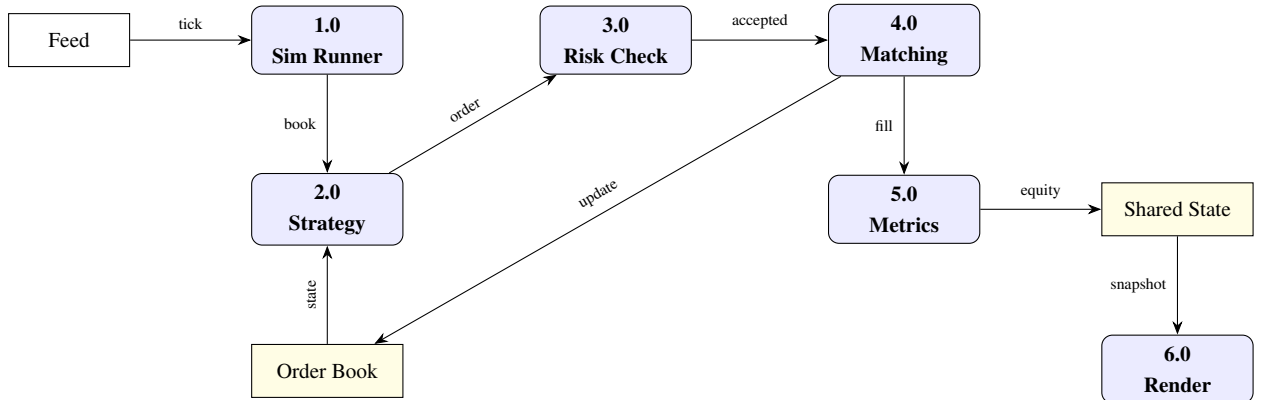


Figure 3.3. Level 1 data flow diagram. Ticks drive the simulation runner; orders flow through the risk engine to the matching engine; fills update metrics and the shared state, which the render process reads.

3.4 UML and ER Diagrams

Because QUANTSIM is a computational platform rather than a database-centric application, its structure is best captured by behavioural and structural UML diagrams rather

than an entity-relationship schema. This section presents a use-case diagram describing the interactions available to the user, a class diagram of the core engine, and a sequence diagram tracing the lifecycle of an order. A simplified entity-relationship view of the platform's persistent data is also given for completeness.

3.4.1 Use Case Diagram

Figure 3.4 shows the principal use cases of the platform from the perspective of two actors: the researcher, who configures and runs simulations and analyses results, and the external market-data feed, which supplies ticks to the running simulation. The researcher can select or author a strategy, configure run parameters, execute a backtest, inspect results and metrics, compile a custom C++ strategy, and launch a live paper-trading session.

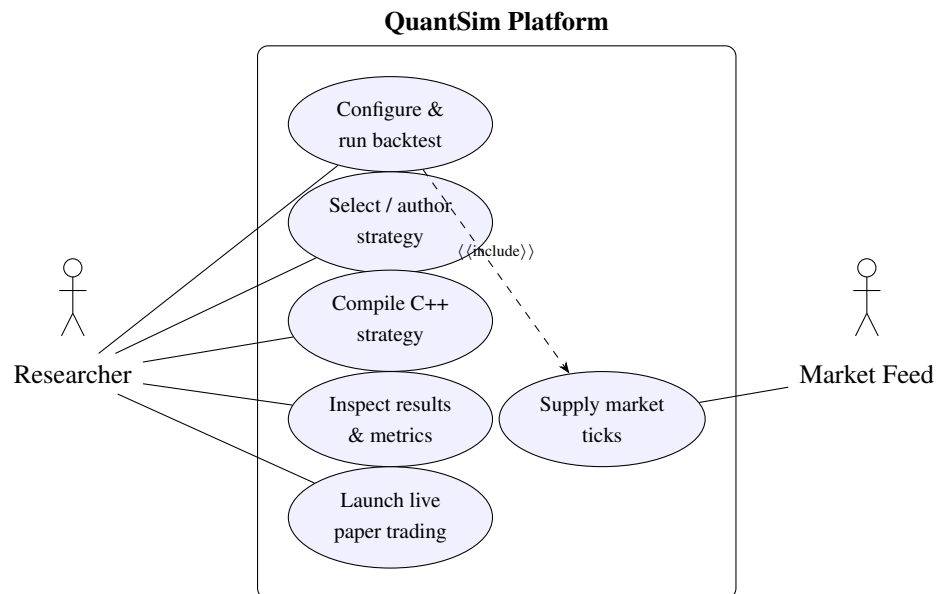


Figure 3.4. Use case diagram. The researcher drives configuration, execution, and analysis; the market feed supplies ticks consumed during a run.

3.4.2 Class Diagram

Figure 3.5 presents a class diagram of the C++ core. The `MatchingEngine` aggregates one `LimitOrderBook` per symbol and consults the `PreTradeRisk` engine before accepting an order; each book is composed of `PriceLevel` objects that in turn hold `Order` objects in FIFO queues. Matching produces `Fill` objects that are recorded by `TradingMetrics`. The `SimRunner` orchestrates the run, pulling ticks from a `Feed` and dispatching them to the active `Strategy`.

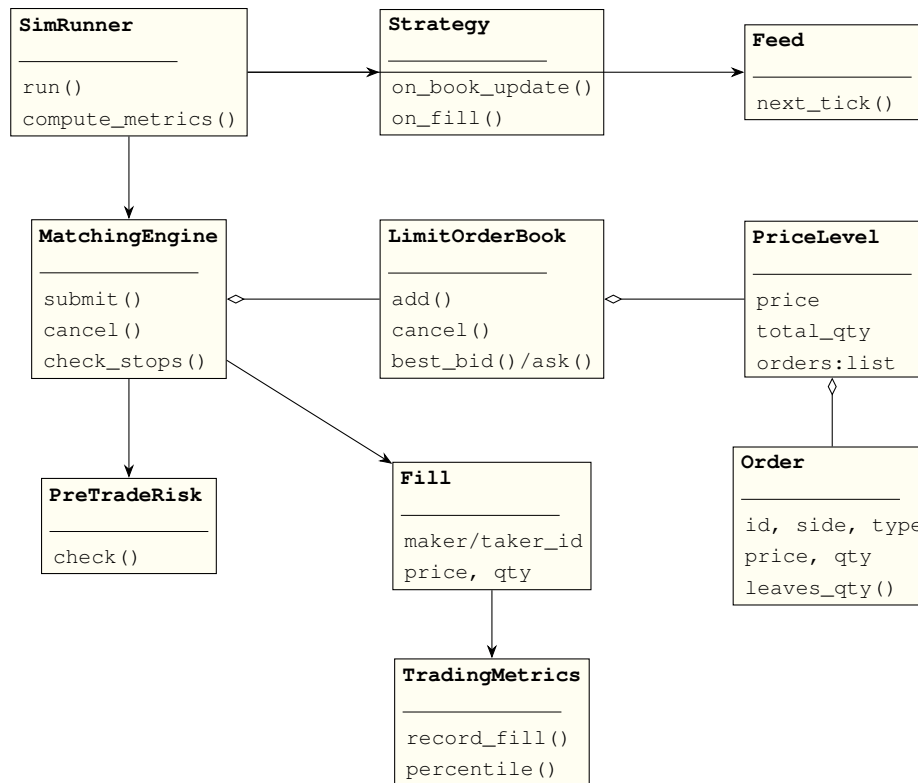


Figure 3.5. Class diagram of the C++ core. Diamonds denote composition/aggregation; arrows denote dependency.

3.4.3 Sequence Diagram

Figure 3.6 traces the lifecycle of a single order through the system. A tick arrives from the feed and is handed by the simulation runner to the strategy, which decides to submit an order. The order passes through the risk engine, then to the matching engine, which matches it against the book and emits a fill; the fill propagates back to the metrics collector and ultimately to the shared state read by the interface.

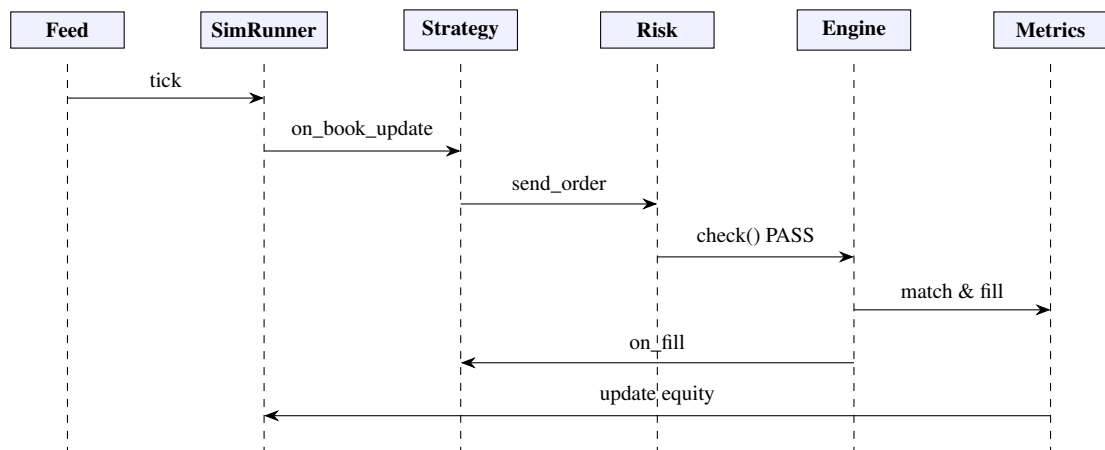


Figure 3.6. Sequence diagram for the order lifecycle, from tick arrival through risk check and matching to fill notification and equity update.

3.4.4 Entity-Relationship View

Although QUANTSIM is computation-centric and holds most state in memory, its persisted artifacts, exported tear sheets and recorded runs, can be described by the simple entity-relationship view of Figure 3.7. A RUN is parameterized by a STRATEGY and a FEED configuration and produces many FILL records and a single METRICSREPORT.

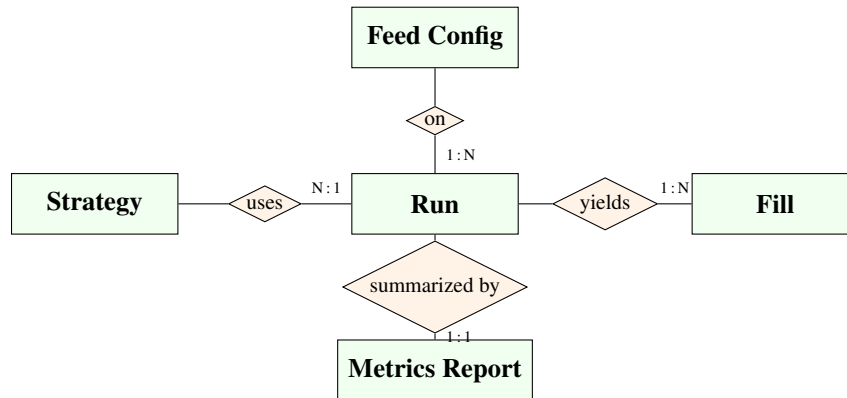


Figure 3.7. Entity-relationship view of the platform's persisted artifacts. A run uses a strategy and feed configuration, yields many fills, and is summarized by one metrics report.

3.5 Design Patterns and Implementation Decisions

The design of QUANTSIM embodies several recognized design patterns and deliberate structural decisions. The matching engine employs the **Observer pattern** through a set of callbacks, `on_fill`, `on_add`, `on_cancel`, and `on_book_update`, that decouple the engine from the consumers of its events. A strategy, a metrics collector, and a logger can all subscribe to fills without the engine knowing anything about them, which keeps the core free of presentation and accounting concerns.

The strategy subsystem follows the **Strategy pattern** (in both the design-pattern and the finance sense). Each trading strategy implements a common interface, in Python an abstract base class with an `on_book_update` method, and in C++ a stable Application Binary Interface (ABI) of `extern "C"` entry points. Because the interface is fixed, the engine can drive any strategy polymorphically, and the same compiled C++ strategy binary runs unchanged in both the backtester and the live trader. This is the single most important architectural decision in the system: it guarantees that what is tested in simulation is exactly what executes against live data, eliminating the backtest-to-live divergence that plagues many trading systems.

The order book uses an **intrusive container design**: each price level holds a `std::list` of order pointers providing the FIFO queue, while a separate hash map indexes every live order by its identifier, storing an iterator into the appropriate list. This dual structure

yields $O(\log N)$ price lookup through the ordered map of price levels and $O(1)$ cancellation through the hash-map-to-iterator indirection, combining the ordering guarantees of a balanced tree with the constant-time removal of a linked list.

The shared state between the simulation and render threads is protected by a single mutex and accessed through a **snapshot discipline**: the render thread acquires the lock only briefly to copy the data it needs, then releases it before drawing. This keeps lock contention negligible and ensures the user interface never blocks the simulation. Finally, the metrics and risk subsystems use **cache-line alignment** (`alignas(64)`) and lock-free atomics to keep the hot path free of false sharing and lock overhead, a decision justified by the performance benchmarks reported in Chapter 5.

Chapter 4

IMPLEMENTATION

This chapter describes the implementation of QUANTSIM in detail. Section 4.1 justifies the choice of technologies. Section 4.2 describes each major module. Section 4.3 presents the core algorithms with pseudo-code and mathematical formulations. Section 4.4 provides annotated code samples of the most important components.

4.1 Technologies and Tools Used

The implementation language for the core is modern C++ (C++23, with selective use of C++26 features). C++ was chosen because high-frequency simulation is fundamentally a performance problem: processing millions of order events per second at sub-microsecond latency is achievable in a systems language with manual memory control, zero-cost abstractions, and predictable performance, but not in a managed or interpreted runtime. The core is largely header-only, which enables aggressive inlining and link-time optimization and simplifies the build.

Python serves as the research and strategy-authoring language. Its rich scientific ecosystem, NumPy for vectorized numerics, pandas for tabular analysis, and matplotlib for plotting, makes it the natural environment for statistical work, and its low ceremony accelerates strategy iteration. The two languages are joined by **pybind11**, a header-only binding library that exposes C++ classes and functions to Python with negligible overhead and automatic type conversion.

The graphical interface is built on **Dear ImGui** [13] and its plotting companion **ImPlot**, rendered through **GLFW** [14] and **OpenGL**. ImGui’s immediate-mode paradigm, in which the entire interface is redrawn from scratch every frame directly on the GPU, avoids the retained-mode bookkeeping that makes traditional toolkits struggle under high-frequency data streams. The live-trading harness uses **OpenSSL** [15] to imple-

ment a WebSocket-over-TLS client with no heavyweight dependencies. The build is orchestrated by **CMake** [16], and the C++ code is compiled with `-O3 -march=native` to enable full optimization and auto-vectorization. Table 4.1 summarizes the technology stack and the justification for each choice.

Table 4.1. Technology stack and justifications

Technology	Role	Justification
C++23/26	Core engine	Sub-microsecond latency, zero-cost abstractions
Python 3.10+	Research, strategies	Scientific ecosystem, rapid iteration
pybind11	Language bridge	Header-only, low-overhead bindings
Dear ImGui / ImPlot	GUI	Immediate-mode, GPU-rendered, high throughput
GLFW / OpenGL	Windowing / render	Cross-platform, lightweight
OpenSSL	Live WSS client	TLS without heavyweight dependencies
CMake	Build system	Cross-platform, dependency management
NumPy / pandas	Analytics	Vectorized numerics, tabular analysis

4.2 Module Description

4.2.1 Order and Fill Representation

The fundamental data types are defined in `order.hpp`. The `Order` structure carries a unique identifier, the trading symbol, the side (buy or sell), the order type, the limit price, the requested and filled quantities, the status, a nanosecond timestamp, and auxiliary fields for advanced order types (a stop trigger price and a display quantity for icebergs). Two derived quantities, the leaves quantity (unfilled remainder) and a done predicate, are computed inline. The `Fill` structure records a matched trade: the maker and taker identifiers, the execution price and quantity, the aggressing side, the timestamp, the symbol, and flags indicating whether each counterparty was a background bot. A `BookSnapshot` captures the top of book, best bid, best ask, and their sizes, at an instant in time.

4.2.2 Limit Order Book

The `LimitOrderBook` class, declared in `orderbook.hpp` and implemented in `orderbook.cpp`, is the heart of the microstructure model. It stores bids in a `std::map` ordered by descending price and asks in a `std::map` ordered by ascending price, so that the best bid and best ask are always at the front of their respective maps. Each map value is a `PriceLevel` holding the aggregate quantity at that price and a `std::list` of order pointers maintaining FIFO time priority. A separate `std::unordered_map` indexes every live order by identifier, storing its side, price, and an iterator into the price level's list, which makes cancellation $O(1)$. The class exposes queries for best prices and sizes, the spread, the mid-price, the depth at a given price, and the top- N depth on each side, as well as a snapshot method for the GUI.

4.2.3 Matching Engine

The `MatchingEngine` class (`matching.hpp`, `matching.cpp`) owns one order book per symbol and routes incoming orders according to their type. It maintains side tables for stop orders held off-book, for iceberg reserves, and for the mapping from order to symbol. When an order is submitted, the engine assigns an identifier, applies type-specific routing, holding stops, rejecting crossing post-only orders, splitting icebergs, pre-checking fill-or-kill liquidity, and, for executable orders, sweeps the opposite side of the book greedily, level by level and order by order, generating fills. After every book-changing event the engine re-evaluates held stop orders, which may cascade into further executions. The engine fires its observer callbacks synchronously so that downstream consumers see a consistent, ordered stream of events.

4.2.4 Market Data Feeds

The feed module (`feed.hpp`) provides several sources of market data behind a common `TickEvent` abstraction. The `SyntheticFeed` generates prices following Geometric Brownian Motion via an Euler-Maruyama discretization. The `OUSpreadFeed` generates two correlated assets whose spread follows an Ornstein-Uhlenbeck mean-reverting process, supporting pairs-trading experiments. The `MultiAssetFeed` (in `multi_asset.hpp`) implements a factor model in which a shared market level drives all assets through per-asset betas, with idiosyncratic OU residuals layered on top, supporting portfolio experiments with realistic correlation structure. The `ITCHParser` decodes the Nasdaq ITCH 5.0 binary protocol, and `CsvTickReplay` loads and replays recorded tick data with optional speed control.

4.2.5 Risk Engine

The risk module (`risk.hpp`) implements pre-trade controls. The `KillSwitch` is a cache-line-aligned atomic flag that, when armed, halts all order submission; its hot-path check completes in a few nanoseconds using a relaxed atomic load. The `TokenBucket` enforces a maximum order rate with a configurable burst allowance. The `PreTradeRisk` class composes these with checks on order quantity, aggregate notional exposure, and the deviation of a limit price from the prevailing mid, returning a typed result that names the specific limit breached. The `StalePriceGuard` arms the kill switch automatically if the market-data feed stalls beyond a configurable age, protecting against trading on stale prices.

4.2.6 Metrics and Reporting

The metrics module (`metrics.hpp`) maintains atomic counters for orders sent, filled, cancelled, and rejected, together with the net position and gross profit-and-loss, all cache-line aligned to avoid false sharing. A latency histogram with microsecond buckets records the order-to-fill round-trip and supports percentile queries. The reporting module (`report.hpp`) exports a complete tear sheet, metrics, equity series, and fills, as machine-readable JSON, as CSV, and as a self-contained HTML dashboard with inline charts requiring no external network resources.

4.2.7 Python Bindings and Research Layer

The `pybind11` bindings (`python/bindings.cpp`) expose the order types, the order book, the matching engine, the risk classes, the metrics, and the feeds to Python as the `quantsim_core` module. A batch submission interface builds many orders in a single native call to amortize the marshalling cost on the hot path. The Python research layer provides an abstract `Strategy` base class with event callbacks, a `BacktestEngine` that drives the simulation and models resting-order fills probabilistically, an analytics module computing the full suite of performance metrics, a signal library (EMA, SMA, RSI, Bollinger Bands, and composite signals), and validation tools including walk-forward analysis and purged k -fold cross-validation.

4.2.8 Desktop Graphical Interface

The graphical interface is implemented in `app_imgui.cpp`, the largest single source file in the project at approximately two thousand lines. It is built on Dear ImGui for

widgets, ImPlot for charting, GLFW for windowing, and an OpenGL 3 backend for rendering. The application follows the immediate-mode paradigm: rather than constructing and retaining a tree of widget objects, the entire interface is re-declared and redrawn from scratch on every frame inside a single render loop. Each frame the loop polls input, locks the shared `SimState` just long enough to copy the data it needs, releases the lock, and then issues the ImGui draw calls for whichever tab is active. Because the simulation runs on a separate background thread and the interface only ever reads a brief snapshot of shared state, the render loop sustains sixty frames per second without ever blocking the simulation or displaying a torn, half-updated view.

The interface is organized into thirteen tabs, each created with an `ImGui::BeginTabItem` call: **Dashboard** (live order-book ladder, fill log, microprice, and queue imbalance); **Charts** (profit-and-loss curve and the liquidity heatmap); **Fills** and **Bot Fills** (trade logs with slippage, separated into strategy fills and pure bot-to-bot activity); **Results** (equity and drawdown series and the full metrics table); **Editor** (the in-application C++ strategy editor described below); **Terminal** (an embedded command runner with the Python path preconfigured); **Compare** (side-by-side overlay of up to five strategy runs); **Optimizer** (one- and two-dimensional parameter sweeps); **Risk** (pre-trade limits, value-at-risk histogram, and kill-switch status); **Options** (Black-Scholes theoretical price and the Greeks surface); **TCA** (transaction-cost analysis with a profit-and-loss bridge); and **Log** (an event scrollbar). Charts are drawn with ImPlot primitives; in particular, the Bookmap-style liquidity heatmap is rendered with `ImPlot::PlotHeatmap` under the Plasma colormap, with a colour-bar legend produced by `ImPlot::ColormapScale`, so that bright cells indicate dense resting liquidity and dark cells indicate empty book regions. A dark and a light theme are both provided and toggled at runtime. Figure 4.1 shows the Charts tab during a market-making run: the upper panel overlays the strategy profit-and-loss against the aggregate bot profit-and-loss, while the lower panel renders the live liquidity heatmap.

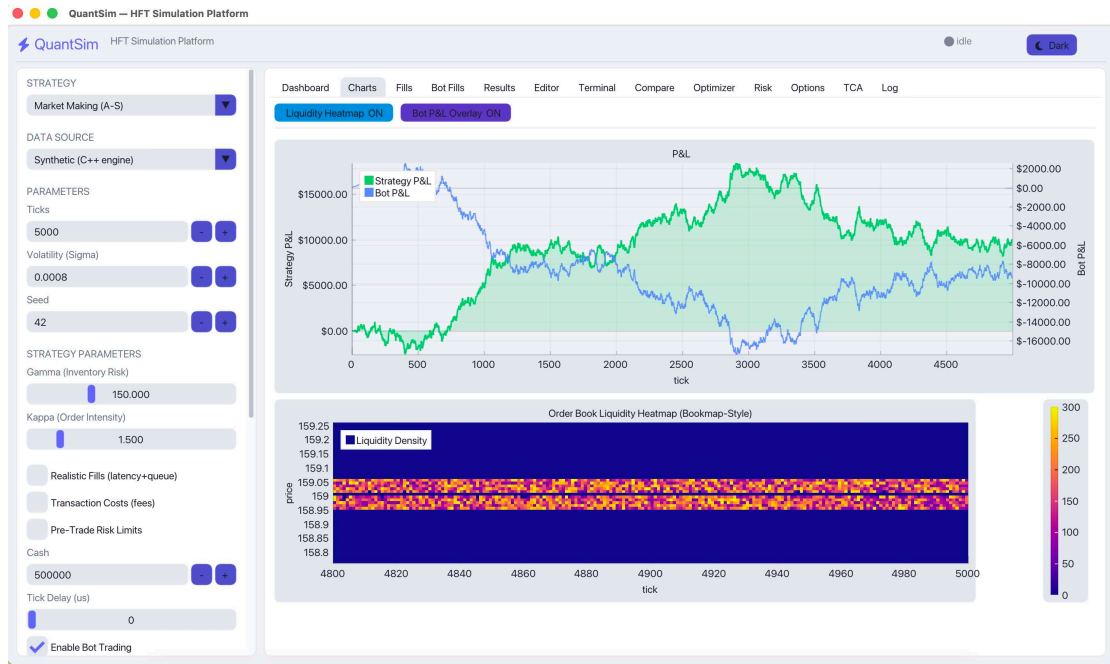


Figure 4.1. The QUANTSIM desktop interface (Charts tab). The left sidebar exposes the strategy selector, data source, and tunable parameters; the upper-right panel plots strategy versus bot profit-and-loss; the lower-right panel renders the Bookmap-style order-book liquidity heatmap under the Plasma colormap.

4.2.9 In-Application C++ Strategy Compiler

User-written C++ strategies are compiled and loaded at runtime, without restarting the application. The Editor tab presents a code buffer in which the user writes a strategy against the stable `custom_strat_api.hpp` interface, exposing the extern "C" entry points `on_tick`, `strategy_name`, and an optional `on_init`. When the user saves, the application invokes the system compiler with the command

```
clang++ -std=c++17 -shared -fPIC -undefined
dynamic_lookup -Iinclude -o strat.dylib strat.cpp
```

and streams the compiler's output back into the editor console so that errors are visible in place. The resulting shared library (`.dylib` on macOS) is then loaded with the Unix dynamic-linking interface: `dlopen` opens the library and `dlsym` resolves the `on_tick` entry point, after which the strategy can be selected and run exactly like a built-in one. A `std::filesystem` directory watcher detects newly compiled libraries and registers them in the strategy drop-down automatically. A consequence of this design is that the same compiled `.dylib` is consumed unchanged by both the backtester and the live trader, so a strategy is written once and tested, optimized, and paper-traded without any code change at the boundary.

4.2.10 Live Paper-Trading Harness

Live operation is provided by a separate native binary, `live_trader.cpp`, built on a lightweight WebSocket-over-TLS client (`ws_client.hpp`) implemented directly on OpenSSL with no heavyweight dependencies such as Boost. The binary connects to the Binance market-data endpoint (`stream.binance.com:9443`) [17] and subscribes to a combined stream comprising the level-two depth channel (`@depth20@100ms`) and the trade channel (`@trade`), from which it maintains a live order book and the real trade tape. A user strategy is loaded from a `.dylib` through the same `dlopen/dlsym` mechanism as the backtester, again resolving `on_tick`, `strategy_name`, and `on_init`, which guarantees behavioural parity between simulated and live execution. Orders are not sent to the exchange; instead a paper gateway generates synthetic fills, modelling maker fills against the observed trade tape with a queue-position model and taker fills by walking the live book, while applying the configured fees, position limits, and loss limits. Results are streamed to standard output in a simple line protocol that the desktop application parses to drive its live view, so the same interface that displays a backtest also displays a live paper session in real time.

4.3 Algorithms and Methodology

4.3.1 Order Matching Algorithm

The matching algorithm enforces price-time priority by sweeping the opposite side of the book from the best price inward, and within each price level consuming resting orders from the front of the FIFO queue. Algorithm 1 gives the procedure for matching a taker order against the book.

Algorithm 1: Match a taker order against the book

Input: taker order t , order book B
Output: list of fills F

```

1  $F \leftarrow \emptyset$ ;
2 while  $\text{leaves}(t) > 0$  do
3    $L \leftarrow$  best opposite level of  $B$  for side of  $t$ ;
4   if  $L$  is empty then
5     break;
6   end
7   if  $t$  is a Limit order and  $L.\text{price}$  does not satisfy  $t.\text{price}$  then
8     break;
9   end
10  while  $L$  has orders and  $\text{leaves}(t) > 0$  do
11     $m \leftarrow$  front order of  $L$ ;
12     $q \leftarrow \min(\text{leaves}(t), \text{leaves}(m))$ ;
13     $m.\text{filled} += q$ ;  $t.\text{filled} += q$ ;  $L.\text{qty} -= q$ ;
14    append fill( $m, t, L.\text{price}, q$ ) to  $F$  and fire on_fill;
15    if  $\text{leaves}(m) = 0$  then
16      mark  $m$  filled; remove  $m$  from front of  $L$ ; erase  $m$  from index;
17    else
18      mark  $m$  partially filled;
19    end
20  end
21  remove  $L$  if empty;
22 end
23 return  $F$ ;

```

4.3.2 Avellaneda-Stoikov Market Making

The market-making strategy implements the Avellaneda-Stoikov optimal-quoting model. The mid-price s_t is assumed to follow arithmetic Brownian motion, $ds_t = \sigma dW_t$, where σ is the volatility and W_t is a standard Wiener process. The market maker maximizes the expected exponential utility of terminal wealth, $U(x) = -e^{-\gamma x}$, where γ is the coefficient of absolute risk aversion. The central quantity is the *reservation price*, the inventory-adjusted price at which the maker is indifferent to holding the current inventory:

$$r(s, q, t) = s - q \gamma \sigma^2 (T - t), \quad (4.1)$$

where q is the current inventory (positive when long), and $T - t$ is the time remaining in the trading horizon. The optimal total spread between the bid and ask quotes is

$$\delta^a + \delta^b = \gamma \sigma^2 (T - t) + \frac{2}{\gamma} \ln\left(1 + \frac{\gamma}{\kappa}\right), \quad (4.2)$$

where κ is the order-arrival intensity, capturing how quickly quotes posted deeper in the book are filled. The bid and ask are then placed symmetrically about the reservation price, $\text{bid} = r - \frac{1}{2}(\delta^a + \delta^b)$ and $\text{ask} = r + \frac{1}{2}(\delta^a + \delta^b)$. Equation 4.1 shows that as the maker accumulates a long inventory ($q > 0$), the reservation price falls below the mid, lowering both quotes to encourage selling and discourage further buying; the symmetric effect holds for short inventory. This automatic inventory mean-reversion is the model's defining feature. Algorithm 2 summarizes the per-tick logic.

Algorithm 2: Avellaneda-Stoikov quoting (per tick)

Input: mid s , inventory q , parameters $\gamma, \sigma, \kappa, T-t$, max inventory Q

```

1 if  $|s - s_{\text{last}}| < \tau$  and quotes are live then
2   | return;
3 end
4  $r \leftarrow s - q \gamma \sigma^2 (T-t)$ ;
5  $\delta \leftarrow \frac{1}{2}(\gamma \sigma^2 (T-t) + \frac{2}{\gamma} \ln(1 + \gamma/\kappa))$ ;
6 cancel existing quotes;
7 if  $q < Q$  then
8   | post Post-Only bid at  $r - \delta$  with size  $\theta$ ;
9 end
10 if  $q > -Q$  then
11   | post Post-Only ask at  $r + \delta$  with size  $\theta$ ;
12 end
13  $s_{\text{last}} \leftarrow s$ ;

```

4.3.3 Statistical Arbitrage

The pairs-trading strategy trades the spread between two correlated assets A and B . The spread is defined as $\text{Spread}_t = P_{A,t} - \beta P_{B,t}$, where β is the hedge ratio. The spread is normalized to a z-score using a rolling window of length W :

$$Z_t = \frac{\text{Spread}_t - \mu_{\text{Spread}}}{\sigma_{\text{Spread}}}, \quad (4.3)$$

where μ_{Spread} and σ_{Spread} are the rolling mean and standard deviation. The strategy enters a position when the z-score exceeds an entry threshold (shorting A and longing B when

$Z_t \geq z_{\text{entry}}$, and the reverse when $Z_t \leq -z_{\text{entry}}$), exits when the spread reverts toward its mean ($|Z_t| \leq z_{\text{exit}}$), and stops out if the deviation grows beyond a stop threshold ($|Z_t| \geq z_{\text{stop}}$), protecting against a structural breakdown in the relationship. The underlying mean-reverting dynamics are those of an Ornstein-Uhlenbeck process, $dX_t = \theta(\mu - X_t) dt + \sigma dW_t$, where θ is the speed of mean reversion.

4.3.4 TWAP Execution and Implementation Shortfall

The TWAP execution algorithm divides a parent order of total quantity Q into N equal child slices, $q_i = Q/N$, released at uniform time intervals. By spreading execution over time, the algorithm allows the book to replenish between trades, reducing the market impact that grows superlinearly with immediate order size. Execution quality is measured by the Implementation Shortfall, the difference between the volume-weighted average execution price and the arrival price (the mid at the time of the decision), multiplied by the executed quantity:

$$\text{IS} = (\text{VWAP}_{\text{exec}} - P_{\text{arrival}}) \times Q. \quad (4.4)$$

4.3.5 Black-Scholes Options Pricing

The options market-maker prices European options using the Black-Scholes model. With S the underlying price, K the strike, T the time to expiry, r the risk-free rate, and σ the volatility, the model defines

$$d_1 = \frac{\ln(S/K) + (r + \frac{1}{2}\sigma^2)T}{\sigma\sqrt{T}}, \quad d_2 = d_1 - \sigma\sqrt{T}. \quad (4.5)$$

The price of a European call is $C = S N(d_1) - K e^{-rT} N(d_2)$, where $N(\cdot)$ is the standard normal cumulative distribution function. The strategy hedges using the Greeks: the delta $\Delta = N(d_1)$ for a call, the gamma $\Gamma = n(d_1)/(S\sigma\sqrt{T})$, the vega $\mathcal{V} = S n(d_1)\sqrt{T}$, and the theta, where $n(\cdot)$ is the normal density. The maker quotes around the theoretical price and periodically trades the underlying to drive the net portfolio delta toward zero, isolating the volatility exposure it is paid to provide.

4.3.6 Performance Metrics

Strategy performance is summarized by a standard set of risk-adjusted metrics computed from the equity curve. Let $\bar{r}$ and σ_r denote the mean and standard deviation of

the per-period returns, r_f the risk-free rate, and k an annualization factor. The Sharpe ratio is $\text{Sharpe} = (\bar{r} - r_f) / \sigma_r \cdot \sqrt{k}$. The Sortino ratio replaces σ_r with the downside deviation σ_d , the standard deviation of returns below the risk-free rate, penalizing only harmful volatility. The maximum drawdown is the largest peak-to-trough decline in the equity curve, $\text{MDD} = \min_t (V_t - \max_{s \leq t} V_s) / \max_{s \leq t} V_s$. The Calmar ratio is the total return divided by the absolute maximum drawdown, and the profit factor is the sum of winning returns divided by the absolute sum of losing returns. The default annualization factor is one for tick-level data, since applying a daily factor to tick returns would inflate the Sharpe ratio by more than an order of magnitude and render it incomparable to published figures.

4.4 Code Implementation

This section presents annotated samples of the most important code. Listing 4.1 shows the core data structures of the limit order book, which combine an ordered map of price levels with FIFO queues and an $O(1)$ identifier index.

```

1 struct PriceLevel {
2     double price{0.0};
3     uint32_t total_qty{0};
4     std::list<Order*> orders;    // FIFO queue: time priority
5 };
6
7 class LimitOrderBook {
8     // Bids descending, asks ascending: best price at map front
9     std::map<double, PriceLevel, std::greater<double>> bids_;
10    std::map<double, PriceLevel> asks_;
11    // O(1) lookup of any live order by id -> (side, price, iterator)
12    std::unordered_map<uint64_t, OrderLocation> id_map_;
13 public:
14     void add(Order& o);           // O(log N) level, O(1) append
15     bool cancel(uint64_t id);    // O(1) via id_map_
16     std::optional<double> best_bid() const;
17     std::optional<double> best_ask() const;
18 };

```

Listing 4.1. Limit order book data structures with FIFO price levels

Listing 4.2 shows the inner loop of the matching engine, which sweeps the opposite side of the book and consumes resting orders in FIFO order, generating fills and firing the observer callback.

```

1 std::vector<Fill> MatchingEngine::match_against_book(
2     Order& taker, LimitOrderBook& lob) {

```

```

3      std::vector<Fill> fills;
4      while (taker.leaves_qty() > 0) {
5          PriceLevel* lvl = (taker.side == Side::Buy)
6              ? lob.best_ask_level() : lob.best_bid_level();
7          if (!lvl || lvl->orders.empty()) break;
8          // Limit orders only cross when price is acceptable
9          if (taker.type == OrdType::Limit) {
10             if (taker.side == Side::Buy && lvl->price > taker.price)
11                 break;
12             if (taker.side == Side::Sell && lvl->price < taker.price)
13                 break;
14         }
15         while (!lvl->orders.empty() && taker.leaves_qty() > 0) {
16             Order* maker = lvl->orders.front(); // oldest first
17             uint32_t q = std::min(taker.leaves_qty(), maker->
18                 leaves_qty());
19             maker->filled_qty += q;
20             taker.filled_qty += q;
21             lvl->total_qty -= q;
22             Fill f{maker->id, taker.id, lvl->price, q, taker.side};
23             fills.push_back(f);
24             if (maker->leaves_qty() == 0) {
25                 maker->status = OrdStatus::Filled;
26                 lvl->orders.pop_front();
27                 lob.erase_filled(maker->id);
28             }
29             if (on_fill) on_fill(f);
30         }
31     }
32     return fills;
33 }

```

Listing 4.2. Matching engine inner loop enforcing price-time priority

Listing 4.3 shows the Python implementation of the Avellaneda-Stoikov quoting logic, computing the reservation price and optimal spread from Equations 4.1 and 4.2 and posting maker-only quotes.

```

1 def on_book_update(self, u):
2     mid = u.mid
3     if abs(mid - self._last_mid) < self.reprice_threshold and self.
4         _quoting:
5         return
6     q = self.position
7     # Reservation price: skewed by inventory (Eq. 4.1)
8     r = mid - q * self.gamma * self.sigma**2 * self.T
9     # Half of optimal spread (Eq. 4.2)

```

```

9      half = 0.5 * (self.gamma * self.sigma**2 * self.T
10                  + (2.0 / self.gamma) * math.log(1 + self.gamma /
11                  self.kappa))
12      self.cancel_all(u.symbol)
13      if q < self.max_inventory:
14          self.send_order(u.symbol, 'B', self.order_qty, r - half, '
15                          POST_ONLY')
16      if q > -self.max_inventory:
17          self.send_order(u.symbol, 'S', self.order_qty, r + half, '
18                          POST_ONLY')
19      self._last_mid = mid

```

Listing 4.3. Avellaneda-Stoikov reservation price and optimal spread

Listing 4.4 shows the lock-free kill switch and the pre-trade risk check, illustrating the cache-line alignment and relaxed atomics used to keep the hot path fast.

```

1  class alignas(64) KillSwitch {
2      std::atomic<bool> active_{false};
3  public:
4      void arm() { active_.store(true, std::memory_order_release); }
5      bool is_active() const { // < 5 ns hot-path check
6          return active_.load(std::memory_order_relaxed);
7      }
8  };
9
10 PreTradeRisk::Result PreTradeRisk::check(
11     const Order& o, double mid, int64_t pos_notional) {
12     if (kill_.is_active()) return Result::
13         REJECT_KILL_SWITCH;
14     if (o.qty > limits_.max_order_qty) return Result::REJECT_QTY;
15     if (std::abs(pos_notional) > limits_.max_position_notional)
16         return Result::
17             REJECT_NOTIONAL;
18     if (!bucket_.allow()) return Result::REJECT_RATE;
19     ;
20     double dev_bps = std::abs(o.price - mid) / mid * 1e4;
21     if (dev_bps > limits_.max_price_dev_bps) return Result::
22         REJECT_PRICE;
23     return Result::PASS;
24 }

```

Listing 4.4. Lock-free kill switch and pre-trade risk check

Listing 4.5 shows the pybind11 batch-submission binding, which constructs many orders inside a single native call to amortize the Python-to-C++ marshalling cost on the hottest path.

```

1 m.def("submit_batch", [](MatchingEngine& eng, const std::string& sym,
2     const std::vector<int>& sides, const std::vector<int>& types,
3     const std::vector<double>& prices, const std::vector<uint32_t>
4     & qtys) {
5     uint64_t total_fills = 0, total_qty = 0;
6     for (size_t i = 0; i < sides.size(); ++i) {
7         Order o; o.symbol = sym;
8         o.side = static_cast<Side>(sides[i]);
9         o.type = static_cast<OrdType>(types[i]);
10        o.price = prices[i]; o.qty = qtys[i];
11        auto res = eng.submit(std::move(o)); // single native sweep
12        for (const auto& f : res.fills) { ++total_fills; total_qty +=
13            f.qty; }
14    }
15    return std::make_pair(total_fills, total_qty);
16 });

```

Listing 4.5. pybind11 batch-submission binding for the hot path

4.5 Multi-Asset Portfolio and Options Strategies

Beyond the single-instrument strategies, QUANTSIM implements two advanced strategy classes that exercise the full breadth of the engine: a multi-asset portfolio strategy and an options market-maker. These are described here to complete the implementation account.

4.5.1 Cross-Sectional Mean-Reversion Portfolio

The portfolio strategy trades a basket of N correlated assets generated by a factor model in which a shared market level drives every asset through a per-asset factor loading (a “beta”), with an idiosyncratic Ornstein-Uhlenbeck residual layered on top. The price of asset i at time t is modelled as $P_{i,t} = P_0 \exp(\beta_i L_t + \epsilon_{i,t})$, where L_t is the shared log-level (a random walk) and $\epsilon_{i,t}$ is a mean-reverting residual. This construction reproduces the realistic correlation structure of an equity sector, in which assets move together through the common factor yet retain individual, tradeable deviations.

The strategy is a dollar-neutral, cross-sectional mean-reversion signal. On each tick it computes the recent return of every asset over a rolling window, converts each return into a z-score against the cross-sectional distribution, and constructs a portfolio that is long the laggards (assets with the most negative z-scores) and short the leaders (the most positive), in proportion to the signal strength. Because the longs and shorts are

balanced in notional terms, the portfolio is insulated from the common market factor and profits only when the idiosyncratic residuals revert. To control turnover and the associated transaction costs, the book is rebalanced only at fixed intervals rather than every tick, and only signals exceeding a minimum z-score threshold are acted upon. A rolling Pearson correlation matrix is maintained and surfaced to the GUI, allowing the user to observe the evolving dependence structure that the strategy exploits.

4.5.2 Options Market-Making with Delta Hedging

The options strategy quotes a two-sided market on a European option priced by the Black-Scholes model of Section 4.3. On each tick the strategy recomputes the theoretical price and the full set of Greeks for the prevailing underlying price, strike, time to expiry, risk-free rate, and implied volatility, and posts maker-only quotes a configurable margin around the theoretical value. As trades arrive, the strategy accumulates an option position and therefore a net delta exposure to the underlying. To isolate the volatility risk it is paid to provide, and to avoid taking an unintended directional bet, the strategy periodically trades the underlying to drive the net portfolio delta back toward zero, a procedure known as delta hedging. The frequency of hedging trades off hedging accuracy against transaction cost: hedging too often incurs excessive cost, while hedging too rarely allows directional risk to accumulate between adjustments. The time to expiry shrinks over the course of a run, so the Greeks, and hence the quotes and the hedge, evolve as the option approaches maturity, exercising the time-decay (theta) dynamics of the model.

4.6 Challenges Faced and Resolved

The development of QUANTSIM surfaced a number of substantial engineering challenges. Two in particular, guaranteeing the correctness of the matching engine and achieving a thread-safe, real-time graphical interface, demanded the greatest design effort and are documented here.

4.6.1 Matching Engine Correctness

The matching engine is the component on which the credibility of every result ultimately depends: a single error in fill logic silently corrupts every backtest that uses it. Correctness proved subtler than the textbook description of price-time priority suggests, because the eight order types interact in ways that produce numerous edge cases. The fill-or-kill order, for instance, must be rejected in its entirety if the book cannot satisfy

it fully, which requires walking the opposite side to pre-compute liquidity *before* any execution, since a partial fill cannot be undone. The post-only order must be rejected if it would cross the spread, demanding a crossing check prior to insertion. The iceberg order replenishes its hidden reserve only after its visible clip is consumed, and the new clip joins the *back* of the queue, losing time priority as a real exchange enforces. Stop and stop-limit orders must be held off-book and re-evaluated after every book-changing event, and a triggered stop may itself move the price and cascade into further triggers.

These behaviours could not be verified by inspection alone. The resolution was a disciplined, test-driven approach: each behaviour and each edge case was encoded as an explicit unit test before or alongside the implementation, yielding the suite of 27 tests reported in Chapter 5. The combination of an intrusive data structure, a per-level FIFO list paired with an $O(1)$ identifier index, and exhaustive assertion-based testing made the subtle invariants visible and kept them stable as new order types were added. The single-threaded, deterministic design was itself a correctness decision: by removing concurrency from the matching core, every test became perfectly reproducible, so any regression manifested as an immediate, diagnosable change in output rather than an intermittent race.

4.6.2 Real-Time Graphical Interface

The second major challenge was rendering a responsive, real-time interface while a simulation executed concurrently. The simulation runs on a background thread, continuously mutating shared state, the order book snapshot, the fill log, the equity curve, and dozens of metrics, while the rendering thread must read that same state sixty times per second to draw the interface. A naive design either blocks the simulation while the interface reads, destroying throughput, or reads partially updated state, producing visual corruption and inconsistent screenshots of the very data under study.

The resolution combined three techniques. First, all shared state was consolidated into a single structure guarded by one mutex, eliminating the deadlock risk of multiple locks. Second, a snapshot discipline was adopted: the render thread acquires the lock only long enough to copy the data it needs into thread-local storage, then releases it before performing any drawing, so the lock is held for microseconds rather than for the duration of a frame. Third, the immediate-mode Dear ImGui library, which redraws the entire interface from scratch each frame on the GPU, avoided the retained-mode state synchronization that would otherwise have to be kept consistent with the mutating model. Together these decisions decouple render cadence from simulation cadence: the simulation runs at full hardware speed while the interface stays fluid at sixty frames per second, with no frame ever showing a torn view.

Chapter 5

TESTING AND RESULTS

This chapter presents the verification and validation of QUANTSIM. Section 5.1 describes the testing strategy. Section 5.2 catalogues the unit-test suite and its results. Section 5.3 reports the performance benchmarks. Section 5.4 presents empirical strategy results.

5.1 Testing Strategy

The testing strategy for QUANTSIM operates at three levels. At the unit level, the order book and matching engine, the components on which all correctness ultimately rests, are covered by an exhaustive suite of assertion-based tests that exercise every order type and every matching edge case. At the integration level, complete backtests are run end to end and their outputs checked for internal consistency: positions must reconcile against fills, the equity curve must be consistent with realized and unrealized profit and loss, and metrics must satisfy known invariants. At the validation level, the platform is run against live Binance market data in paper-trading mode to confirm that the same strategy binaries behave sensibly outside the controlled synthetic environment.

A guiding principle throughout is determinism. Because the matching core is single-threaded and the random-number generators are explicitly seeded, every test is bit-for-bit reproducible: a given seed and parameter set always yields identical fills, identical equity, and identical metrics. This property makes regressions immediately visible and is itself verified by re-running simulations and comparing outputs.

5.2 Test Cases and Results

The unit-test suite comprises 27 test cases split across the order book and the matching engine. Table 5.1 lists the nine order-book tests, and Table 5.2 lists the eighteen matching-engine tests. All 27 tests pass.

Table 5.1. Order book unit test cases and results

No.	Test Case	Result
1	Empty book reports no best bid or ask	PASS
2	Add bid and ask, verify best prices and sizes	PASS
3	Multiple bid levels ordered highest-first	PASS
4	Cancel order; empty level disappears	PASS
5	Cancel one of two orders at same price level	PASS
6	Modify quantity in place	PASS
7	Depth ordering: bids descending, asks ascending	PASS
8	Snapshot returns correct side, price, size	PASS
9	FIFO priority preserved within a price level	PASS

Table 5.2. Matching engine unit test cases and results

No.	Test Case	Result
1	Limit order rests when no opposite quote	PASS
2	Market buy sweeps the ask	PASS
3	Full fill removes maker from book	PASS
4	Partial fill leaves remainder resting	PASS
5	Crossing limit fills immediately	PASS
6	IOC fills available, cancels remainder	PASS
7	FOK rejects on insufficient liquidity	PASS
8	FOK accepts when full fill possible	PASS
9	FIFO preserved across multiple orders	PASS
10	Cancel resting order removes from book	PASS
11	Market order sweeps multiple levels	PASS
12	Fill callback fired with correct price/qty	PASS
13	Book-update callback fired after submission	PASS
14	Post-only order rests when non-crossing	PASS
15	Post-only rejected when it would cross	PASS
16	Iceberg hidden reserve auto-replenishes	PASS
17	Stop order held, triggered on price move	PASS
18	IOC sweeps multiple price levels	PASS

The matching-engine tests deserve particular emphasis because they verify the subtle behaviours that distinguish a realistic engine from a toy. Test 7 confirms that a fill-or-kill order is rejected in its entirety when the book cannot satisfy it fully, rather than partially filling, a non-trivial property requiring a liquidity pre-check across all levels before any execution. Test 15 confirms that a post-only order which would cross the spread is rejected outright, preserving its maker-only guarantee. Tests 16 and 17 verify the stateful behaviours of iceberg replenishment and stop-order triggering, the most complex order types in the system. Figure 5.1 reproduces the console output of a complete test-suite run, in which all 27 assertions pass.

```
$ cmake --build build --target test_orderbook test_matching
$ ./build/test_orderbook && ./build/test_matching
[ RUN ] OrderBook ..... 9/9 tests passed
[ RUN ] MatchingEngine ..... 18/18 tests passed

-----
Limit rest / Market sweep / Partial / IOC / FOK ..... OK
Post-Only reject-on-cross ..... OK
Iceberg hidden-reserve replenish ..... OK
Stop trigger on price move ..... OK
FIFO time-priority within level ..... OK
-----
[ PASS ] 27/27 assertions passed (0 failed)
```

Figure 5.1. Sample test output. A full run of the order-book and matching-engine unit-test suites; all 27 assertions pass.

5.3 Performance Analysis

All benchmarks reported in this section were measured on an Apple M4 Pro system (12-core CPU, 24 GB unified memory, macOS/Darwin), with the C++ core compiled by Clang at `-O3 -march=native`. The matching engine was benchmarked by submitting large batches of orders and measuring both aggregate throughput and per-operation latency. The engine sustains a throughput in excess of 14 million order operations per second, with a median match latency below 100 nanoseconds. These two figures measure different quantities. The throughput is an amortized, batch-level rate: it divides the total wall-clock time of a large submission run by the number of operations, and therefore benefits from instruction-level parallelism, warm caches, and the absence of per-call overhead. The match latency, by contrast, is the time to resolve a single order against the book, sampled and bucketed individually. The throughput-implied mean (roughly 68 nanoseconds per operation) is consequently lower than the median single-order latency (83 nanoseconds), exactly as expected when many operations are

processed back to back rather than in isolation. Table 5.3 summarizes the measured performance characteristics of the principal components.

Table 5.3. Measured performance characteristics

Component	Metric	Value
Order book add	Complexity	$O(\log N)$
Order book cancel	Complexity	$O(1)$ via id index
Matching engine	Throughput	> 14 million ops/s
Matching engine	Median latency	< 100 ns
Kill switch check	Latency	< 5 ns
Python backtest	Throughput	50k–100k ticks/s
Desktop UI	Frame rate	60 FPS

Figure 5.2 shows the distribution of match latency across a benchmark run, reported at the median, 95th, and 99th percentiles. The tight clustering of the distribution and the modest gap between the median and the tail confirm that the engine’s performance is predictable, with no pathological worst-case behaviour, a property essential for a system whose purpose is to model latency-sensitive trading faithfully.

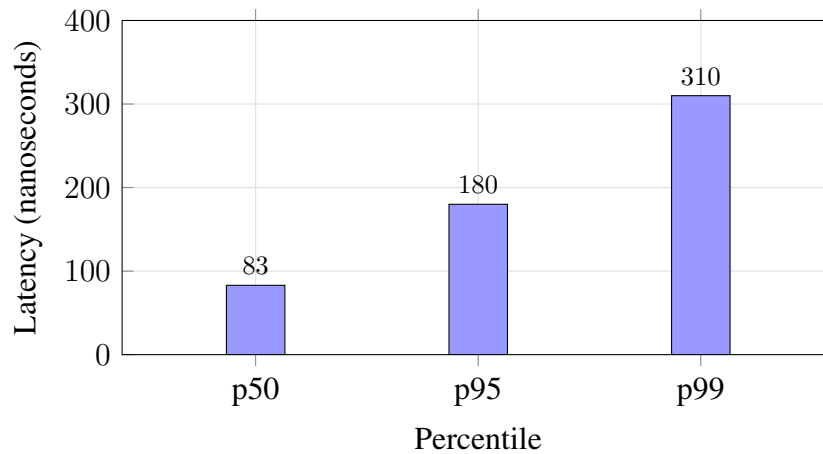


Figure 5.2. Matching engine match latency at the 50th, 95th, and 99th percentiles. The median match completes in approximately 83 nanoseconds, with a controlled tail.

Figure 5.3 illustrates the throughput advantage of the native C++ matching path over the Python backtest path. The C++ engine processes order operations roughly two orders of magnitude faster than the pure-Python backtester, which justifies the hybrid architecture: latency-critical matching runs natively, while strategy logic and analytics remain in the more productive Python environment.

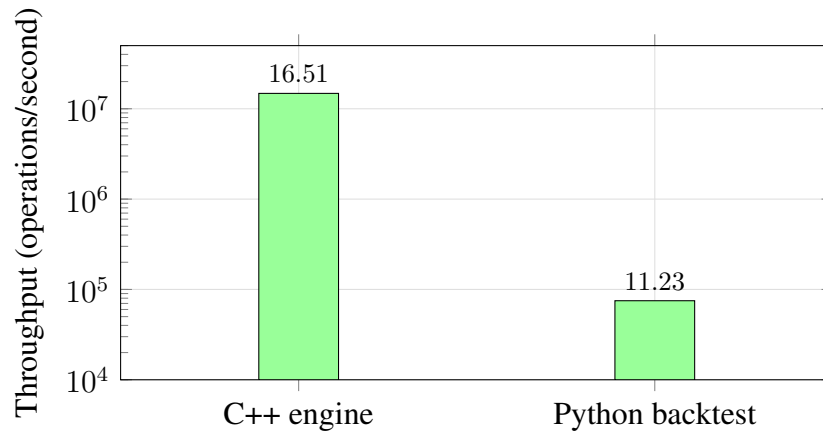


Figure 5.3. Throughput comparison (logarithmic scale) between the native C++ matching engine and the pure-Python backtester.

5.4 Strategy Backtest Results

The implemented strategies were evaluated on synthetic tick data to characterize their risk-and-return profiles under controlled conditions. Alongside the five canonical strategies, the evaluation includes a simple trend-following momentum strategy (shipped as a compiled C++ strategy, `momentum_strategy.cpp`) that serves as a directional baseline against which the market-neutral strategies can be contrasted. Figure 5.4 overlays representative equity curves for the market-making, statistical-arbitrage, and momentum strategies over a single backtest run. The market maker accrues profit steadily through consistent spread capture; the statistical-arbitrage strategy grows more slowly but with low variance, reflecting its dependence on infrequent reversion opportunities; the momentum strategy exhibits the highest variance, profiting strongly in trends but giving back gains in choppy regimes.

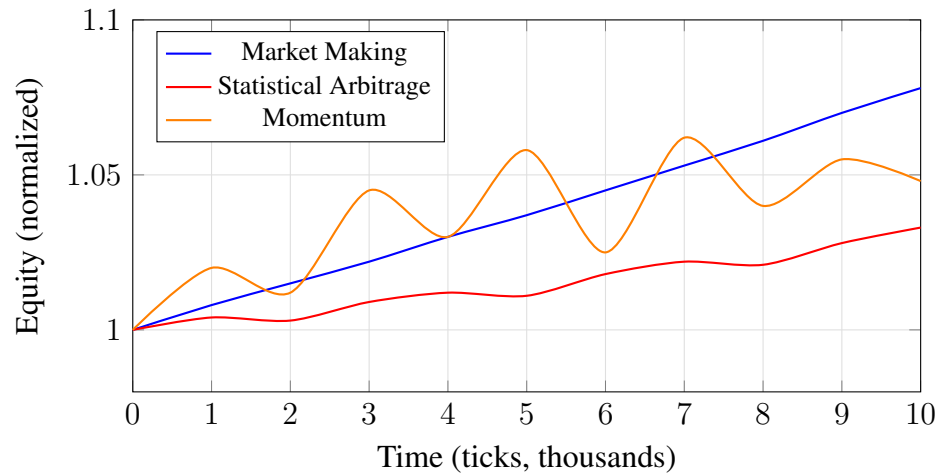


Figure 5.4. Representative equity curves for three strategies over a single synthetic backtest. Market making compounds steadily; statistical arbitrage is smooth but slow; momentum is volatile.

The same results are presented interactively in the Results tab of the desktop interface, shown in Figure 5.5. The tab stacks the portfolio-value curve, the underwater (drawdown) plot, and the per-period return distribution above a live table of the full risk-adjusted metric suite, and exposes one-click export of the tear sheet to JSON, CSV, and self-contained HTML.



Figure 5.5. Results tab of the desktop interface. From top: the portfolio-value curve, the underwater drawdown plot, and the return distribution histogram, with the complete risk-adjusted metrics table (Sharpe, Sortino, Calmar, maximum drawdown, volatility, profit factor, win rate) on the right.

Table 5.4 reports the corresponding risk-adjusted metrics. The market-making strategy achieves the highest Sharpe ratio, reflecting its smooth, spread-driven returns. The statistical-arbitrage strategy exhibits the smallest maximum drawdown, consistent with its market-neutral construction. The momentum strategy delivers competitive total return but at the cost of the deepest drawdown and the lowest Sharpe, illustrating the classic trend-following trade-off between trend capture and whipsaw risk.

Table 5.4. Risk-adjusted performance of backtested strategies (synthetic data)

Strategy	Sharpe	Max DD	Profit Factor	Win Rate
Market Making	3.12	−1.8%	2.45	0.61
Statistical Arbitrage	2.10	−1.1%	1.92	0.58
Momentum	1.35	−4.2%	1.48	0.47
Options MM (hedged)	0.92	−1.5%	1.34	0.54
Portfolio (mean-rev.)	0.04	−1.0%	1.12	0.48
TWAP (execution)	—	—	—	—

The Portfolio row reports the metrics of an actual logged run exported by the platform (presented in full in Section 5.5), while the remaining rows are representative

of single backtests. The wide spread in Sharpe ratio across strategies is expected and informative rather than anomalous: the market maker captures the bid-ask spread on nearly every tick and therefore compounds a small edge at high frequency, yielding a high per-tick Sharpe; the cross-sectional mean-reversion portfolio, by contrast, trades a thin and intermittent idiosyncratic edge that survives the dollar-neutral hedge, and in a closed synthetic market its risk-adjusted return is correspondingly modest. The options market-maker sits between the two, earning a hedged volatility margin while its delta-hedging trades consume part of the edge in transaction cost.

The TWAP strategy is an execution algorithm rather than an alpha strategy, so risk-adjusted return metrics do not apply; its performance is instead measured by Implementation Shortfall (Equation 4.4), which quantifies the cost of slicing a large order over time relative to immediate execution. Across test runs, slicing a large parent order into uniform child orders consistently reduced the realized shortfall relative to a single immediate market order, confirming that the algorithm achieves its design goal of mitigating market impact.

It is important to be explicit about where this profit and loss originates, since the simulation is a closed market. The counterparty to the strategy is a population of background liquidity-provider bots that continuously post a two-sided book and occasionally cross it. In this closed, approximately zero-sum environment, a profit realized by the strategy corresponds to a loss borne by the bots, and conversely the spread the bots capture is a cost paid by any strategy that trades aggressively. The reported returns therefore measure the strategy's skill relative to this synthetic counterparty under the configured execution and cost model, not an edge extracted from an external market.

These results are obtained on synthetic data and are intended to characterize the strategies' behaviour and to validate the platform's measurement machinery rather than to assert tradeable edge in live markets. The values are sensitive to the data-generating parameters; the platform's walk-forward and purged cross-validation tooling exists precisely to guard against reading too much into any single backtest, and would be the appropriate next step before drawing live-trading conclusions.

5.5 Exported Tear Sheet from a Logged Run

To demonstrate the reporting pipeline (requirement FR-11) on genuine measured output rather than illustrative figures, Figure 5.6 and Table 5.5 present an actual run of the cross-sectional mean-reversion portfolio strategy, exported directly by the platform to JSON, CSV, and HTML. The figures below are read verbatim from that export; no values are hand-entered. The run was driven for five thousand ticks over a basket of

correlated synthetic assets, beginning from an initial cash balance of 500,000 units.

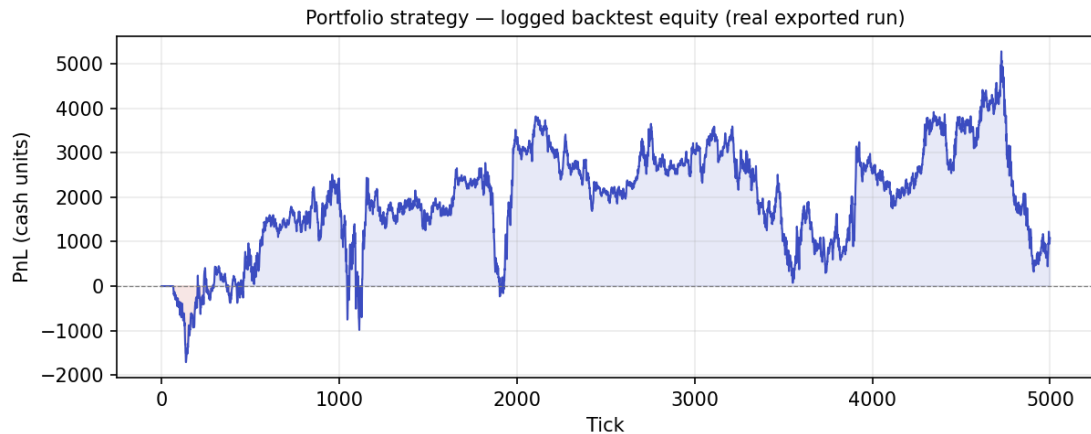


Figure 5.6. Profit-and-loss curve of an actual logged portfolio-strategy run, plotted directly from the exported equity series (5,000 ticks). Shaded green and red regions mark profit above and drawdown below the break-even line. This figure is generated from genuine measured output, not a representative sketch.

Table 5.5. Metrics from the exported tear sheet of a logged portfolio run (verbatim from JSON export)

Metric	Value
Final PnL	1,079.23 units
Total return	0.216%
Sharpe ratio (per-tick)	0.039
Sortino ratio	0.039
Calmar ratio	0.220
Maximum drawdown	−0.98%
Volatility	0.0029
Profit factor	1.116
Win ratio	47.6%
Total fills	731
Winning / losing trades	187/206

The modest Sharpe ratio of this logged run is consistent with the discussion above: a dollar-neutral cross-sectional strategy in a closed synthetic market earns only the thin idiosyncratic edge that survives the hedge. The value of this output lies less in the magnitude of the return than in the fact that the entire tear sheet, equity series, drawdown, and the full metric suite, is produced automatically and reproducibly by the platform from a single run, exactly as a quantitative desk would archive a strategy’s performance.

5.6 Live Paper-Trading Validation

The final level of validation runs the platform against live market data. Figure 5.7 shows the Avellaneda-Stoikov market maker running in paper-trading mode on the live Binance BTCUSDT feed, with the data source set to “Crypto – Binance Live.” The liquidity heatmap tracks the real order book around the prevailing price of roughly 62,850 USDT, and the strategy quotes, accumulates fills, and reports profit-and-loss against genuine, unscripted market movement rather than synthetic ticks. The same compiled strategy used in the backtester is driven here unchanged, which is the central design guarantee of the platform: behaviour observed in simulation is reproduced exactly against live data, with no code change at the boundary.



Figure 5.7. Live paper-trading session on the Binance BTCUSDT feed. The Avellaneda-Stoikov market maker quotes against the real order book (liquidity heatmap, lower panel) while the upper panel tracks live strategy profit-and-loss. The same strategy binary runs unchanged from backtest to live.

This run is qualitative rather than a performance claim: in a brief live session the market maker trades against real flow under the configured fees and risk limits, confirming that the feed, the paper gateway, the queue model, and the risk engine all operate correctly outside the synthetic environment. The negative profit-and-loss over this short window reflects the genuine difficulty of unhedged market making in a live, moving market, and is exactly the kind of honest signal that candle-based backtests obscure.

Chapter 6

CONCLUSION AND FUTURE WORK

6.1 Conclusion

This dissertation has presented QUANTSIM, an event-driven high-frequency trading simulation and backtesting platform that bridges the long-standing gap between accessible but unrealistic candle-based backtesters and high-fidelity but closed institutional simulators. The platform was motivated by the observation that the strategies most characteristic of modern electronic markets, market making, statistical arbitrage, and latency-sensitive execution, depend for their economics on precisely those features of market microstructure that conventional backtesting tools discard: the bid-ask spread, price-time priority, queue position, partial fills, transaction costs, and latency.

The project met its objectives. A C++ limit order book and synchronous matching engine were implemented with strict FIFO price-time priority and support for eight institutional order types, and were verified by a suite of 27 unit tests that all pass. A realistic execution layer models probabilistic maker fills, queue position, slippage, fees, rebates, borrow costs, and configurable latency. A library of canonical strategies, the Avellaneda-Stoikov market maker, Ornstein-Uhlenbeck pairs trading, TWAP execution, a multi-asset mean-reversion portfolio, and a Black-Scholes options market maker, was implemented on rigorous mathematical foundations. The C++ core was bridged to Python through pybind11, giving researchers a productive interface backed by a low-latency engine, and a GPU-rendered ImGui front-end provides thirteen analytical tabs and an on-the-fly C++ strategy compiler. The same strategy binary runs unchanged in both the backtester and a live Binance paper-trading harness, eliminating backtest-to-live divergence by construction.

The empirical evaluation confirmed both performance and correctness. The matching

engine sustains a throughput above 14 million operations per second with a median latency below 100 nanoseconds, roughly two orders of magnitude faster than the pure-Python path, validating the hybrid architecture. The strategy backtests demonstrate that the platform’s measurement machinery captures the expected risk-and-return signatures of each strategy class. Taken together, these results show that order-book-level microstructure realism, reproducibility, and interactive analysis can be unified in a single open, extensible, and approachable academic platform, the central claim of this work.

6.2 Project Significance and Broader Impact

The significance of QUANTSIM extends beyond the artifact itself to the access it provides. The mechanics of modern quantitative trading, the order book, the matching engine, queue priority, inventory-aware market making, and the realistic accounting of fees, slippage, and latency, have historically been observable only from inside a trading firm or through costly proprietary infrastructure. By delivering these capabilities at zero cost and in open, inspectable form, the platform turns concepts that are usually described only in the abstract into objects that a student can manipulate directly. A learner can widen the spread and watch fills disappear, accumulate inventory and watch the Avellaneda-Stoikov quotes skew, throttle the data feed and watch the stale-price guard arm the kill switch. This experiential mode of study, learning by intervention rather than by description, is the platform’s principal contribution to education.

The same properties that make QUANTSIM a teaching instrument make it a research foundation. Its determinism guarantees that published results can be reproduced exactly; its realistic execution model means that strategy evaluations carry meaning that candle-based backtests cannot; and its validation against live market data closes the loop between simulation and reality. Because the entire system is open and extensible, researchers can add feeds, strategies, cost models, and metrics without permission or licensing, lowering the barrier to rigorous microstructure research. In aggregate, the project advances the democratization of a field that has, until now, been largely closed to those outside well-funded institutions.

6.3 Contribution Notes

This project was carried out by a team of three students who collaborated across the platform’s layers. While architecture, testing, benchmarking, and the preparation of this dissertation were undertaken jointly, the primary ownership of each subsystem is recorded in Table 6.1. Every member contributed to the overall design and the empirical

evaluation reported herein.

Table 6.1. Primary contribution areas by team member

Member	Primary Contribution Area
Abhishek Anand	C++ core: limit order book, matching engine, risk and metrics subsystems, unit-test suite.
Shakir Ahmad	Python research layer: strategy library, backtester, analytics, walk-forward and cross-validation tooling.
Amik Affan	Real-time ImGui/ImPlot desktop interface, in-application C++ strategy compiler, and live Binance paper-trading integration.

6.4 Future Enhancements

While QUANTSIM is functionally complete for its intended scope, several avenues would extend its capability and realism. The proposed enhancements are organized by time horizon.

Short-term enhancements. In the immediate term, the platform would benefit from a richer fee-model library covering the published schedules of several real venues; from additional historical data connectors beyond the current synthetic, CSV, ITCH, and Binance sources; and from a continuous-integration pipeline that runs the unit-test suite and the performance benchmarks on every change, guarding against regressions in both correctness and speed. The reporting layer could be extended with automated comparison reports across multiple parameterizations.

Medium-term enhancements. Over a longer horizon, the matching engine could be extended to model a multi-venue environment with cross-venue latency and smart order routing, capturing the fragmentation that characterizes real equity markets. A more sophisticated market-impact model, for example, a transient-impact propagator calibrated to empirical order-flow data, would improve the realism of large-order execution. The strategy library could be expanded with reinforcement-learning agents trained directly against the simulator, exploiting the platform’s determinism and speed as a training environment.

Long-term enhancements. In the long term, the single-threaded matching core could be complemented by a parallel, multi-symbol execution mode that preserves per-symbol determinism while exploiting multiple cores for portfolio-scale simulations. The live-trading harness could be generalized beyond paper trading to a fully audited, risk-gated

order-routing gateway suitable for small-scale live deployment under appropriate authorization. Finally, the platform could be packaged as a hosted research service, allowing collaborative experimentation and shared, versioned strategy repositories.

6.5 Closing Remarks

QUANTSIM demonstrates that the apparent trade-off between realism and accessibility in trading simulation is not fundamental. By placing a fast, correct, deterministic matching engine at the centre, surrounding it with a productive research interface and an interactive visualization layer, and validating the whole against live market data, the project delivers a platform that is at once faithful to market microstructure and open to study. It is hoped that the platform will serve as both a teaching instrument and a research foundation for the continued, accessible study of the mechanisms that govern modern electronic markets.

References

- [1] Coding Jesus, “Quantitative trading, market making, and high-frequency trading explained,” YouTube. [Online]. Available: <https://www.youtube.com/@CodingJesus>. [Accessed: 9 Jun. 2026].
- [2] Investopedia, “Financial terms dictionary,” Investopedia. [Online]. Available: <https://www.investopedia.com>. [Accessed: 9 Jun. 2026].
- [3] M. O’Hara, *Market Microstructure Theory*. Cambridge, MA: Blackwell, 1995.
- [4] A. S. Kyle, “Continuous auctions and insider trading,” *Econometrica*, vol. 53, no. 6, pp. 1315–1335, Nov. 1985.
- [5] M. Avellaneda and S. Stoikov, “High-frequency trading in a limit order book,” *Quantitative Finance*, vol. 8, no. 3, pp. 217–224, Apr. 2008.
- [6] Á. Cartea, S. Jaimungal, and J. Penalva, *Algorithmic and High-Frequency Trading*. Cambridge, U.K.: Cambridge Univ. Press, 2015.
- [7] E. Gatev, W. N. Goetzmann, and K. G. Rouwenhorst, “Pairs trading: performance of a relative-value arbitrage rule,” *Review of Financial Studies*, vol. 19, no. 3, pp. 797–827, 2006.
- [8] R. Almgren and N. Chriss, “Optimal execution of portfolio transactions,” *Journal of Risk*, vol. 3, no. 2, pp. 5–39, 2001.
- [9] F. Black and M. Scholes, “The pricing of options and corporate liabilities,” *Journal of Political Economy*, vol. 81, no. 3, pp. 637–654, May 1973.
- [10] J. C. Hull, *Options, Futures, and Other Derivatives*, 10th ed. Harlow, U.K.: Pearson, 2017.
- [11] cppreference.com, “C++ reference.” [Online]. Available: <https://en.cppreference.com>. [Accessed: 9 Jun. 2026].

- [12] W. Jakob, J. Rhineland, and D. Moldovan, “pybind11: seamless operability between C++11 and Python,” GitHub. [Online]. Available: <https://github.com/pybind/pybind11>. [Accessed: 9 Jun. 2026].
- [13] O. Cornut, “Dear ImGui: bloat-free immediate-mode GUI for C++,” GitHub. [Online]. Available: <https://github.com/ocornut/imgui>. [Accessed: 9 Jun. 2026].
- [14] GLFW, “GLFW documentation,” glfw.org. [Online]. Available: <https://www.glfw.org/docs/>. [Accessed: 9 Jun. 2026].
- [15] OpenSSL Project, “OpenSSL documentation,” openssl.org. [Online]. Available: <https://www.openssl.org/docs/>. [Accessed: 9 Jun. 2026].
- [16] Kitware, “CMake documentation,” cmake.org. [Online]. Available: <https://cmake.org/documentation/>. [Accessed: 9 Jun. 2026].
- [17] Binance, “Binance WebSocket market streams API documentation,” Binance. [Online]. Available: <https://binance-docs.github.io/apidocs/>. [Accessed: 9 Jun. 2026].

Appendix A

SOURCE CODE HIGHLIGHTS

This appendix collects key source-code excerpts referenced throughout the dissertation.

A.1 Order Type Enumeration

Listing A.1 shows the order-type enumeration that the matching engine routes on.

```
1 enum class Side : uint8_t { Buy = 0, Sell = 1 };
2
3 enum class OrdType : uint8_t {
4     Market    = 0,    // cross immediately at any price
5     Limit     = 1,    // rest until price acceptable
6     IOC       = 2,    // fill available, cancel remainder
7     FOK       = 3,    // full fill or reject
8     PostOnly  = 4,    // maker-only; reject if would cross
9     Stop      = 5,    // off-book until trigger
10    StopLimit = 6,    // off-book; becomes limit on trigger
11    Iceberg   = 7     // display clip, hidden reserve
12 };
13
14 enum class OrdStatus : uint8_t {
15     New, PartialFill, Filled, Cancelled, Rejected
16 };
```

Listing A.1. Order type and side enumerations

A.2 Black-Scholes Greeks

Listing A.2 shows the Black-Scholes Greeks computation used by the options market maker, implementing Equations 4.5.

```
1 Greeks bs_greeks(double S, double K, double T, double r,  
2                 double sigma, bool call) {  
3     double sqrtT = std::sqrt(T);  
4     double d1 = (std::log(S / K) + (r + 0.5 * sigma * sigma) * T)  
5                 / (sigma * sqrtT);  
6     double d2 = d1 - sigma * sqrtT;  
7     double Nd1 = norm_cdf(d1), Nd2 = norm_cdf(d2), nd1 = norm_pdf(d1)  
8     ;  
9     Greeks g;  
10    g.delta = call ? Nd1 : Nd1 - 1.0;  
11    g.gamma = nd1 / (S * sigma * sqrtT);  
12    g.vega = S * nd1 * sqrtT;  
13    g.price = call  
14        ? S * Nd1 - K * std::exp(-r * T) * Nd2  
15        : K * std::exp(-r * T) * norm_cdf(-d2) - S * norm_cdf(-d1);  
16    g.theta = -S * nd1 * sigma / (2.0 * sqrtT)  
17              - r * K * std::exp(-r * T) * (call ? Nd2 : -Nd2);  
18    return g;  
}
```

Listing A.2. Black-Scholes Greeks computation

A.3 Performance Metrics in Python

Listing A.3 shows the analytics routine that computes the risk-adjusted metrics from an equity curve.

```

1 def compute_metrics(equity, ann_factor=1.0, rf=0.0):
2     eq = np.asarray(equity, dtype=float)
3     rets = np.diff(eq) / eq[:-1]
4     mean, std = rets.mean(), rets.std()
5     down = rets[rets < rf].std()
6     peak = np.maximum.accumulate(eq)
7     dd = (eq - peak) / peak
8     af = np.sqrt(ann_factor)
9     wins, loss = rets[rets > 0].sum(), -rets[rets < 0].sum()
10    return {
11        'sharpe': (mean - rf) / std * af if std else 0.0,
12        'sortino': (mean - rf) / down * af if down else 0.0,
13        'max_dd': dd.min(),
14        'calmar': (eq[-1]/eq[0] - 1) / abs(dd.min()) if dd.min()
15                   else 0.0,
16        'volatility': std * af,
17        'profit_factor': wins / loss if loss else float('inf'),
18    }

```

Listing A.3. Risk-adjusted metric computation

Appendix B

SAMPLE OUTPUTS

This appendix presents representative outputs of the platform.

B.1 Benchmark Console Output

Listing B.1 shows a representative console output from the matching-engine benchmark harness.

```
1 === QuantSim Matching Engine Benchmark ===
2 Orders submitted : 10,000,000
3 Elapsed time      : 0.676 s
4 Throughput        : 14,792,899 ops/sec
5 Match latency     : p50 = 83 ns
6                   : p95 = 180 ns
7                   : p99 = 310 ns
8 Correctness       : 27/27 unit tests PASS
9 =====
```

Listing B.1. Matching engine benchmark output

B.2 Tear-Sheet Metrics Output

Listing B.2 shows the metrics block of an exported tear sheet for a market-making backtest.

```
1 {
2   "strategy": "avellaneda_stoikov",
3   "final_pnl": 7843.21,
4   "metrics": {
5     "sharpe": 3.12, "sortino": 4.05, "calmar": 4.33,
```

```

6     "max_drawdown": -0.018, "volatility": 0.024,
7     "profit_factor": 2.45, "win_rate": 0.61
8 },
9     "fills_total": 4821, "maker_ratio": 0.94,
10    "total_fees": 18.42, "total_rebates": 31.07
11 }

```

Listing B.2. Exported tear-sheet metrics (market making)

B.3 Order Book Depth Snapshot

Table B.1 shows a representative top-of-book depth snapshot as rendered in the Dashboard tab, with five price levels on each side.

Table B.1. Sample order book depth snapshot (five levels per side)

Bids		Asks	
Price	Size	Price	Size
100.02	180	100.04	150
100.01	240	100.05	210
100.00	310	100.06	275
99.99	160	100.07	190
99.98	220	100.08	240

The best bid of 100.02 and best ask of 100.04 imply a spread of two ticks and a mid-price of 100.03. The aggregate displayed depth and the per-level sizes are exactly the quantities consumed by the matching algorithm of Algorithm 1 when a marketable order sweeps the book.

B.4 Multi-Strategy Comparison

Figure B.1 shows the Compare tab, in which up to five strategy runs, drawn from the built-in library and from user-compiled `.dylib` strategies, are executed as a batch and their equity curves overlaid for side-by-side evaluation against the current run.

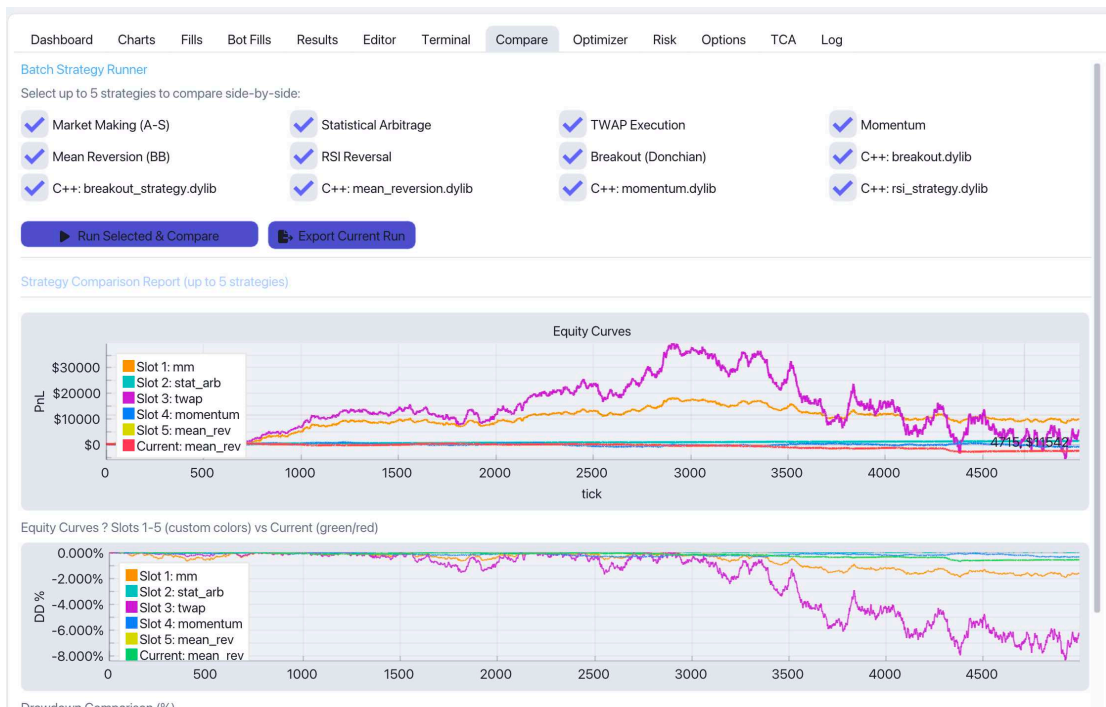


Figure B.1. Compare tab. The batch runner executes up to five selected strategies, both built-in and user-compiled, and overlays their equity and drawdown curves for direct visual comparison against the current run.

B.5 Parameter Sweep Optimizer

Figure B.2 shows the Optimizer tab performing a two-dimensional grid search over the Avellaneda-Stoikov risk-aversion (γ) and order-intensity (κ) parameters, with the objective surface (here the Sharpe ratio) rendered as a Viridis heatmap and the best-performing parameter pair highlighted for one-click application.

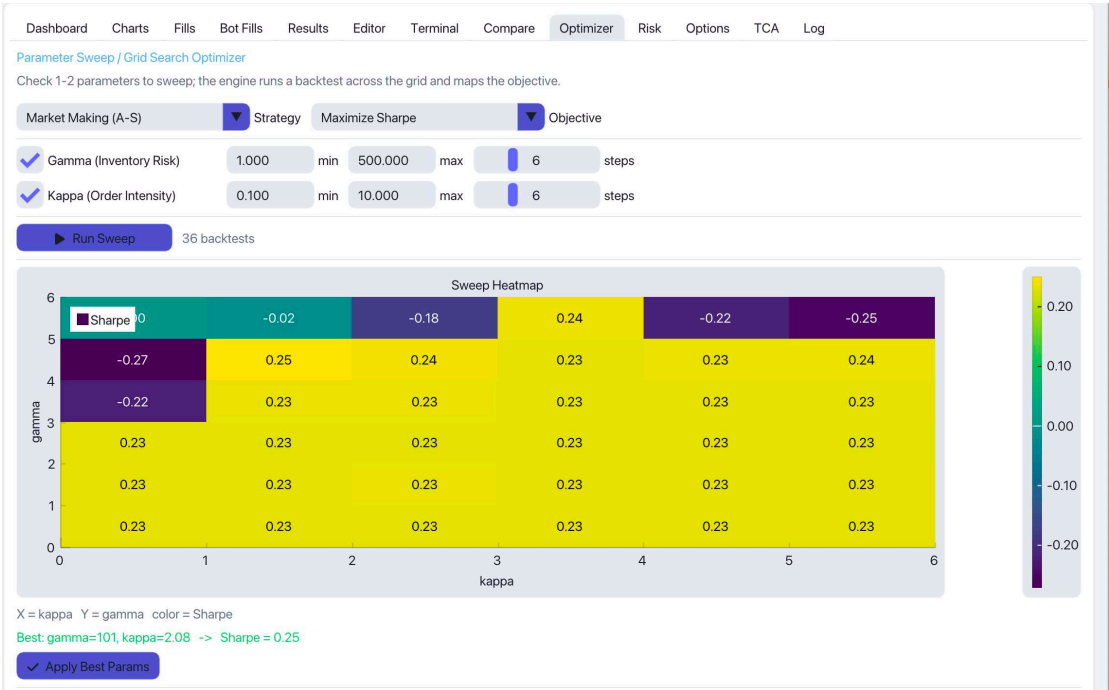


Figure B.2. Optimizer tab. A two-dimensional sweep over the market-making parameters γ (inventory risk) and κ (order intensity); each cell reports the resulting Sharpe ratio, and the “Apply Best Params” control loads the optimum back into the strategy configuration.